

Rust Forge

Welcome to the Rust Forge! Rust Forge serves as a n... documentation useful for members of [The Rust Pro](#) mistakes, typos, or want to add to the Rust Forge, fe [Rust Forge GitHub](#).

Help Wanted

Want to contribute to Rust, but don't know where to projects that have marked issues that need help and

Repository		D
rust		The Rust L
cargo		The Rust p
crates.io		Source co
www.rust-lang.org		The Rust v

Current Release Versions

Channel	Version	Will be stable on
Stable	1.85	February 20 2025 UTC
Beta	1.86	April 03 2025 UTC
Nightly	1.87	May 15 2025 UTC
Nightly +1	1.88	June 26 2025 UTC

See the [release process](#) documentation for details c up to a release.

No Tools Breakage Week

To ensure the beta release includes all the tools, no week before the beta cutoff (except for nightly-only

Beta Cut	No Break
1.87	March 26 2025
1.88	May 07 2025 →

External Links

- [Bibliography](#) of research papers and other pro
- [Rust Pontoon](#) is a translation management sys

Platforms

Rust uses a number of different platforms for organising communications between teams. This does not currently cover all platforms, rather documenting the policies for a select few platforms.

Twitter

The Rust project has a number of official Twitter accounts currently maintained by the infrastructure team.

- [@rustlang](#)
- [@RustStatus](#)
- [@cratesiostatus](#)
- [@rustasync](#)

Twitter Guidelines

The project runs the Twitter account [@rustlang](#). The account is managed by a team of volunteers.

The account will mostly tweet links to the Rust blog and Rust-related content. We will retweet:

- links to blog posts about Rust, retweeting the original
- questions about Rust, so all followers can help
- Meetup or conference announcements
- announcements of new Rust projects
- anything else relevant

We will not retweet:

- content that bashes other programming languages or is otherwise unconstructive in its discussion of language/technology
- Personal announcements ("Today I start my job")
- Learning Rust updates ("Today I started to learn Rust")

The Direct Messages are open to everyone. If someone wants to contact us, they should send the tweet via DM. The vast majority of tweets are about Rust, keeping it to the above rules. Requests of an author to remove something will be honored.

Additionally account handlers may look through the account for content.

The account will only follow a small number of Project Rust folks. At the time of writing (February 2022) this is only [@cra](#)

Access

Currently access to all four accounts is granted together. We should split this into more fine-grained access. Some automation should be given to accounts to automatically tweet about upcoming events.

Access is limited to a small set of folks in the [twitter](#) space. Changes should ping infra admins for provisioning.

People with access to 1password should:

- Never change the password or take other administrative actions (done by infra admins)
- Exclusively use the project-hosted instance to store the password. Do not save it to any other password database, including your personal one.
- Never share the password with others (even if you think you can trust them)
 - All access should always go through regular channels. Do not accidentally leaking the password by pasting it into an email (e.g., email)
- Be aware that the password may change regularly.

If you believe you should have access, please file a Request for Access (RFA) requesting it and note in the description that you've read and agree to the

Discord

Rust's Discord is currently used by a variety of team Documentation, as well as their working groups. It is a tool for Domain Working Groups, and provides a space for Rust users, contributors, and beginners.

Where to go for help with

Discord's support center provides documentation and [settings](#).

Getting started

1. Understand community standards

Discord, like all official Rust spaces, is governed by community standards. When joining the conversation there, you can prepare to read the [Moderation Guidelines](#). It is also useful to read

2. Access channels

To access the Rust Discord, visit <https://discord.com>. If you don't have a Discord account, you can register for one. Once you have access, your first action should be agreeing to the server rules in the instructions in #welcome.

3. Configure notifications

It is a good idea to disable notifications for channels that you will not be overwhelmed with messages. To do this, click on the server name banner (titled "The Rust Project") and turn off Notifications from the dropdown. Then follow the instructions provided [on the Discord Support site](#).

Appropriate conversation

Discussions should be related to the channel purpose. Off-topic discussions should be related to team business. All channels are for Rust-related topics related to the Rust project. Discussion of (for example

appropriate.

Channels

The following channels are relevant to newcomers to

- welcome - Where you agree to the CoC.
- rust-usage - This is a channel where you can ask Rust language use questions. The [Rust Users Forum](#)
- beginners - Here, you can meet people who be
- contribute - Interested in contributing to the Rust channel, you can subscribe to the [This Week In Rust](#) opportunities are regularly posted. It may also [teams](#).

Channels outside of General are for contributors to

Messages

Discord conversation takes place when people are aware and expect that your messages will receive a response or action in place. Depending on how your notifications are configured, a top of the Discord icon in your system tray when new messages to communicate with a specific individual, right-click the icon and select “Message” in the dropdown menu.

Read-only view

Set up a Discord account (as described in [Getting Started with Discord](#)). There is not currently a read-only archive view

Email

While most of Rust's discussion happens on other platforms, we occasionally need a way to approach individuals or groups. We do this through Mailgun (provided by Mozilla). We create and manage our lists through the [rust-lang/team](#) repository. Our email domain is `ferris@rust-lang.org`.

Sending a public broadcast

If your teams need to reach everyone in the Rust or `all@`. It is recommended that you only use this mailing list to contact every member, such as for organising a meeting or for security alerts.

Keeping responses private

When sending a message to `all@`, do not put `all@` in the `To` field. Replies to your broadcast will also be sent to everyone in the `To` field, and place `all@` in the `Bcc` field to keep your team.

GitHub

GitHub is where the Rust project hosts all of its code and discussions.

Organisations

- `rust-lang` — The Rust project organisation.
- `rust-embedded` — The Embedded Working Group
- `rustwasm` — The WebAssembly Working Group
- `rust-cli` — The Command Line Application Working Group
- `rust-secure-code` — The Secure Code Working Group
- `rust-gamedev` — The Game Development Working Group

rust-lang organization policy

The following is the policy for management of the `rust-lang` organization.

Access

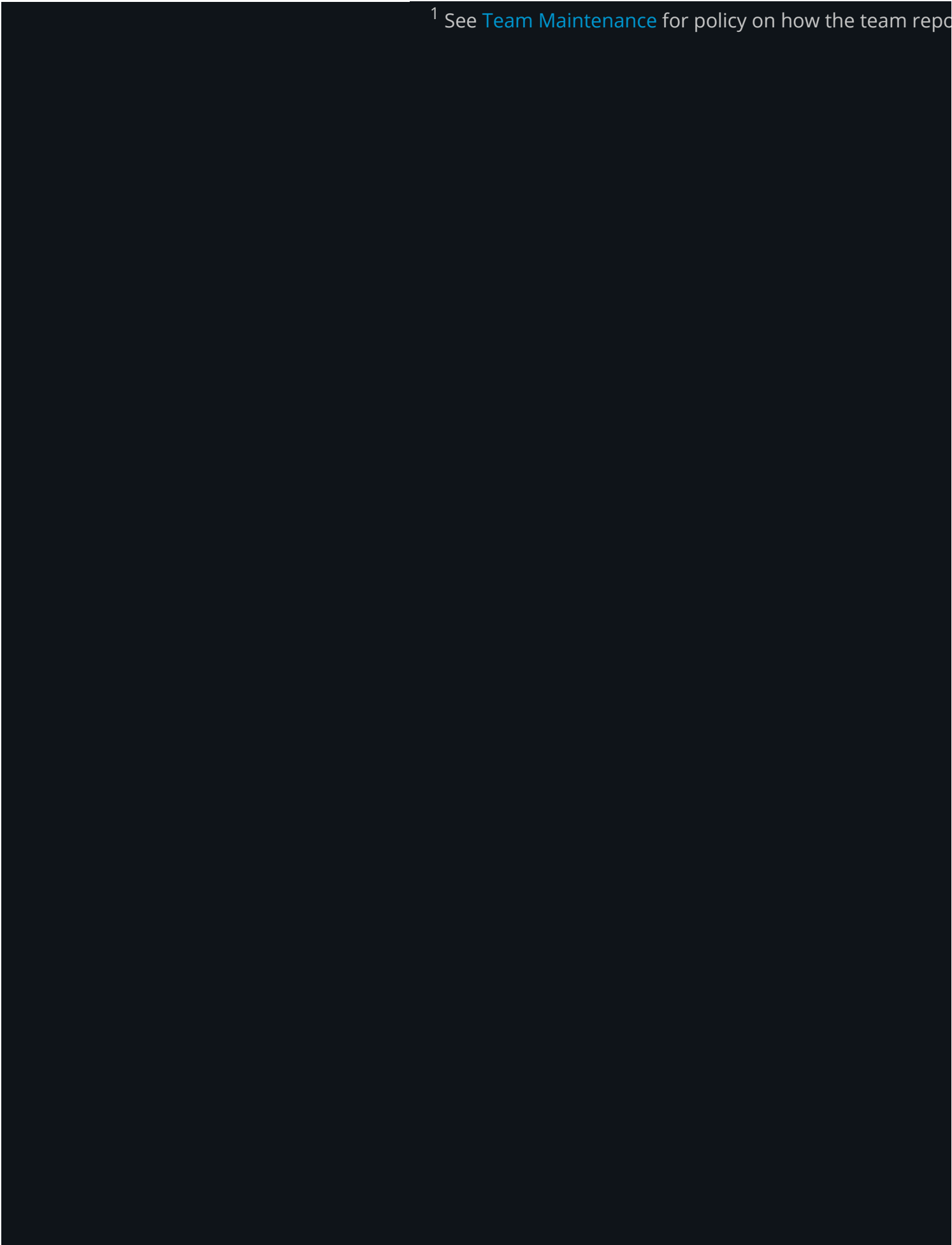
All access to the `rust-lang` GitHub organization is managed by the Infrastructure Team. Teams that want to assign access levels, or create new repositories, must submit a Request to that repository to request the change.

The `Infrastructure Team` is responsible for overall access management of the GitHub organization. Selected members of the Infrastructure Team are designated as repository owners if their work requires it.

All GitHub accounts used to interact with the Rust-Lang organization (owner or non-owner) must have `2FA` enabled. This is enforced by the Infrastructure Team.

Bot accounts controlled by the Infrastructure Team require at least any level of access required for them to work at the organization.

¹ See [Team Maintenance](#) for policy on how the team reports



Zulip

Rust's Zulip is used by a number of teams, notably the Rust team, along with their working groups.

Zulip can be an unintuitive platform to get started with, so we have a [getting started guide](#). For more detail, examine the [Zulip documentation](#).

Where to go for help with using Zulip

If you're testing a feature, or want to get help, the [#rust](#) channel is the best place to go. If you're elsewhere, the best thing to do is to create a new topic in the [#rust](#) channel.

Getting started

It is recommended to first look at the official [getting started guide](#) before diving in. It's a bit special and reading the documentation before you start is a good idea.

You'll definitely want to configure the streams that you want to subscribe to; the default set is quite limited, and there are many more streams available. Subscribing to a stream is very low cost – it is similar to the cost of a stream in other chat systems, but logs are available for all streams, regardless of whether you are subscribed to them.

It's not necessary to introduce yourself, but feel free to do so in the [#rust](#) channel.

User groups

User groups can be pinged by anyone with the [@<group>](#) syntax. Groups can be created by anyone, and can be used to discuss specific topics.

Users should feel free to join (or leave) groups on their own. You should feel free to create groups as needed, though it is currently rare. You should name your group similar to how you would name a channel.

purpose, though groups can be more fine-grained ([compiler/meeting](#) currently does not have a dedica

Appropriate conversation

In most streams, you should try to keep conversatio
[#general](#) stream is a bit broader, but even there, d
to Rust (though may not relate to projects of any pa
expected to be used for discussions related to the R
(for example) wildlife or sightseeing are not approp

Streams

These are similar to “channels” on other platforms (
On the other hand, you can choose which streams y
more than channels on other platforms. Read [Zulip](#)

Streams are appropriate for any Rust official group.
groups, teams are all examples of official groups. Th
represented in the [team repository](#).

Default streams

**This section is still under debate, and it is not yet
is non-normative, and should not be used yet fo
instance.**

The default set of streams is chosen to allow incomi
one place to go that can then, if necessary, direct th

Currently that means that every top-level group pre
Specifically, no stream that contains a [/](#) will be ena

Currently this set is:

- [general](#)

- t-lang
- t-compiler
- t-libs
- project-ffi-unwind
- project-inline-asm
- project-safe-transmute
- rust-survey-2019
- wg-async-foundations
- wg-database
- wg-formal-methods
- wg-secure-code
- wg-traits
- zulip

An alternative, minimalistic, approach is to use:

- general
- zulip
- announce
- new members

as the default set, which would push people into creating their own channels starting out.

Stream naming

A stream should be named such as `#t-{team}/{group}-topic`. More levels of nesting are allowed, but we don't want "subgroups" as well, though you may want to create them. Keeping the stream name short is good for usability. Streams which share the same prefix.

If no top-level team exists, or the group spans multiple teams, then the top level team should be omitted.

Streams should be clearly communicated as being for a specific purpose. It can be broad, but it should likely include a group of people (e.g., people who are having trouble with the compiler, or people working on the compiler). Furthermore, we do not have a general place for community projects not affiliated with the Rust project.

wish to use Zulip, it is [free for open source](#).

When a new stream is created, you should announce it. This is done automatically by Zulip.

Topics

A topic is attached to every message within a given stream (or within streams). Topics are generally transient, and are used for discussion on a topic. Thinking of topics like email subjects is helpful.

New conversation in a given stream should almost always start a preexisting one. Unlike (for example) GitHub issues, you should not start a past topic on the same subject. Do not spend too much time beyond trying to make it short. Topics should generally be short (loosely two to three words), to make sure it is visible in the stream.

You should eagerly fork new discussion topics into a new stream, done with the tail of another topic (if accidentally you forked a discussion).

To fork from an existing topic, see Zulip's documentation.

Messages

Zulip is a unique platform which combines synchronous and asynchronous communication in one location. You should not generally expect to receive a response quickly, and unlike (for example) GitHub issues, there is no reason to “re-ping” on a particular issue every few hours. Messages vanish into history, being isolated to a specific topic.

Linkifiers

Our Zulip supports a lot of helpful linkifiers, and we encourage you to request. See [the documentation](#) for the format. Properly formatted linkifiers are essential for the format. Properly formatted linkifiers are essential for the format.

Generally, `github-org/repo#123` works for linking to a few more “special cased” repositories.

Don’t forget that standard Markdown syntax for linking to issues is `#123`.

We support linking to issues on repositories inside the `rust-lang/` prefix. For example:

- `rust-lang/rfcs` with `RFC#3434` or `rfc#3434`
- `rust-lang/async-book` with `async-book#2334`
- `rust-lang/cargo` with `cargo#2334`

`rust-lang/rust` issues can be linked without needing a prefix:

- `rust-lang/rust` with `#4545` or `rust#4545`

We currently support linking to commits on these repositories:

- `rust-lang/rust` with 40-character long SHAs, e.g. `25434f898b499876203a3b95c1b38bad5ed2cc5d`

Read-only view

Our Zulip instance has the web-public streams beta enabled for public streams. Please let us or Zulip developers know if you have any feedback. The previous solution to the web-public view was to use the `web-public` view to the web public view.

Zulip Moderation

Zulip, like all official Rust spaces, is governed by the moderation team. If you have any concerns, please feel free to escalate to the moderators.

However, though the moderation team is the top-level authority, there is a channel where you can seek help with moderation within Zulip.

One method for reaching the Zulip administrators is to ping the `admin.239bd484c0347d2d43214d8581f3e125.show-secrets` stream. See [this page](#) for details on how this works.

You can also ping the `@mods` group on Zulip; note that this is not a stream.

It is not currently possible for normal users to self-administrate (i.e. become a moderator or admin user). However, each individual stream, including private streams, has its own moderation settings.

For admins/moderators

Some common actions for moderators are listed on the [Zulip moderation page](#).

Notably,

- in “[Organization permissions](#)” we can restrict user permissions, including disabling the “Join” button (this is the “no new users” button)

New admins/moderators should add themselves to the `@admins` group. Note that this is something that any user can do!

Rust Blog Guidelines

Context

The Rust project maintains two blogs. The “main blog” (blog.rust-lang.org/inside-rust). This document takes to write a post for each of those blogs, as well as choose which blog is most appropriate.

How to select the right blog: audience

So you want to write a Rust blog post, and you’d like to get it on. Ultimately, there are three options:

- The main Rust blog
 - Suitable when your audience is “all Rust developers”
 - Written from an “official position”, even if it’s just a team member
- The team Rust blog
 - Suitable when your audience is “all Rust developers”
 - Written from an “official position”, even if it’s just a team member
- Your own personal blog
 - Everything else

There are two key questions to answer in deciding where to post:

- Are you speaking in an “official capacity” or as a private citizen?
- Who is the audience for your post?

In general, if you are speaking as a “private citizen”, you should post on your own personal blog.

If, however, you are writing in an **official capacity**, it’s a good fit. Note that this doesn’t mean you can’t write on Rust’s blog are signed by individuals, and, in fact, many of the posts on Rust’s blog are signed by individuals, and, in fact, many of the posts on Rust’s blog are signed by individuals, and, in fact, many of the posts on Rust’s blog are signed by individuals. However, those posts are typically documenting the progress of the Rust project. An example is Aaron Turon’s classic post on [Rust’s lang](#)

Sometimes, the posts are describing an exciting project that represents the project as a whole (e.g., [Manish Gore's Concurrency in Firefox Quantum](#)).

To decide between the main blog and the team blog, **is the audience** for your post. Posts on the main blog are for Rust or potential users — they tend to be lighter on technical details, requiring as much context. Posts on the team blog are for Rust familiarity with Rust.

Writing for the Main Rust blog

The Leadership Council ultimately decides what to post on the main blog.

Post proposals describing exciting developments from the Rust project as well as posts that describe exciting applications of Rust are “promotional cross-posting” with other projects, however they are not.

If you would like to propose a blog post for the main Rust blog, you should be a [Leadership Council member](#). It is not suggested to join the main Rust blog that add posts without first discussing it with a Leadership Council member.

Release note blog posts

One special case are the regular release note posts. These are managed by the release team and go on the main Rust blog.

The blog posts are published on the same day as the release. The release team running the release. Releases always have a blog post.

Before publishing a release post, it goes through a checklist:

1. The milestone (e.g. for [1.39.0](#)) for the release is identified.
2. PRs that we think are sufficiently important are identified and headlined. The writing of a blog post typically happens in the document.
3. Headlined items are sometimes written by different people. The release team review each subsection.

4. The blog post draft is submitted as a PR on the repository before the release.

Team Rust blogs

Teams can generally decide for themselves what to blog about.

Typical subjects for team Rust blog posts include:

- New initiatives and calls for participation
- Updates and status reports from ongoing work
- Design notes

To propose a blog post for the team blog of a particular team, contact the team lead or some other team representative.

Calendars

Many Rust teams and working groups have regular meetings and events. It's often difficult to quickly to manage all the calendar events.

That's why we have automation available for generating calendar events. It can be found in the [calendar](#) repository for its usage.

You can use it to create and update calendar invites and the tool will then generate `.ics` files from them, which can be used with various calendar tools.

Triagebot

Triagebot (AKA rustbot) is a general-purpose bot used by the rust-lang organization, usually involving sending comments on GitHub issues or pull requests. The following pages explain the available commands.

Commands are usually issued by writing comments on GitHub issues or pull requests. Which commands that are available depends on which repository you are in. A repository has a `triagebot.toml` where you can see which features are enabled.

For example, the following comment:

```
@rustbot label A-diagnostics A-macros
```

will set the given labels on a GitHub issue or pull request. You need repository direct permissions to do that in the GitHub UI.

GitHub commands

Commands on GitHub issues or pull requests are usually entered in a single comment. They can be followed by the command anywhere in the comment, including in code blocks, markdown code blocks, inline code spans, HTML blocks, etc. Commands can be entered in a single comment.

Triagebot also allows editing of a comment. If you delete a comment, then triagebot will ignore the edit. However, if you make a new comment, then those commands will be processed.

Configuration

Individual GitHub repositories can configure triagebot by having a `triagebot.toml` in the root of the default branch. The configuration file lists the commands needed for each feature.

For example, the `rust-lang/rust` configuration file is located at `lang/rust/blob/master/triagebot.toml`.

When first adding `triagebot.toml` to a new repository, you need to set permissions for the bot to operate. This can be done by adding the `lang/team` database to add `bots = ["rustbot"]` to the `lang` directory.

Common command summary

The following are some common commands you may use:

Command	Description
<code>@rustbot claim</code>	Assigns an issue to the user.
<code>@rustbot release-assignment</code>	Removes your assignment from an issue.
<code>@rustbot assign @octocat</code>	Assigns an issue to a user.
<code>@rustbot ready</code>	Indicates a PR is ready for review.
<code>@rustbot author</code>	Indicates a PR is assigned to the author.
<code>@rustbot blocked</code>	Indicates a PR is blocked by something.
<code>@rustbot label A-diagnostics A-macros</code>	Adds two labels to a PR.
<code>@rustbot label -P-high</code>	Removes a label from a PR.
<code>@rustbot ping windows</code>	Posts a comment to the Windows ping group.
<code>@rustbot prioritize</code>	Requests prioritization of a PR.
<code>r? @octocat</code>	Assigns a PR to a user.
<code>r? libs</code>	Assigns a PR to a random member of the libs review group.
	Assigns a random member of the libs review group.

`r? rust-lang/cargo`

the cargo team.

The following are some common commands you m

Command	Description
<code>@triagebot read</code>	Waits for people to read a in a meeting.
<code>@triagebot end-topic</code>	Checks if everyone is done a topic in a meeting.
<code>@triagebot end-meeting</code>	Checks if everyone is ready meeting.

Implementation

The source code for triagebot can be found at [https](https://github.com/rust-lang/cargo) you are interested in extending triagebot, the docum guidance on how to get started.

Agenda Generator

The lang team uses the agenda generator to assist v

Usage

The agenda generator can be viewed at <https://triag>

Configuration

This feature has no configuration.

Implementation

See `src/agenda.rs`.

Issue Assignment

The issue assignment commands allows any user to

Usage

Issue assignment is done by entering one of these c

- `@rustbot claim` — Assigns the issue to yours
- `@rustbot release-assignment` — Removes th
assignee or a team member can release an as
- `@rustbot assign @user` — Assigns a specific
other users.

Due to GitHub restrictions, not all users can be dire
with write permission to the repo, or rust-lang organ
assigned. If triagebot is unable to directly assign the
and edit the top-level comment with a message tha

Configuration

Issue assignment is enabled on a repository by the
`triagebot.toml`:

```
[assign]
```

Implementation

See `parser/src/command/assign.rs` and `src/hand`

PR Assignment

Triagebot handles automatic and manual assignment and warmly welcoming new users when they post a PR.

Rust contributors can track and manage their own view of PR integration. See [Tracking PR assignment](#).

Usage

Automatic assignment of new PRs is handled by the `triagebot.toml`, described [below](#).

Manual assignment can be done by posting a comment with the following commands:

- `r? @octocat` — Assigns a specific user.
- `r? octocat` — The `@` is optional.
- `r? libs` — Chooses a random person from the `triagebot.toml`. For example, for the `rust-lang/rust` repo, `triagebot.toml` for a list of ad-hoc group names.
- `r? rust-lang/libs` — The `rust-lang/` org name is required.
- `r? rustdoc` — Chooses a random person from the `rust-lang/rustdoc` database for a list of team names.
- `r? rust-lang/rustdoc` — The org name prefix is required. It is recommended that you do not use `@`, as that will assign the team to the PR.

When choosing a user from a team, triagebot only looks at the first team (it ignores subteams).

When looking up a name, triagebot will first look at the `triagebot.toml` and if it doesn't match either of those it assumes it is a team name.

PRs can only be assigned to users with write permissions, or team members with read permissions, or anyone who has been assigned to the PR.

Ghost

Using `r? ghost` in the initial PR top-level comment triagebot's auto-assignment. `ghost` is GitHub's placeholder. It is used here for convenience. This is typically used when you don't want any assignments or noise.

Configuration

PR assignment is enabled on the repository by having a `triagebot.toml`:

`triagebot.toml`:

```
# These are ad-hoc groups that can be referenced in the table below.
# The values may contain GitHub usernames, or group names.
# The `@` is optional.
# Group names should be lowercase.
[assign.adhoc_groups]
libs = ["@joshtriplett", "@Mark-Simulacrum", "@thomcc"]
# Can reference other groups.
compiler = ["compiler-team", "compiler-team-contributors"]
compiler-team = ["cjgillot", "estebank"]
compiler-team-contributors = ["compiler-error-handlers"]
# Can reference rust-lang teams.
libs = ["rust-lang/libs-api"]
# This is a special group that will be used if no other group matches.
fallback = ["@Mark-Simulacrum"]

# This specifies users, groups, or teams to assign to.
# Triagebot will pick one person to assign.
# Paths are gitignore-style matches.
[assign.owners]
# Examples of assigning individuals.
"Cargo.lock" = ["@Mark-Simulacrum"]
"/library/std/src/sys/windows" = ["@ChrisDenton"]
# Example of assigning to a group.
"/library/std" = ["libs"]
# Supports gitignore patterns.
"*.*" = ["@octocat"]
# If you want to match all files, `*` should be used.
"*" = ["@octocat"]
# Can use teams from the rust-lang teams data.
"/src/tools/cargo" = ["@rust-lang/cargo"]
```

If the `owners` map is configured, then triagebot will post a comment on which files were modified in the PR. The existence of this map gives the ability for users to post a comment with `r? name` to assign a user.

Vacation

If a reviewer wants to temporarily prevent themselves from reviewing (e.g. on vacation or manually) they can add themselves to the special

```
[assign]
users_on_vacation = ["jyn514", "ChrisDenton"]
```

Additional new PR trigger options

Triagebot will also post a welcome message to the user if certain factors are met:

- PR authors who have not previously made any commits will get a welcome message.
- PR authors who have made commits will get a welcome message.
- If the initial PR comment has an `r?` command.

There are several options in `triagebot.toml` for configuring these options:

```
[assign]
# If set, posts a warning message if the PR is opened on a non-default
# branch (usually main or master).
warn_non_default_branch = true
# If set, the welcome message to new contributors will include
# a contributing guide.
contributing_url = "https://rustc-dev-guide.rust-lang.org/contributing.html"
```

Additionally, triagebot will post a comment with a warning message if the PR contains submodules.

Exceptions to default branch warning

Some PRs may have a different default branch than the repository.

possible to add exceptions based on the PR title, when targeting a different branch than specified.

```
[assign]
warn_non_default_branch.enable = true

[[assign.warn_non_default_branch.exceptions]]
title = "[beta" # title contains "[beta" in
branch = "beta"
```

Implementation

See `parser/src/command/assign.rs` and `src/hand`

Tracking PR assignment

If you contribute in some capacity to the Rust comp assigned pull requests to be reviewed.

You can check your current review assignment in tw

- by visiting this [GitHub URL](#)
- by interacting with the `triagebot` on the [Zulip](#) thread. You can open a direct message session [link](#) (requires Zulip login).

This chapter will describe how to interact with the `t`

Configuration

Tracking the PR assignment is enabled on the git rep table in `triagebot.toml`. No additional configurati

Usage

Open a Direct Message session with the `triagebot` these commands:

- `work` — Will emit an error and show the availa
- `work show` — Will show your Github usernam to you for review (on the `rust-lang/rust` git r

Implementation

See `parser/src/handlers/pr_tracking.rs`.

Autolabels

Auto labels will automatically apply labels to GitHub issues based on the `[autolabel]` configuration in `triagebot.toml`.

Usage

Auto labels have no manual control. See [labeling](#) for more information.

Configuration

Triggered by labels

Labels can be added when another label is added. The `trigger_labels` field specifies which labels will cause this to trigger.

```
# Automatically applies the `I-prioritize` label to issues where the label
# listed below is added to an issue (unless the issue already has the
# labels listed in `exclude_labels`).
[autolabel."I-prioritize"]
trigger_labels = [
    "regression-untriaged",
    "regression-from-stable-to-stable",
    "regression-from-stable-to-beta",
    "regression-from-stable-to-nightly",
    "I-unsound",
]
exclude_labels = [
    "P-*",
    "T-infra",
    "T-release",
    "requires-nightly",
]
```

Exclude labels support shell-like `*` glob patterns.

Triggered by files

Labels can be added based on which files are modified. The `trigger_files` config option specifies which files will cause the label to be added with `starts_with`.

```
# Adds the `T-compiler` label to any PR that
# `src/test/ui` unless it already has a `T-*`
[autolabel."T-compiler"]
trigger_files = [
    "compiler",
    "tests/ui",
]
exclude_labels = [
    "T-*",
]
```

Triggered by new PRs

Labels can be added to any PR when it is opened. See [this](#) to enable this. For example:

```
[autolabel."S-waiting-on-review"]
new_pr = true
```

Triggered by new issues

Labels can be added to any issue when it is opened. See [this](#) to enable this. For example:

```
[autolabel."new-issue"]
new_issue = true
```

Implementation

See [src/handlers/autolabel.rs](#).

Close

The close command can be used to close a GitHub i

Usage

To close an issue or pull request, any rust-lang team

```
@rustbot close
```

This will immediately close the issue or PR.

Configuration

This feature is enabled on a repository by having a

```
[close]
```

Implementation

See `src/handlers/close.rs` and `parser/src/comm`

Documentation Updates

Triagebot automatically generates a PR to `rust-lang` all of the book submodules. This PR requires manual currently managed by `@ehuss`.

Usage

There are no settings or manual controls for this feature.

Implementation

See `src/handlers/docs_update.rs`.

GitHub Releases

Triagebot can be used to automatically create releases using the relevant section of the changelog as the release notes when doing this.

Usage

Any time you push a git tag, or update the contents of the changelog, triagebot will synchronize *all* tags with the releases. That is, any tag will create a new release. Additionally, the text of all the releases will be the text in the changelog.

Tags that don't have entries in the changelog will not be updated.

Configuration

To enable automatically creating GitHub Releases, add the following to the root of your repository:

```
[github-releases]
format = "rustc"
project-name = "Rust"
changelog-path = "RELEASES.md"
changelog-branch = "master"
```

The `format` defines which format the changelog file uses to extract the relevant section from it. You can add and remove entries in `src/changelogs/`. The currently supported formats are:

- `rustc`: follows the custom style of rustc's [RELEASES.md](#).

The `project-name` defines what the title of the release will be: `{project-name} {tag}`.

The `changelog-path` and `changelog-branch` keys control

when searching for the changelog.

Implementation

See `src/handlers/github_releases.rs` and `src/c`

Glacier

Triagebot can be used to automatically generate PRs on [lang/glacier/](#) that contain code snippets that cause a

Usage

Enter the code you want to post on the [Rust Playground](#), then copy the link for “Direct link to the gist”. Then post a comment with that link as:

```
@rustbot glacier "https://gist.github.com/rust-play/3d9134282f880c93bfe65e7db6b0680f"
```

Note that the link must be in double quotes.

Configuration

This feature is enabled on a repository by having a

```
[glacier]
```

Implementation

See [parser/src/command/glacier.rs](#) and [src/hand](#)

Issue Transfer

The `transfer` command allows you to transfer a GitHub issue to another repository.

Usage

To transfer an issue to another repository, enter a comment with the following command:

```
@rustbot transfer <repository-name>
```

It is recommended to also include a comment explaining the transfer. For example:

```
Transferring to rust-lang/cargo since this issue implements diagnostic reports.
```

```
@rustbot transfer cargo
```

IMPORTANT: There will be no visual indication that an issue has been transferred. Due to GitHub API limitations, you will not see any activity on the issue or view the issue in its new location. It may take a few minutes for the data to sync.

WARNING: Transferring is a partially destructive command. It will delete milestones that don't exist in the target repository and will not transfer comments.

The transfer command is limited to team members of the rust-lang org. Only transfers will happen to repositories in the rust-lang org. Also, the target repository must have triagebot enabled on it.

Configuration

The source repository must have an empty `transfer` command in its repository. Issues can be transferred to any repository that has triagebot enabled).

[transfer]

Implementation

See `parser/src/command/transfer.rs` and `src/ha`

Labeling

You can apply GitHub labels to an issue or PR by posting a comment. Labels can be very helpful for searching, tying issues together, and indicating a formal way, such as the status.

The Triage WG helps with labeling issues. If you are new to the process, see the [Triage WG procedure](#).

Usage

The general form of the comment should be `@rustbot label` followed by a separated list of labels to add or remove. You can remove labels using the `-` character. Some examples:

- `@rustbot label A-diagnostics A-macros`
- `@rustbot label +T-lang -T-compiler` — Remove `T-compiler`

The syntax for the command is somewhat flexible, so you can use whatever suits your pleasure. Some examples of variants you can use:

- `@rustbot label: +T-lang, -T-compiler`
- `@rustbot label: +T-lang and -T-compiler`
- `@rustbot modify labels to +T-lang and -T-compiler`
- `@rustbot modify labels: +T-lang and -T-compiler`
- `@rustbot modify labels to +T-lang -T-compiler`

The command can be terminated with a `.`, `;`, or the end of the line.

Formally the grammar is:

Command → `@rustbot` `modify` *? label-word* `to` *? label-list*

label-word →

`label`
| `labels`

label-list →


```

label-delta
| label-delta and label-list
| label-delta , label-list
| label-delta , and label-list

```

```
label-delta →
```

```

+ label
| - label
| label

```

```
label → [^.,!?:\n() ]+
```

Permissions

All labels can be assigned by rust-lang organization (prioritization, and wg-async). Users not on a team can be authorized in `triagebot.toml`. It is encouraged for labels to be applied by anyone. An example of one is `accepted`, since accepting a backport to beta is usual.

Configuration

Labeling support is enabled on a repo by having a `label` section in `triagebot.toml`.

```
[relabel]
```

Permissions for allowing unauthenticated labeling is controlled by the `allow-unauthenticated` list:

```
[relabel]  
# any label is allowed to be set by team mem  
lang/team)  
# but these can be set by anyone in the worl  
allow-unauthenticated = [  
    "C-*", # any C- prefixed label will be a  
    authorization with rust-lang/team  
    "!C-bug", # but not C-bug (order does no  
]
```

Implementation

See `src/handlers/autolabel.rs`.

Major Changes

Triagebot helps with automated processing of [Major](#)

Usage

The process starts when the appropriate label is set. The [lang/compiler-team](#) repo has a [major change template](#) with a `major-change` label. Triagebot will detect this and create a Zulip discussion, and post a comment to the issue with a link to the discussion.

If a team member writes a comment on the GitHub issue with `@rustbot seconded`, then triagebot will set the appropriate label and post a comment to Zulip.

If a team member adds the `major-change-accepted` label to the GitHub issue, then triagebot will post a comment to Zulip to let people know that it has been accepted.

Configuration

This feature is enabled by the `[major-change]` table in the `config.toml` file.

```
[major-change]
# Issues that have this label will start the
# Defaults to "major-change".
enabling_label = "major-change"

# Label to apply once an MCP is seconded.
second_label = "final-comment-period"

# Label to apply when an MCP is created.
# Typically this is used to track what needs
meeting_label = "to-announce"

# When this label is added to an issue, that
proposal
# which sends an update to Zulip.
# Defaults to "major-change-accepted".
accept_label = "major-change-accepted"

# Optional extra text that is included in the
is opened.
open_extra_text = "cc @rust-lang/compiler @rust-lang/infra"

# The Zulip stream to automatically create to
# Can be found by looking for the first number
# https://rust-lang.zulipchat.com/#narrow/stream/233931
zulip_stream = 233931

# An Zulip group or username to tag in the Zulip
# proposal has been seconded.
zulip_ping = "T-compiler"
```

Implementation

See `src/handlers/major_change.rs`.

Mentions

Triagebot can leave a comment on PRs that touch code of people who want to review any change to those files with a message to the author.

Usage

Mentions are triggered automatically when a PR is opened based on the configuration in `triagebot.toml` of the repository.

Configuration

To enable mentions, add entries to the `[mentions]` table in `triagebot.toml`. The value in the table should be a path in the repo. Triagebot will trigger a comment on PRs that **starts with** the given path. For example, `library/std` will trigger a comment on PRs that touch the `library/std` directory like `library/std/src/path.rs`.

There are two optional values that can be specified for each entry:

- `cc` — A list of strings of users to ping. They should be in the form `@rust-lang/clippy`. If this is not specified, no users will be pinged.
- `message` — This is the message that will be included in the comment. If not specified, the comment will say `Some changes were made to the`.

Example:

```
[mentions."src/tools/cargo"]
cc = ["@ehuss"]

[mentions."src/rustdoc-json-types"]
message = """
rustdoc-json-types is a public (although it's not yet released).
If possible, consider changing `src/librustdoc-json-types` to `src/librustdoc-json-types`
otherwise, make sure you bump the `FORMAT_VERSION` in `src/librustdoc-json-types`
"""
```

Implementation

See `parser/src/mentions.rs` and `src/handlers/m`

Merge Conflicts

The `merge-conflicts` feature detects if a Pull Request has a comment asking the author to resolve the conflict.

Usage

This is triggered automatically when a commit is merged into open PRs to have a merge conflict. The bot will post a comment that looks like this:



The latest upstream changes (possibly #152) are unmergeable. Please [resolve the merge conflicts](#).

Note that it may take a minute or so for the comment to appear.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

```
[merge-conflicts]
```

There are several optional keys that you can include:

- `remove` — A list of labels to remove from the PR when a merge conflict occurs.
- `add` — A list of labels to add to the PR when a merge conflict occurs.
- `unless` — A list of labels that, if already present, prevent adding or removing labels.

Example

```
[merge-conflicts]
remove = ['S-waiting-on-bors']
add = ['S-waiting-on-author']
unless = ['S-blocked', 'S-waiting-on-crater',
waiting-on-review']
```

Implementation

See src/handlers/merge_conflicts.rs.

No Merge Policy

The [no-merge policy](#) informs users if they have merged a PR. Some repositories prefer to only use a rebase-oriented workflow.

Usage

This is triggered automatically if a PR has merge commits. The comment on the PR if it detects merge commits. The comment also explains how the user can avoid merge commits.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

```
[no-merges]
```

There are three optional values that can be specified:

- `exclude_titles` — A list of strings of title segments. Any PR containing these substrings will not be checked for merge commits.
- `labels` — A list of strings of label names to add when merge commits are detected.
- `message` — Override the default message posted. The default message will always be followed up with “The following merge commits were detected:” then a list of the merge commits.

Default message

There are merge commits (commits with multiple parents). Please use a [no merge policy](#) so these commits will need to be rebased.

be merged.

You can start a rebase with the following comma

```
$ # rebase
$ git pull --rebase https://github.com/rus
$ git push --force-with-lease
```

Example

```
[no-merges]
# PRs with the following labels will be skip
exclude_labels = ["rollup", "sync"]
# Add the following labels to PRs with merge
labels = ["has-merge-commits", "S-waiting-on-
# Post the following warning message as a co
message = ""
This repository does not allow merge commits
Your PR cannot be merged until it is rebased
""
```

Implementation

See `src/handlers/no_merges.rs`.

Nominate

The nominate commands are used for nominating i

Usage

There are multiple commands that can be issued in nomination:

- `@rustbot beta-nominate <team>` — Adds the team's label. This indicates that the issue is no team should decide whether to accept or reject
- `@rustbot nominate <team>` — Adds the `I-no`. This is used to nominate an issue for the team
- `@rustbot beta-accept` — Adds the `beta-acc` been approved for beta backport, and someone care of applying the backport.
 - `@rustbot beta-approve` — An alias for

Only rust-lang team members may use the nominat

Only teams that are listed in the [configuration](#) can b

If you need to nominate multiple teams, add each o encourage descriptions of what to do targeted at ea summary.

Configuration

This feature is enabled on a repository by having a The `nominate.teams` table lists the team names, and used for that team.

```
[nominate.teams]  
compiler = "T-compiler"  
release = "T-release"  
core = "T-core"  
infra = "T-infra"
```

Implementation

See [src/handlers/nominate.rs](#) and [parser/src/c](#)

Note

The `note` command can be used to update the top summary.

Usage

A summary note can be added to a GitHub issue by command:

```
@rustbot note summary-title
```

The word after `note` is then added as a link to the t

```
<!-- TRIAGEBOT_SUMMARY_START -->

### Summary Notes

- ["summary-title" by @username](link-to-comment)

Generated by triagebot, see [help](https://github.com/rust-lang/triagebot/wiki/Note) for how to add more
<!-- TRIAGEBOT_SUMMARY_END -->
```

with a link to the comment where you posted the `n`

The title word can be a sequence of characters matching `[^.,:!?;\n()] +`. Or it can be a quoted string like

Additional notes will get appended to the list:

```
<!-- TRIAGEBOT_SUMMARY_START -->

### Summary Notes

- ["first-note" by @username](link-to-comment)
- ["second-note" by @username](link-to-comment)
- ["summary-title" by @username](link-to-comment)

<!-- TRIAGEBOT_SUMMARY_END -->
```

This summary section should **not** be edited by hand

Removing an existing summary

Notes can be removed by writing a comment with `@
title`, where `summary-title` is the word used when
remove the entry from the summary list.

Configuration

This feature is enabled by having a `[note]` table in

```
[note]
```

Implementation

See `parser/src/command/note.rs` and `src/handle`

Notifications

The notifications system helps a user keep track of

Usage

Each registered team member has a notifications page

`https://triage.rust-lang.org/notifications?user=username`

Whenever you are mentioned on GitHub with a direct mention (`@rust-lang/libs`) anywhere in the rust-lang.org repository, you will be added to the notifications list.

The notifications list can also be edited via Zulip by any organization member. Any organization member can edit their notifications page or the notifications page of other organization team members. To do so, the editor must edit the `people/username.toml` file in the [team repository](#). When talking to it for the first time; please `mark` it.

The following commands are supported:

- `acknowledge <url>` (or short form `ack <url>`)
- `acknowledge <idx>` (or short form `ack <idx>`)

These both acknowledge (and remove) a notification

- `acknowledge all` or `acknowledge *` (or short form `ack all` or `ack *`)

This acknowledges and removes all notifications.

- `add <url> <description...>` (multiple words)

This adds a new notification to the list.

- `move <from> <to>`

This moves the notification at index `from` to the index `to`.

- `meta <idx> <metadata...>`

This adds some text as a sub-bullet to the notification if the text is removed.

- `as <github username> <command...>`

This executes any of the above commands as if you

Configuration

There is no configuration for this feature.

Implementation

See `src/handlers/notification.rs`, `src/notification.rs`, and `src/db/notifications.rs`.

Pinging

Triagebot can be used to “ping” teams of people that belong to certain teams. This is useful because sometimes we want to notify people but we don’t want to add all the members in the notification (that would imply that they are members of the Rust team, which would decorate their names with “member” and so forth). Triagebot can reach the [notification groups](#).

When a team is pinged, we will both post a message to the repository and a message will include a `cc` line that `@`-mentions all the members of the ping group.

Usage

On repositories with a ping group configured, any Rust team (e.g. prioritization, and wg-async) can write a GitHub comment like:

```
@rustbot ping windows
```

which would cause triagebot to post a comment notifying the members of the ping group.

Teams that can be pinged

To be pinged, teams have to be created in the [Rust team list](#). Teams that will be marked as `marker-team`, meaning that they can be pinged. The [Icebreakers LLVM team](#) is an example.

Additionally, the team needs to be configured in the `triagebot.toml` file.

Configuration

To enable the team (e.g. `TeamName`) to be pinged, you need to add it to the `triagebot.toml` file at the root of a repository, like

```
[ping.TeamName]
message = """\
Put your message here. It will be added as a
so it can include Markdown and other markup.
"""\
label = "help wanted"
```

This configuration would post the given message and label to the issue.

You can also define aliases to add additional labels to the message. This can be useful to add mnemonic labels or accommodate different naming conventions (e.g. “llvms” instead “llvm”), see the following example:

```
[ping.cleanup-crew]
alias = ["cleanup", "cleanups", "shrink", "release"]
message = """\
message content...
"""\
```

This will allow the command `@rustbot ping cleanup` to work as well as the aliased variants, ex.:

```
@rustbot ping cleanup
@rustbot ping shrink
...
```

Check out [the rust-lang/rust configuration](#) for an up-to-date example.

Implementation

See [parser/src/command/ping.rs](#) and [src/handle.rs](#) for the implementation.

Rendered link

Rendered links are simple hyperlinks that are automatically generated by triagebot.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

```
[rendered-link]
trigger-files = ["posts/"]
```

The `trigger-files` key configures which directories trigger the “Rendered link” pointing to the first file matching the pattern.

Implementation

See `src/handlers/rendered_link.rs`.

Requesting Prioritization

Users can request an issue to be prioritized by the P

Usage

On repositories configured for prioritization, any us

```
@rustbot prioritize
```

which will add the `I-prioritize` label to the issue t
issue needs prioritization.

Configuration

This feature is enabled on a repository by the `[prio`

```
[prioritize]  
# Name of the label used for requesting prio  
label = "I-prioritize"
```

Implementation

See `parser/src/command/prioritize.rs` and `src/`

Review Changes Requested

This feature will automatically adjust the labels on a review with changes requested.

Usage

When creating a pull request review, click the “Request changes” button. This will automatically remove the review labels and add a label to indicate that the PR is waiting on the author.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

```
triagebot.toml:
```

```
[review-submitted]
# These labels are removed when a review is submitted
review_labels = ["S-waiting-on-review"]
# This label is added when a review is submitted
reviewed_label = "S-waiting-on-author"
```

Implementation

See `src/handlers/review_submitted.rs`.

Review Requested

This feature will automatically adjust the labels on a request when it requests a review from an assignee.

Usage

In the list of reviewers, click the “Re-request review” button. This will automatically remove the “waiting on the author” label and add the “waiting on review” label to indicate that the PR is waiting on the review.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

`triagebot.toml`:

```
[review-requested]
# Those labels are removed when PR author requests a review
remove_labels = ["S-waiting-on-author"]
# Those labels are added when PR author requests a review
add_labels = ["S-waiting-on-review"]
```

Implementation

See `src/handlers/review_requested.rs`.

Rustc Commit Tracking

Triagebot keeps a database of commits to the `rustc` since the GitHub API for fetching this information can be used by the `rustc-perf` system.

Usage

The top-level bors merge commits can be fetched from `commit-list`.

Configuration

This has no configuration, it is processed automatically.

Implementation

See `src/db/rustc_commits.rs` and `src/handlers/`

Shortcuts

Shortcuts are simple commands for performing common actions.

Usage

Shortcut commands can be issued by writing a GitHub comment.

ready

```
@rustbot ready
```

This indicates that a PR is ready for review. This adds the `S-ready` label on the pull request and removes both `S-waiting-on-reviewer` and `S-waiting-on-author`.

`@rustbot review` or `@rustbot reviewer` are aliases for `@rustbot ready`.

author

```
@rustbot author
```

This indicates that a PR is waiting on the author. This adds the `S-waiting-on-author` label on the pull request and removes both `S-waiting-on-reviewer` and `S-waiting-on-merge` if present.

blocked

```
@rustbot blocked
```

This indicates that a PR is blocked on something. This adds the `S-blocked` label on the pull request and removes both `S-waiting-on-author` and `S-waiting-on-reviewer` if present.

Configuration

This feature is enabled on a repository by having a

```
[shortcut]
```

Implementation

See `parser/src/command/shortcut.rs` and `src/ha`

Triagebot Dashboard

The triage dashboard is used to assist with triaging

Usage

The triage dashboard for repositories can be found

Any rust-lang repository can be viewed with the form `lang.org/triage/<owner>/<repo>`.

Configuration

This feature has no configuration.

Implementation

See `src/triage.rs`.

Zulip Meeting Management

Triagebot can respond to some commands in Zulip

Usage

Enter a message in Zulip addressed to `@triagebot`

Document reading

```
@triagebot read
```

This command will cause triagebot to post a comment reading some document, and are ready to start discussing something like:

```
Click on the :book: when you start reading (a
Click on the :checkered_flag: when you finish
```

Users can then click the emoji reaction buttons to indicate and then again when they are finished.

End topic

```
@triagebot end-topic
```

This command will cause triagebot to post a comment is ready to move on to the next topic. The message

```
Does anyone have something to add on the current
React with :working_on_it: if you have something
React with :all_good: if not.
```

Users can then click the emoji reaction buttons to indicate

`@triagebot await` is an alias for `end-topic`.

End meeting

```
@triagebot end-meeting
```

This command will cause triagebot to post a comment to end the meeting. The message looks something like:

```
Does anyone have something to bring up?  
React with :working_on_it: if you have something to discuss  
React with :all_good: if you're ready to end the meeting
```

Users can then click the emoji reaction buttons to indicate their status.

Configuration

This feature has no configuration, it is available to all users. A user ID needs to be configured in the [teams database](#).

Implementation

See [src/zulip.rs](#).

Zulip Notifications

Triagebot can send messages to Zulip based on various events.

Usage

Zulip notifications are automated based on the content of pull requests. They can be triggered based on the addition or removal of labels, or when a pull request is reopened.

For example, the `rust-lang/rust` repository is configured to send a Zulip message whenever an issue is tagged with the `A-ec2021` label. The "2021" stream, which looks something like:

```
triagebot
```

```
Issue #109298 "ICE Subslice unexpected because of edition=2021" has been added.
```

Configuration

This feature is enabled on a repository by having a `triagebot.toml` file in the repository's root directory.

```
# Triggers a Zulip notification based on the
[notify-zulip."label-name"]
# The Zulip stream to post to.
# Can be found by looking for the first number in the URL
# lang.zulipchat.com/#narrow/stream/131828-t-compilers
zulip_stream = 245100 # #t-compiler/wg-priorities

# The Zulip topic to post to.
# {number} is replaced with the issue/PR number
# {title} is replaced with the issue/PR title
topic = "#{number} {title}"

# The message to post when the label is added
# Supports {number} and {title} substitution
message_on_add = "Issue #{number} \"#{title}\" has been nominated"

# The message to post when the label is removed
# Supports {number} and {title} substitution
message_on_remove = "Issue #{number}'s nomination has been
all for participating!"

# The message to post when the issue/PR is closed
# Supports {number} and {title} substitution
message_on_close = "Issue #{number} has been closed. No more
participating!"

# The message to post when the issue/PR is reopened
# Supports {number} and {title} substitution
message_on_reopen = "Issue #{number} has been reopened. More
participating!"

# The Zulip notification will not be posted if the issue/PR has
these labels.
# Please replace the `{team}` placeholder with the team that will be
notified for the nomination
# (ex. `I-compiler-nominated`, `I-lang-nominated`)
required_labels = ["I-{team}-nominated"]
```

Implementation

See [src/handlers/notify_zulip.rs](#).

GitHub Actions created

This automation triggers an automatic close & reopen of pull requests using the `actions` user, i.e., from a GitHub actions job. This eliminates the need for needing a manual poke from some human.

Configuration

This feature is enabled on a repository by having a `triagebot.toml`:

```
[bot-pull-requests]
```

Implementation

See `src/handlers/bot_pull_requests.rs`.

Community

This section documents the processes of the comm

External Links

- The [Community team](#) GitHub repository conta community team organizes.
- The [RustBridge](#) website contains information o event.
- [Rustlings](#) is an project with small exercises des used to reading and writing Rust.
- The [State of Rust](#) is an annual community surv from Rust users, and all those who are interes generally.

Compiler

This section documents the Rust compiler itself, its development, and how to provide bug fixes for the compiler.

External Links

- The [Rustc Dev Guide](#) documents how the compiler works and provides information to help get new contributors involved.
- Rustc's [internal documentation](#).
- The [Compiler team](#) website is the home for all compiler-related discussions.
- oli-obk's [FIXME page](#) lists all of the **FIXME** comments in the compiler.

Cross Compilation

This subsection documents cross compiling your co

Windows

1. Acquire LLD somehow. Either your distro provides it or you can build it from source.
2. You'll need an lld-link wrapper, which is just lld with some flags as link.exe. You may either have a wrapper or you can write some sort of script to wrap lld.
3. If you want to be able to cross compile C/C++ you'll need a compiler, which is clang pretending to be cl.
4. You'll need libraries from an existing msvc installation to link code against. You'll need the VC++ libraries from the Windows SDK or system libraries from either the Windows 8.1 or Windows 10 SDK. Approximate paths which may vary depending on what is installed. Copy them over to your non-windows machine.
 - VS 2015: `C:\Program Files (x86)\Microsoft Visual Studio\2015\Community\VC\Tools\MSVC\14.00.50724\lib`
 - VS 2017: `C:\Program Files (x86)\Microsoft Visual Studio\2017\Community\VC\Tools\MSVC\14.16.26324\lib`
 - Windows 10 SDK: `C:\Program Files (x86)\Windows Kits\10\Lib\10.0.14393.0\x-
x64`
 - Windows 8.1 SDK: `C:\Program Files (x86)\Windows Kits\8.1\Lib\x-
x64`
5. If you want to cross compile C/C++ you'll also need to add the above paths with `include` to get the appropriate headers.
6. Set your LIB and INCLUDE environment variables to point to the relevant directories for the correct architecture.
7. In your .cargo/config add `[target.x86_64-pc-windows-gnu]` or whatever your lld pretending to be link.exe is.
8. For cross compiling C/C++, you'll need to get the appropriate target. I tested it to cross compile, I have no idea whether it works or not.
9. Install the appropriate target using rustup and `msvc` while building. Hopefully it works. If it doesn't, let me know.

Cross-team Collaboration

If you are a member of another team and would like to collaborate with the compiler team..

..for discussion

Write a comment on a GitHub issue describing the problem, the decision needs to be made/what opinion is sought; (e.g. compiler team, etc) and add the `I-compiler-nominated` label. Add `@rustbot label +I-compiler-nominated` in your comment.

Once nominated, the issue will be discussed in a upcoming meeting. The compiler team doesn't always get through all nominated issues in one meeting for your issue to be discussed.

Once discussed, a member of the team will comment on the issue, summarizing the discussion and linking to the relevant Zulip chat.

..to be fixed

If there is an existing working relationship between a team and a contributor to the compiler, then the first option for tasks to be completed is to ping that contributor and ask them to fix the issue. It is recommended that pings take place in public Zulip channels.

- ..other contributors that have free time have the capacity to fix the issue
- ..other compiler team members/leadership can help if the request is reasonable (see the rest of this section for more details on team commits to prioritizing on behalf of other teams)

It is worth considering the available bandwidth of the team being made of, and whether their areas of expertise match the issue.

When there is not an appropriate contact in the compiler team, write a comment on a GitHub issue (or create an issue if one does not exist).

completed. Teams should nominate issues for the c

- ..are not already tracked by/part of an existing
 - ..are blocking/impeding the work of the other
 - ..aren't absolutely mission-critical - a soundness
- the stabilization of something otherwise comp
- be prioritized by the [prioritization working group](#)
- compiler team's other processes for these bug
- label, you can add the `I-prioritize` label and

A detailed description of the feature being requested
wherever possible (so that the compiler contributor
a solution that would solve the problem for the request
requesting team isn't explicitly listed as the point-of
of the comment will be assumed to be the point-of-

Add the `I-compiler-nominated` label to a issue (you
`compiler-nominated` to do this).

Once nominated, the issue will be discussed in a up
team doesn't always get through all nominated issue
than one meeting for your issue to be discussed. In
issue may..

- ..be accepted, in which case it will be assigned
label removed. Once assigned, a member of the
work is completed after a reasonable time, the
compiler team will find someone else to comp
- ..or not accepted (e.g. due to insufficient band
being unable to find an appropriate contributo
this case, the compiler team will reply to the n
will remove the nomination label.

Review Policy

This document describes our review policy for contributions. The intended audience are contributors and reviewers alike.

The purpose of this policy is to help contributors shape their review - by clarifying the Rust project expectations - and provide a handy list of common things to check. The project is about how to work together.

It is the purpose of code reviews to:

- **Reduce** the risk of introducing **bugs** and usability issues.
- Keep our code **maintainable: readable, documented**.
- Ensure that changes are made with the big picture in **mind**. This is particularly relevant for changes that are problematic or undesirable in the larger context.

Reviewing accomplishes this by bringing in another person's change from a different perspective, which increases the chance of catching errors early and uncovering potential blind spots in the reasoning.

Basic Reviewing Requirements

There are a number of requirements that need to be followed to be effective:

- Reviewers must have a sufficient **understanding** of the change.
 - This is important to help spot non-obvious issues in the given change.
- Pull request authors must provide:
 1. A concise high-level **description of the change**, unless the change is extremely trivial, like a typo.
 2. For the rationale to be even more useful, reviewers should look for potential **points of contention, compromise, or alternative approaches** that have been discussed in the **documentation, discussions and context**.
 - Reviewing code is difficult, and reviewers

to do it. Jump-starting the review process together the intention and context of a pull request together with the PR description. This helps reviewers understand the PR up but also improve the quality of the review.

- Reviewers must have a good idea on whether the change is worth merging.
 - Knowledge of the code under review is a prerequisite for answering this question.
 - Procedure wise, the reviewer also needs to consider:
 - Can the reviewer make the decision on whether the PR is ready to merge?
 - Does the PR need to go through the T-compiler Final Comment Period? Do we need to allow last comments or concerns before merging? Do we need a full [Request For Comments](#) (RFC)?
 - Does the PR need reviews and/or sign-off from the T-lang?
 - Can the changes break stable code or introduce breaking changes? If the PR contains changes that need ecosystem impact, how do we handle that?
 - Will the PR introduce significant performance regression, is it justified? Does the PR contain any security fixes?
 - Can the reviewer perform the review in a timely fashion?
 - Is the reviewer impartial enough to review the PR from a different perspective? E.g. due to co-authorship or due to the PR made by the reviewer) or other factors?

Reviewing Checklist

The following list of questions will help reviewers answer the above criteria in good shape and meet the above criteria:

Checklist for PR authors and reviewers

- Does the PR message have
 - A concise **high-level description** of the change?
 - A clear **rationale** for doing them? (*why* are we doing this?)

- If non-trivial and if suitable, **how** the b implemented?
 - A list of potential **points of contention**
 - **Links to relevant issues**, RFCs, MCPs
- Does the PR need a **regression test**? Does t **coverage**?
- Does the change need to be covered by a m already covered? If there is already a MCP o the PR blocked on that?
- Does the PR need a **perf run**?
- Does the PR need reviews and/or sign-offs f lint expansions if the ecosystem impact is la
- Does the PR affect other teams in a non-triv get a heads-up? E.g. changes to rustfmt or r
- Would someone trying to understand the P **reconstruct what's going on**?
- Is the new code **properly documented**? Is c date?
- Does the changes in the PR need updates to
- Does the PR introduce a **regression** of any
 - Error message quality
 - Maintainability (e.g. complex code, no
 - Any specific target platforms
 - Downstream tooling (e.g. linkers, debu
 - Compile times
 - Memory usage
 - Targets (e.g. baselines, target features

Checklist for reviewers

- Am I the **right person to review** this PR:
 - Does the changes in this PR fall under
 - Do I understand the code well enough
 - Would I be able to spot non-obvious si
 - Would I be able to fix a bug introduced
 - Can I do the review in a timely fashion
 - Do I feel pressure to quickly approve t
 - Am I impartial enough?
- Before merging:

- Is the **PR title and description** still accurate?
- Is the **commit history clean** enough? (no merge commits in the PR history).
- Does the PR correctly/incorrectly close issues?
- Do I need to roll reviewers from other relevant teams?

Guidance for dealing with common PR issues

In most cases common sense is enough for deciding whether to accept a PR, but sometimes there are gray areas where it is not immediately obvious. This section lists a few common cases together with guidance on how to deal with them.

I don't think I am a good fit for reviewing this PR

It is completely normal that you get (randomly) assigned a PR that you don't feel comfortable reviewing it. Here is what you can do in a concrete case:

- If the change seems really big or contentious, or if you are not familiar with the code (see below). The reviewer does not and should not review it. Instead, you should ask for a more experienced and/or significant PR.
- If you know just the right person for the review, you can ask them to review it. Use `@username>`. It's polite to leave a comment asking them to review it, but you don't have to make sure beforehand that they will accept. If they don't, you can ask someone else.
- If the change is not too complicated and you don't have a strong opinion, a rolled compiler reviewer will also have trouble reviewing it. You can ask a compiler reviewer with `r? compiler`.
- If the change is complicated, or you expect a rolled reviewer will just lead to multiple rerolls, you should ask for a `compiler/private` review to ask the rest of the team to review it, or even if the team is comfortable with the change.
- If the change is intended for another team, roll it to them. (example `r? compiler`)

You can also always ask for help on the `#t-compiler` channel. That being said, you are always welcome to do an informal review if you have time.

comfortable with) and then pass the PR on to the first reviewer. The first reviewer will get helpful feedback sooner, subsequent reviews will be faster, and the PR might also improve your own understanding of divergence.

It is unclear if something constitutes a major change

Deciding if something is a “major change” requiring a triage meeting is subjective. The official guidelines are [here](#). When in doubt, err on the side of a triage meeting. You can also nominate the PR for discussion at a triage meeting by tagging it `+I-compiler-nominated`. If you do so, please state a **concrete question** for the compiler team to answer at the meeting, if suitable.

Example triage meeting nomination message

```
Nominating for T-compiler triage meeting to discuss the PR
`-Z unstable-options` also accept value(s) for
`options=values` to
guard *unstable values* of *stable flags*

Relevant discussion: <https://i.am.a.link>
See also #123456.

@rustbot label +I-compiler-nominated
```

Discussion or rationale is too intransparent

Sometimes there are PRs that seem to be the result of a discussion or rationale. They usually have a title like “Discussion: ...” or the PR message is “r? @xyz”. Even though the change has been suggested by a compiler team member, the PR message is not self-contained.

Contributors may stumble across the PR several years later and find the PR with absolutely zero context that was discussed. This information is not available to future contributors. The PR message should include relevant context. Including relevant context will very often help the PR to be accepted.

The PR message should give a self-contained description of the change and the rationale for it.

is being changed and anything else that might be of

Try to put yourself in the shoes of someone who has to fix a bug related to the code touched by the PR. Give a clear rationale for the way things are.

Reviewer and PR author report to the same employer

There is no rule that prevents two employees of the same company from reviewing each other's PRs. We assume compiler team reviewers to be team members to do so.

The concerns in such a case are no different than for any other PR. Consider the mechanisms and principles we articulated above and apply them to whatever their employer. Does the PR concisely describe the changes made? Does it give a clear, transparent rationale for the changes? Can contributors down the line follow the reasoning? Have points of contention been discussed and clearly resolved?

If you are in doubt if something is contentious, give your opinion and ask for another opinion. If the proposed change has a significant impact, you can discuss in a [#t-compiler](#) zulip topic or a [Proposal](#).

Reviewing and Mentoring

In the course of mentoring someone through a PR it is common to have ended up effectively co-writing the changes. This can be a good thing: a reviewer is effectively approving their own changes. However, it is important to take into account when deciding how to proceed.

- If the general direction of the changes has already been agreed and the concrete advice given during mentoring covers all minor technical issues, then no further review is needed.
- Similarly, if any contentious decisions have been discussed in the PR with other compiler team members and a consensus has been reached from the general direction that has been agreed, then no further review is required either.

- If the PR was opened as a response to a concern (the changes are not entirely trivial) then it is advised by someone else. However, the initial reviewer should bring the PR into good shape before handing it off.

In general, it is advisable to ask for a second opinion in such cases, just to increase the chance of understanding what the mentor might have.

Nobody understands the code that's being reviewed

Sometimes there is a bug in some code that nobody understands. If the authors are unavailable and it is hard to gauge the issue, in a case it is a good idea for reviewers to add the `I-compiler-team` label (stay the main reviewer) or assign a compiler team lead the `waiting-on-team` label.

In both cases, the PR will be brought in the weekly team meeting, which is valuable to gather and document as much information as possible, such as a description of the problem being fixed, points of confusion, alternatives that have been considered, et cetera. It is also an issue to document the lack of understanding of such questions, concerns and bugs, and it can be resolved with better understanding.

Reviewers should ask PR authors to add this kind of information and/or to the PR message (which will become part of the commit message).

PR makes a change to support use of rust in other projects

This will need to be determined on a case-by-case basis.

In general, we should allow changes making things more general, as long as the owners of the code are comfortable with them).

As a concrete example: if someone is using the mirror for something public, it is likely not a problem, but there

module- or crate-private on purpose, as they uphold. So basically, just assign such PRs to the relevant people due to having told highfive that they want to get pin

Require a doc comment on such APIs identifying what concerns, and for what kinds of purpose.

If this is possibly contentious, ask for an [MCP](#).

Note that this can non-obviously bound supposedly consumers. Convey to the external consumers (that can offer the convenience so as long as it does not impose a burden on the compiler, e.g. gets in the way of refactoring guarantees are promised.

The PR is very large and complicated

Reviewers are **not** expected to stomach PRs that are expected from contributors to split their work to make a series of more digestible PRs which are individually general, before submitting large impact changes, it is discussed the design previously with the relevant team reference such discussions.

When in doubt, bring the PR to the attention of the team (nominate for compiler triage meeting), and the team

- The team can find suitable reviewers who can break a large change into smaller logical PRs that are pin also in the context of the larger change.
- The team does not have the bandwidth, or team or able to accept the large change as-is. In such a decision to postpone or close, and clearly communicate the author to explain the reasoning. It is very frustrating only for it to be rejected anyway.

Technical Aspects of Reviewing

Every PR that lands in the compiler and its associated crates has one person who is knowledgeable with the code in charge of it.

When a PR is opened, you can request a reviewer by leaving a comment with a description. If you don't do so, rustbot will automatically select reviewer candidates, determined by the files affected by the PR.

It is common to leave a `r? @username` comment at the top of the PR, where `@username` is the name of a reviewer from someone else. This will also reassign the PR.

It is possible to request reviews from multiple reviewers by leaving multiple comments like the one above.

```
Rolling both a T-compiler and T-bootstrap review requires both compiler and bootstrap changes.
```

```
r? compiler
r? bootstrap
```

bors

We never merge PRs directly. Instead, we use bors. A user with bors privileges (e.g., a [compiler team member](#)) will leave a comment like `@bors merge` to indicate that they approve the PR.

People with bors privileges may also leave a `@bors cancel` comment to indicate that the PR was already approved by `@username`. This will prevent bors from merging the PR.

Finally, in some cases, PRs can be “delegated” by writing `delegate=username`. This will allow the PR author or another user to merge the PR by issuing `@bors merge` commands like the ones above (but without the need for a PR).

Reverts

If a merged PR is found to have caused a meaningful regression, the policy is to revert it quickly and re-land it later once the regression is fixed.

A “meaningful regression” in this case is up to the judgement of the Rust community. A revert is not a failure.

Some criteria to consider if a revert is warranted:

- A bug in a stable or otherwise important feature changes runtime behavior, or triggers a (warning) in real-world code. Especially if the bug is reachable in production gates.
- If a bug or change (incl. ICEs) is particularly easy to trigger.
- If a bug or change significantly degrades compiler performance.
- If a test is flaky and unreliable.

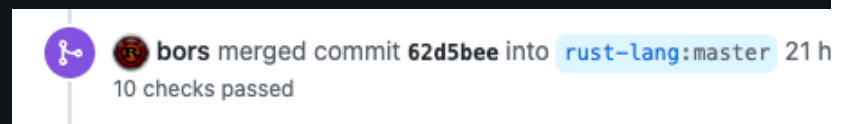
When these criteria are in doubt, and especially if reverting a PR. This has three benefits:

1. It allows bleeding edge users (esp. nightly or beta) to get bugs on HEAD with a higher degree of certainty that they won't be introduced.
2. It takes pressure off the original PR author and prevents them from feeling like they have to fix it *immediately*.
3. It might prevent the significant bug or regression from making it into the nightly/beta/stable build.

Before being reverted, a PR should be shown to cause a high degree of certainty (e.g. bisection on commits, or bisection by compiler team members pointing to this PR, or it's shown to be a compiler bug). Only revert with lower certainty if the issue is particularly severe.

Creating reverts

The easiest method for creating a revert is to use the "bors merged commit" button that appears next to the "bors merged commit abcd" message on a new pull request.



Alternatively, a revert commit can be created using the following command:

```
$ git revert -m 1 62d5bee
```

Don't rely *only* on the default commit title and message to make a revert commit meaningfully, and link to the relevant PR. Link to the specific PR that is being fully or partially reverted. Link to the discussions. Retain the commit hash being reverted.

Example revert commit title and message

```
Revert #131669 due to ICEs
```

```
Revert <https://github.com/rust-lang/rust/commit/703bb982303ecab02fec59f415c07494b98e4559e4b13a9c>: reports:
```

- <https://github.com/rust-lang/rust/issue/131669>
- <https://github.com/rust-lang/rust/issue/131669>

```
The changes can be re-landed with those changes.
```

```
This reverts commit 703bb982303ecab02fec59f415c07494b98e4559e4b13a9c and changes made to f415c07494b98e4559e4b13a9c
```

It's polite to tag the author and reviewer of the original PR. You can use the following message template for the revert.

```
Reverts rust-lang/rust#123456
cc @author @reviewer
```

```
This revert is based on the following report:
PR:
<link to issue or comment(s)>
```

```
In accordance with the compiler team [revert policy],
meaningful regressions should be reverted and re-landed
fixed (and a regression test has been added, where
[revert policy]: https://forge.rust-lang.org/
```

```
Fear not! Regressions happen. Please rest assured that this
represent a negative judgment of your contribution, but we view it
positively to Rust in the future. We simply want to ensure that
existing use cases working, and keep the compiler more stable.
r? compiler
```


Please include a temporary regression test in a separate PR. If a regression is actually addressed by the revert commit, the regression test can be adapted or removed following the PR.

If you have r+ privileges, you can self-approve a revert. To cause new regressions on its own, make sure the revert is “worse than the poison”. If non-trivial, please wait for approval from another compiler reviewer via `r? compiler` if the matter is more urgent.

Generally speaking, reverts should have elevated privileges to the PR they are reverting. If a non-rollup PR is shown to cause a regression, it can be marked `rollup=always`. The revert author is encouraged to authoring rollups to reschedule rollups or interleave them as suitable.

Forward fixes

Often it is tempting to address a regression by posting a new PR reverting the regressing PR, instead augments the overall change. However, if real-world users have a problem, this practice is strongly discouraged unless one of the following conditions are met:

- A high-confidence fix is already in the bors queue.
- The regression has made it to a release branch and a fix is needed. Often the “smallest possible change” (a fix doesn’t introduce *new* regressions). The offending PR is reverted on the main branch; this is left to the discretion of the reviewer.

While it can feel like a significant step backward to have to revert a PR, it is much easier to reland the PR once a fix can be committed. This takes pressure off of you and your reviewers to act on the issue fully. It also is an opportunity to take a step back and think about the issue fully.

Rollups

All reviewers are strongly encouraged to explicitly mark a PR as a rollup. This should be part of a `rollup`. This is usually done either with `$ROLLUP_STATUS` or with `@bors $ROLLUP_STATUS` when the PR is ready for review.

with one of the following:

- `rollup=always` : These PRs are very unlikely to have significant implications. Example scenarios:
 - Changes are limited to documentation, CI, or a build failure.
 - Changes cannot have performance implications.
 - Your PR is not landing possibly-breaking changes.
 - Feature stabilization without other changes.
 - When in doubt do not use this option!
- `rollup=maybe` : This is the default if `@bors r+` is used. Use this if you have some doubt that the change should be rolled up. If you use `rollup=maybe` and it is not the default, there is usually no need to explicitly specify the rollup level from a previous command.
- `rollup=iffy` : Use this for mildly risky PRs (more than `rollup=maybe` but less than `rollup=never`). Example scenarios:
 - The PR is large and non-additive (note: adding tests is fine to rollup).
 - Has platform-specific changes that are not easily testable.
 - May be affected by MIR migrate mode.
- `rollup=never` : This should *never* be included in a rollup (you must explain why you have chosen this). Example scenarios:
 - May have performance implications.
 - May cause unclear regressions (we would need to test specifically, as it would be hard to identify the cause).
 - Has a high chance of failure.
 - Is otherwise dangerous to rollup.
 - Messes too much with:
 - LLVM or code generation
 - bootstrap or the build system
 - build-manifest
- `rollup` : this is equivalent to `rollup=always`
- `rollup-` : this is equivalent to `rollup=maybe`

Priority

Reviewers are encouraged to set one of the rollup s

priority. Bors automatically sorts based on the rollup (which is always the lowest), and also by PR age. If you do use bors, use your best judgment to balance fairness and urgency with priority.

The following is some guidance for setting priorities

- 1-5
 - P-high issue fixes
 - Toolstate fixes
 - Reverts containing the above
 - Beta-nominated PRs
 - Submodule/Subtree updates
- 5+
 - P-critical issue fixes
- 10+
 - Bitrot-prone PRs (particularly very large changes)
 - Urgent PRs (e.g. urgent reverts)
 - Beta backports
- 20+
 - High priority that needs to jump ahead of the queue
 - Fixes or changes something that has a high priority PR in the queue.
- 1000
 - Absolutely critical fixes
 - Release promotions

Expectations for r+

bors privileges are binary: the bot doesn't know who is right and what code you are not. They must therefore be used sparingly. If you do not know well – you can definitely **review** something, but to someone else for the final r+.

Similarly, never issue a `r=username` command unless you are sure and the code has not changed substantially since the last review. A person has explicitly indicated that another contributor's review is needed.

Rebasing is fine and often necessary, but changes in priority need review. It is very helpful for the reviewer if the PR author

what has changed since last review, in addition to re comments.

Please refer to [bors documentation for bot usage](#).

Social aspects of reviewing

First and foremost, PR authors and compiler review [Code of Conduct](#). Simply speaking, a reviewer is expected to be respectful to the author, even if the reviewer disagrees with the change.

Reviewers are encouraged to consider matters from the author's perspective as well. If a change is stuck due to procedural reasons or without any resolution (including a resolution that the reviewer is not comfortable accepting such a change at present time, but thank the author for the contribution anyway), and accrues constant merge conflicts, it can be closed.

If some discussions are getting heated, ask the [moderators](#) for help.

So you want to add a new flag to rustc

So you want to add a new command-line flag to rustc

Is this a perma-unstable option?

The first question to ask yourself is:

- Is this a “perma-unstable” option meant only for `err-as-bug`)?

If so, you can just add it in a PR, no check-off is required

Other options

If this option is meant to be used by end-users or to be used by the compiler, however, it represents a “public commitment” on the part of the Rust team to maintain, and hence there are a few more details to consider

There are two main things to take care of, and they must be completed:

- Proposal and check-off
- Implementation and documentation

Finally, some options begin as unstable and only get stabilized later; you will also need:

- Tracking issue and stabilization

Proposal and check-off

The “proposal” part describes the motivation and details of the change to add. It doesn’t necessarily have to be very long. It takes the form of a

Proposal.

The proposal should include the following:

- **Motivation:** what is this flag used for?
- **Design:** What input does the flag take and what does it do?
- **Implementation notes:** You don't have to talk about implementation details, but if there are any key things to note (i.e., it would be a breaking change), you might note them here.
- **Precedent, links, and related material:** Are there any similar flags in other compilers/linkers/tools, like clang or lld?
- **Alternatives, concerns, and key decisions:** What other options were considered? If so, why did you pick this design?

Note that it is fine if you don't have any implementation details to discuss.

Also, one good approach to writing the MCP is basically to write the flag and then you will have to write anyway to explain to users how the flag works. You can add additional notes on alternatives and so forth that are helpful.

Once you've written up the proposal, you can [open a pull request](#). If the MCP is promoting a permanent change, a full compilation of the code is required, a "second". This can be done by `@rfcbot fcp merge`.

Implementation, documentation

Naturally your new option will also have to be implemented in the compiler. You can open an option and open up a PR. Often, this implementation is done after the MCP is created, and that's fine – we'll just ask you to implement it.

See the [Command-line Arguments](#) chapter in the rustc book for more details on how to name and define a new argument.

A few notes that are sometimes overlooked:

- Many options begin as "unstable" options, either by being marked as such or by requiring `-Zunstable-options` to use.
- You should document the option. Often this documentation is added to the MCP text. Where you add this documentation is up to you, but it should be available on stable Rust:

- If it is unstable, then document the option whose sources are in [src/doc/unstable-book](#).
- Once the option is stabilized, it should be documented whose sources as in [src/doc/rustc](#).

Stabilization and tracking issue

Typically options begin as unstable, meaning that they are in `-Zunstable-options`.

Once the issue lands we should create a tracking issue and stabilization can be proposed.

Stabilization generally proceeds when the option has a good implementation seems to be working as expected for a while.

Remember that when stabilization occurs, document the option in the Unstable Book to the Rustc Book.

Major Change Proposals

Introduced in [RFC 2904](#), a “major change proposal” is a process the compiler team uses for architectural changes that are not typically used for small user-facing changes like adding new features. Major changes also require an `rfcbot fcp` to get full approval from the compiler team. Modifications to the Rust language itself require a full team’s purview).

Motivation

As the compiler grows in complexity, it becomes harder to manage on. We don’t currently have a clear channel for people to propose “major changes” that may impact other developers (and receive feedback).

Our goal is to create a channel for signaling intention to open a PR (and perhaps cc’ing others on that PR) before meeting proposal or RFC.

Goals

Our goals with the MCP are as follows:

- Encourage people making a major change to voice their intent and what they plan to do.
- Ensure that folks in the compiler team are aware of the change and have a chance to respond.
- Ensure that every proposal has a “second”, meaning someone who thinks it’s a good idea.
- Ensure that major changes have an assigned advocate.
- Avoid the phenomenon of large, sweeping PRs getting stuck in someone’s review queue.
- Avoid the phenomenon of PRs living in limbo because full approval is required for them to land.

Major Change Proposals

If you would like to make a [major change](#) to the compiler,

- Open a tracking issue on the [rust-lang/compiler](#) [template](#).
 - A Zulip topic in the stream `#t-compiler/` will be created for you by a bot.
 - If concerns are raised, you may want to respond to those concerns.
 - Alternatively, you can submit a [design memo](#) for a more focused discussion.
- To be accepted, a major change proposal needs:
 - One or more **reviewers**, who commit to reviewing the person making the proposal, if they interpose.
 - A **second**, a member of the compiler team, to approve the one originating the proposal.
 - A **final comment period** (a 10 day wait time).
 - The FCP can be skipped if the change has no objections are considered unlikely. This is usually the case if a lot of prior discussion, for example on a Zulip topic, has taken place.
- Once the FCP completes, if there are no outstanding objections:
 - If those PRs make outward-facing changes, the MCP or the PR(s) must be approved via a PR.

Conditional acceptance

Some major change proposals will be conditionally accepted. We want to see the work land, but we'd like to re-evaluate the proposal after the design after we've had time to gain experience. We want to evaluate things we'd like to evaluate, and ideally a timeline.

Deferred or not accepted

Some proposals will not be accepted. Some of the proposals

- You may be asked to do some prototyping or a decision is reached
- The idea might be reasonable, but there may not be time or there may just be too many other things going on
- The idea may be good, but it may be judged too complex to maintain, and not worth the benefit
- There may be flaws in the idea or it may not suit the project

What happens if someone opens a PR for a major change *without* doing this?

The PR should be closed or marked as blocked, with a message explaining why. A proposal should be made first.

If the PR description already contains suitable text to explain the change, you can simply copy and paste that into an MCP issue. Using this approach ensures that the tooling and process works smoothly.

Can I work on code experimental PRs if they are not accepted?

Of course! You are free to work on PRs or write code that is experimental and they should not land, nor should they be merged (unless folks want to).

What constitutes a major change?

The rough intuition is “something that would require a change to the [rustc book](#)”. In other words:

- Something that alters the architecture of some part of the compiler, or what the rustc-dev-guide aims to document.
- A simple change that affects a lot of people, such as a change to the public API.

- common types or changing coding convention
- Adding a compiler flag or other public facing change (ultimately) in the rustc book. This is only appropriate for minor things and not major things that may impact a lot of things. These require a full FCP before landing on stable, but you can propose the idea.)

Note that, in some cases, the change may be deemed to be required to move forward. This could occur with changes with sufficiently large changes to the architecture. This would require a call.

Note that whether something is a major change is not determined by the number of lines of code that are affected. Renaming a variable, adding a line, and even require edits to the rustc-dev-guide. At the same time, changing names that are very broad or changing a change (for example, renaming from the `tcx` context to `tcx` would be a major change).

Public-facing changes require RFC

The MCP “seconding” process is only meant to be used for changes to the architecture we plan to use. It is not sufficient to stabilize public-facing changes like adding a `-C` flag. For that, an RFC is required, if the change is large enough).

For landing compiler flags in particular, a good approach is to introduce a `-Z` flag and then “stabilize” the flag by adding it to the `rfcbot fcp`).

Major change proposals are not sufficient for language changes or cargo.

Steps to open a MCP

- Open a tracking issue on the [rust-lang/compiler](#) repository using the [template](#).

- Create a Zulip topic in the stream `#t-compiler`
 - The topic should be named something like “compiler-team#123”, which describes the issue.
 - The stream will be used for people to ask

What kinds of comments should issue in compiler-team repo?

Please direct technical conversation to the Zulip stream.

The compiler-team repo issues are intended to be for procedural purposes. Note that to “second” a design or offer to review is familiar with the code, typically but not necessarily.

- Announcing that you “second” or approve of the proposal
- Announcing that you would be able to review the proposal
- Noting a concern that you don’t want to be overruled
- Announcing that the proposal will be entering the next phase

How does one register as reviewer or raise an objection?

These types of procedural comments can be left on the issue (or in a message in Zulip). See the previous section. To facilitate the concerns please use the following syntax to format the message:

```
@rfcbot concern reason-for-concern
<long description of the concern>
```

And the following syntax to lift a concern when resolved:

```
@rfcbot resolve reason-for-concern
```

Who decides whether a concern

Usually the experts in the given area will reach a consensus. If there is a need for a “tie breaker” vote or judgment call, the co

What are some examples of major changes in the past?

Here are some examples of changes that were made through the major change process:

- overhauling the way we encode crate metadata
- merging the gcx, tcx arenas
- renaming a widely used, core abstraction, such as `Vec`
- introducing cargo pipelining
- adding a new `-C` flag that exposes some min

What are some examples of things that require the major change process?

Here are some examples of changes that are too big to handle through the normal process, which at least would require auxiliary design meetings before they can proceed:

- introducing incremental or the query system
- introducing MIR or some new IR
- introducing parallel execution
- adding ThinLTO support

What are some examples of things that don't merit a major change process?

Here are some examples of things that don't merit a

- adding new information into metadata
- fixing an ICE or tweaking diagnostics
- renaming “less widely used” methods

When should Major Change Prop

Major Change Proposals can be closed:

- by the author, if they have lost interest in purs
- by a team lead or expert, if there are strong ob team that don't look likely to be overcome.
- by folks doing triage, if there have been three people should feel free to re-open the issue if

Membership

This team discusses membership in the compiler team membership:

- members: regular contributors with r+ rights, [infrastructure](#)
- maintainers: members who have committed to the compiler and health of the compiler team

The path to membership

People who are looking to contribute to the compiler. They may tackle “one off” issues, or they may get involved in a working group. They don’t know much about the compiler team privileges. They are assigned to issues using the triage mentor or mentoring instructions.

Compiler team member

Once an individual has been contributing regularly to the compiler to the level of a **compiler team member** (see the section below). This title indicates that they are someone who

It is hard to define the precise conditions when such a person is promoted to member is not just a function of checkmarks, but the sense is that someone is ready when they have demonstrated

- “Staying power” – the person should be contributing consistently. This might for example mean that they have contributed to the compiler for a significant period.
- “Independence and familiarity” – they should be able to take on tasks, at least within the scope of their expertise, and plausibly be able to mentor others on simple Rust issues.
- “Cordiality” – compiler team members will be expected to be held to a higher standard with respect to the [Code of Conduct](#). They should obey the letter of the CoC but also its spirit.

Being promoted to member implies a number of pr

- Members have `r+` (approve a pull request) pr expected to use those powers appropriately, and have access to control perf/rustc-timer and other documentation for `bors` and `r+` [here](#).

Tip: some baseline rules around bors permissions: you have done a check for malicious code first and are reasonably confident that you can effectively r

- Compiler team members are members of the labels and be assigned to issues.
- Members become a part of the rust-lang/compiler receive pings when people are looking to add
- Members are [listed](#) on the rust-lang.org web p

It also implies some obligations (in some cases, opti

- Members will be asked if they wish to be added
- Members may take part in various other [main
- Members are held to a higher standard than o [Code of Conduct](#).

What it means to be a compiler

Once you're a member of the compiler team, a num

- You will gain access to a private Zulip stream, where ideas in very draft state are shared. Come and members!
- You will be subscribed and gain write access to Check [this GitHub page](#) to see which repositories they are pretty quiet or obsolete, so don't wo

Tip: Github automatically adds you as subscriber permission too. You can disable this in the set

- You will also be subscribed to the `all@rust-lang.org` mailing list. Please check how subscriptions to mailing lists work. (maybe a few emails per year), it's a way to communicate. We will not send you spam from this address.

Maintainers

After being a compiler team member for a year, members can become a **compiler team maintainer**. This implies being a contributor, but are actively helping to shape the direction of the compiler (or multiple parts).

- Compiler team maintainers are expected to participate in [compiler team activities].
- Compiler team maintainers are identified with the `compiler-team` website.

How promotion decisions are made

After an individual has been contributing to the compiler team, they can be nominated by an existing compiler team member or compiler team lead. Leads will lead if their contribution history is sufficient for team promotion.

The compiler team leads will check with the rest of the team about concerns with extending a membership invitation to a new member (barring no objections), an invitation will be extended to the individual.

If the invitation is accepted by the individual, the compiler team will update the `compiler-team` repository to reflect their new role.

Not just code

It is worth emphasizing that becoming a member of the compiler team does not necessarily imply writing PRs. There are a wide variety of ways to contribute to the compiler team.

support the compiler and which should make one eligible for elevation. This would include organizing meetings, participating in discussions, writing documentation, working on the rustc compiler, and so on. The criterion for elevation to compiler team member, in addition to active participation, is **shaping the direction of the team or compiler.**

Alumni status

If at any time a compiler team member or maintainer wishes to leave the team, they can opt to put themselves into alumni status. When they do, they will be removed from Github aliases and the like, but they will still receive pings and messages. They will also not have r+ access to the compiler team repository, **however still remain members of the GitHub organization.**

People in alumni status can ask to return to “active” status. Such requests would ordinarily be granted automatically barring extraordinary circumstances.

People in alumni status are still members of the team and they may publicly indicate that, though they should not claim to be active in which they were active as well.

Entering or leaving the Maintainer role

After a compiler team member has committed to active participation in becoming a Maintainer, they may wish to take a break from their responsibilities either temporarily or indefinitely. In such cases, the compiler team leads know or open a PR themselves to remove them from the Maintainer marker team and ping them to let them know.

In the future, if the former Maintainer would like to return to active status, they may request re-instatement from the compiler team leads. Such requests would be granted automatically barring extraordinary circumstances.

Compiler team alumni

Likewise, if any member of the compiler team would

contribution and interaction with the team, they can open a PR themselves to the [team](#) repo, moving the

If an alumni member would like to resume compiler can request re-instatement from the compiler team granted.

Automatic alumni status after 6 months

If a member or maintainer has been inactive in the ask them if they would like to go to alumni status. If they can be placed on alumni status. If they would prefer fine, but they will get asked again periodically if they

Prioritization

This section documents the processes of the prioritization

Prioritization WG - Process

This document details the procedure the WG-prioritization follows during the weekly meeting of **T-compiler**. The working group reviews **compiler** regressions, identifying possibly critical (and other) issues and building the agenda for the weekly **T-compiler** meeting. The main points to be discussed.

General issues review process

- Check the status of the issue
- Try moving it forward if possible (ex. stimulate author / reviewer)
- Ask for more info if it's needed
- Is there an MCVE for the issue already?
- Check if it's a regression and label it accordingly
- Figure out the area the issue belongs to and label it
- [Ping notify groups](#) or relevant teams
- Assign if possible
- Nominate the issue if it's unclear and needs to be discussed

Generating the T-compiler meeting agenda

The **T-compiler** agenda is generated from a template. We suggest working the following steps in this order:

Prepare agenda content

1. Add **T-compiler** labels where appropriate

- Issues labeled with **I-prioritize**
- Pull requests nominated for the stable release
- Pull requests nominated for the beta release

- Issues labeled `I-compiler-nominated` (i.e. need compiler work)
- Pull requests waiting on a team's feedback
- Issues classified with priority `P-high`

2. Assign a priority label to issues where needed

Regressions labeled with `I-prioritize` are signaling that they need attention and are waiting. When this label is added to an issue, the `tr` notification for `@WG-prioritization` members on `the`

To assign a priority, we replace the `I-prioritize` label with `P-medium` or `P-low` and adding a succinct comment about the issue prioritization occurred, example of a template:

```
WG-prioritization assigning priority (Zulip discussion)
@rustbot label -I-prioritize +P-XXX
```

Ideally, we want all `T-compiler` issues with a `I-prioritize` label assigned, or strive to reach this goal: sometimes difficulties in being assigned a priority label, either because the regression cannot be reproduced and an MCVE would be needed, or because cannot be reproduced and an MCVE would be needed, or because the issue reporter or ping the `ICEbreaker` ("ICEbreaker Compiler Errors") needs a reduction (add a comment `icebreakers-cleanup-crew`)

Keep an eye also on regressions (`stable`, `beta` and `nightly`) and assignee.

3. Accept MCPs

An MCP is a `Major Change Proposal`, in other words a change that needs a bit more thought and discussion within the community. The life cycle of an MCP is described in the document `MCP`. Prioritization is keeping an eye on them and accepting them after a `comment-period` for 10 or more days.

To accept an MCP, remove `final-comment-period`

label and close the issue. A notification to the relevant working group is automatically sent by the `triagebot`.

Generate the meeting's agenda

Run triagebot's CLI to generate the agenda. You need to set the `lang/triagebot` environment variables: `GITHUB_TOKEN` and optionally `PUBLIC_GCAL_ID` (if this env var is not found, it will be automatically copy&pasted from here).

To generate the meeting's agenda, run:

```
$ cargo run --bin prioritization-agenda
```

Copy the content of the generated agenda on HackMD.

Add performance logs

Paste the markdown file of this week's [performance report](#) into the agenda, removing emojis (to make the text readable).

Announce the meeting on Zulip

About two hours before the scheduled meeting, create a Zulip message in the `#t-compiler/meetings` channel titled "[weekly] YYYY-MM-DD meeting" using the following template:

```
Hi @*T-compiler/meeting*; the triage meeting is scheduled for [time] hours.
*WG-prioritization* has done pre-triage in #*prioritization/alerts*
@*WG-prioritization* has prepared the [meeting agenda]

Working group checkins for today:
- @**WG-foo** by @**person1**
- @**WG-bar** by @**person2**
```

Working Group checkins rotation are generated by the `triagebot`.

outdated and could probably be merged into the `tr`

Checkins about the progress of working groups are to be sure we don't miss on important progresses.

Add details to the Agenda

1. Summarize stable/beta nominations

These are pull requests that the compiler team *might* channel. Example a `stable-to-beta-regression` fix beta release channel. A `stable-to-stable-regress` warrant a point release (i.e. release a `1.67.1` after

Follow the [General issues review process](#).

2. Summarize PRs waiting on team

These are pull requests waiting on a discussion / decision (more than one team).

Try to follow the [General issues review process](#). Expect *quickly* resolved in a triage meeting.

3. Fill up the “Oldest PRs waiting for review”

This is probably the less automatable part of the agenda. `triagebot` will emit a list of 50 pull requests ordering idea is to issue mentions to assigned reviewers during the meeting to stay on top of them. We usually try to keep the number of PRs each meeting.

There are two human factors here to keep in consideration:

- Pull requests reviewers are volunteers, we respect their time. We don't want to remind them *too often* that there are pull requests waiting for them. Therefore we usually wait 2 or 3 weeks before reviewing pull requests. It seems like a long time to wait for something to be accomplished in the meanwhile! Anyway, we are

these metrics.

- Contributors taking their time to submit a pull request as an appreciation so we try to not have them wait too long for context about their work (or motivation to drive it)

Striking a balance between these two diverging forces is a “hard-won knowledge” that comes with practice. Other factors that can help progress:

- The review is shared with another team (i.e. Team 2)
- The alternating labels `S-waiting-on-review` and `P-critical` in the life cycle of a pull request are not promptly updated. A pull request to be reviewed but it's *not* labeled `S-waiting-on-review`

4. Add some context to **P-critical** and **P-high**

Try to follow the [General issues review process](#).

5. Summarize **I-compiler-nominated** issues

Issues labeled with `I-compiler-nominated` are general issues that the compiler team dedicate them a special slice of time to discuss (at the end). After the discussion, add a comment on GitHub that the discussion started (so everyone can read). `T-compiler` should be added to the discussion on the issue itself.

Try to follow the [General issues review process](#):

- Check if an issue needs a discussion and add to the agenda
- When added to the agenda, add some context
 - Who the assignee is
 - Is this an issue or a pull request: if it's an issue, what does it fix?
 - Why was it nominated
 - Other important details

6. Final review before the meeting

Re-run the triagebot CLI script and update the agenda.
This is useful when there are last second changes at

Follow-ups after meeting

The meeting is over! Time to cleanup a little bit.

- Lock the agenda file on HackMD assigning write permissions to the triagebot.
the markdown file and commit [it to this repository](#).
- Remove the `to-announce` label from [MCPs](#), unless they were discussed during the meeting (and therefore will be seen in the next meeting).
- Remove `to-announce` FCPs from [rust repo](#), [compiler repo](#), [std repo](#) with the same disclaimer as before.
- Accept or decline `beta nominated` and `stable nominated` labels that have been accepted during the meeting. For more information see [this Zulip discussion](#).
 - To accept a backport, add a `{beta,stable}` label and a `{beta,stable}-nominated` label. Other than these pull requests, it's important to leave the discussion open on Github linking the Zulip discussion.
 - To decline a backport, simply remove the `{beta,stable}` label and add a comment on Github explaining why the backport is not needed. See [this Zulip discussion](#).
- Remove `I-compiler-nominated` label from issues that were discussed. Not all nominated issues are discussed (because of the `I-compiler-nominated` label will stick until next meeting).
- Create a new agenda stub for the following week and link it on Zulip, so it's available for people if they want to discuss it.

Priority levels

As the compiler team's resources are limited, the priority is to identify the most relevant issues to work on, so that we can focus on what matters the most.

Words used in this document:

`issue` refers to bugs and feature requests that are being tracked, and `I-prioritize` is a label flagging the `I-prioritize` label as described below.

This document will define what each label means, and how they are used.

Labels

Labeling an issue as `I-prioritize` starts the prioritization process. It involves removing the `I-prioritize` label and appending one of the following labels below:

- `P-critical`
- `P-high`
- `P-medium`
- `P-low`

Each of these labels defines a strategy the team will use to handle the issue.

- The amount of focus a given issue will receive
- How members of the community can get involved

P-critical

A `P-critical` issue is a potentially blocker issue.

The Working Group will keep track of these issues on a weekly basis during the triage meeting.

Examples of things we typically judge to be “critical”

- Regressions where code that used to compile no longer does
 - Mitigating conditions that may lower priority
 - If the code should never have compiled (e.g. regression affects a large number of users or need a warning period)
 - If the code in question is theoretical, in the wild, or if it only exists in small, niche, or widely used
 - If a regression has been in stable for a release or more, still awaiting a fix, or because the bug has been around for a long time, typically lower the priority as well, because it has raised a ruckus about the regression, that makes it a critical issue. Eg: [an issue that would have been P-high](#)
- Regressions where code still compiles but does not behave as expected (dynamic semantics have changed)
 - Mitigating conditions that may lower priority
 - If code uses feature that is explicitly deprecated in docs state order in which it drops it
- Feature-gated features accessible without a feature flag
 - Mitigating conditions that may lower priority
 - If the pattern is VERY unlikely
- Soundness holes with real-world implications
 - Mitigating conditions that may lower priority
 - Soundness holes that are difficult to exploit
 - Soundness holes that will not affect security or gated unstable feature.
- Diagnostic regressions where the diagnostic is missing or confusing
- ICEs for common scenarios or code patterns
 - Mitigating conditions that may lower priority
 - If the code that triggers the ICE also triggers other errors those errors are emitted before the ICE
 - If the code in question makes use of a feature that the ICE requires a feature gate

A P-critical issue will receive the most attention. It must be fixed as soon as possible, and the rest of the team should help if/when applicable.

P-high

P-high issues are issues that need attention from the compiler team at the point that they need to be discussed at every meeting. They are issues that have a mitigating condition as defined above, or are blockers.

Because there are too many **P-high** issues to fit in a meeting, they should rather be handled asynchronously by the Prioritization WG and moved forward. They can still occasionally be brought up at a meeting if necessary.

The effectiveness of the Prioritization WG will be a good measure to draw the line between **P-critical** and **P-high** issues. **P-critical** issues that compiler meetings become unproductive shouldn't get lost in the list of P-high issues.

P-high issues are issues the teams will mostly work on, they are assigned, and keep an eye on them.

P-medium and P-low

P-medium refer to issues that aren't a priority for the compiler team in the long run. Eg issues that will be fixed after a specific milestone. Issues we would mentor someone interested in fixing, or if someone complains, a community member fixes it, or it's a low priority.

P-low refer to issues that the compiler team doesn't consider worth fixing.

Notification groups

The compiler team has a number of notification groups to draw their attention to issues. Notification groups are optional if they want.

Creating a notification group

If you'd like to create a notification group, here are the steps and approval from the compiler team:

- Propose the group by preparing a [Major Change](#) analogous to some existing group, it is probably better to lead before-hand or as part of the MCP.
- The MCP should specify what GitHub label will be used for the group. Often this is an existing label, such as [C-Compiler](#).

Once the MCP is accepted, here are the steps to actually create the group. We include an example PR from some other group.

- File a tracking issue in the [rust-lang/compiler-team](#) repository to track progress.
- Create a PR against the [rust-lang/team](#) repository to add the group. [Example PR](#).
- Configure the [rust-lang/rust](#) repository to accept the group. [Example PR](#).
- Create a PR for the [rustc-dev-guide](#) amending the guide to mention your group.
- Create a sample PR for the [rust-lang/team](#) repository for yourself. This will be referenced by your blog post. [Example PR](#).
- Create a Zulip stream for the notification group. If you don't do, you can ask on [#t-compiler/wg-meta](#).
- Write an announcement blog post for [Inside Rust](#). [lang.org](#). [Example PR](#).

Compiler-team Triage Meeting

What is it?

The triage meeting is a weekly meeting where we go over regressions, consider beta backports, and other such things. During the meeting, we also do brief check-ins with active working group members who have been working on.

When and where is it?

See the [compiler team meeting calendar](#) for the calendar of meetings. Meetings take place in the #t-compiler stream on the [rust-lang](#) Discord server.

Where can I learn more?

The meeting procedure is documented in [rust-lang/compiler-team-triage](#).

The working group check-in schedule is available on the [rust-lang](#) website.

Compiler-team Steering

What is it?

The “steering meeting” is a weekly meeting dedicated to discussion. The meeting operates on a repeating schedule:

- Week 1: Planning
- Week 2: Technical or non-technical discussion
- Week 3: Technical or non-technical discussion
- Week 4: Non-technical discussion

The first meeting of the 4-week cycle is used for **planning**. The purpose of the planning meeting is to **select the topics for the next three weeks** from a set of topic proposals, which must be uploaded to the steering meeting page before the meeting starts. The planning meeting is also an opportunity to “re-balance” our priorities.

The remaining meetings are used for design or general discussion. In weeks 2 and 3, the meeting is used for **technical** or **non-technical** discussion; it is possible to discuss the same topic, if that topic is complex. **Week 4** is used for **non-technical** discussion. **Week 4 topics**, so as to ensure that we are keeping an eye on the overall health of the team.

Where do proposals come from?

The team accepts proposals via an open submission process. See the [proposal submission](#) page for more details.

Announcing the schedule

After each planning meeting, the topics for the next four weeks are announced on the [compiler-team meeting calendar](#) and a blog post is published.

When and where is it?

See the [compiler team meeting calendar](#) for the calendar of meetings that will take place in the #t-compiler stream on the [rust-lang](#)

Submitting a proposal

If you would like to submit a proposal to the steering meeting, please come on! This page has all the details.

TL;DR

In short, all you have to do is

- [open an issue on the compiler-team repository](#)
 - use the template for meeting proposals
 - you only need a few sentences to start, but in the end, the place we typically expect a more detailed proposal

You don't have to have a lot of details to start: just a rough idea. Especially for technical design discussions, we will try to make a more detailed overview be made available by the time of the meeting.

Examples of good candidates for steering meeting

Here are some examples of possible technical topics for the steering meeting:

- A working group has an idea to refactor the HIR to make it easier. They have sketched out a proposal and would like feedback.
- Someone has encountered a problem that is related to the current data structures. They would like feedback on a possible solution.
- Someone has done major refactoring work on the compiler and would like to explain the work they did and request review.

Steering meetings are also a good place to discuss other topics:

- A proposal to move some part of the compiler to a different crate.
- A proposal to start a new working group.

Note that a steering meeting is **not** required to create a tree crate, but it can be useful if the proposal is complex like a dedicated time to talk out the plans in more detail.

Criteria for selection

When deciding the topics for upcoming meetings, we have the following criteria:

- We don't want to spend time on design work unless we will implement it and support it; this includes reviewing a suitable reviewer.
- We don't want to take on "too many" tasks at once and implement them.
- We also don't want to have active projects that are "too many toes", changing the same set of code in deep ways.

Meetings are not mandatory

It is perfectly acceptable to choose *not* to schedule a meeting (e.g.) there are no proposals available or if nothing is scheduled at this moment. Note that, to keep the "time expectations" generally stick to the same 4-week cycle and simply plan (e.g.) planning things at the last minute.

Adding a proposal

Proposals can be added by opening an issue on the Rust Forge issue template for meeting proposals that gives directions on how to open an issue with a few sentences describing what the proposal is about.

Some details that might be useful to include:

- how complex of a topic you think this is
- people in the compiler team that you think should be involved

Expectations for the meeting

By the time the meeting takes place, we generally will have a write-up or proposal. You can find a [template](#) for such documents created in the form of a hackmd document – usually found in the [minutes](#) directory of the compiler-team repository.

Expectations for a non-technical proposal

The requirements for non-technical proposals are lower. A few sentences or paragraphs may well suffice, if it is sufficient to understand the problem.

Frequently asked questions

What happens if there are not enough proposals? The meeting is mandatory. If there aren't enough proposals in some area, we may opt to not discuss anything.

How to run the planning

Week of the meeting

- Announce the meeting in the triage meeting
- Skim over the list of proposals and ping people about their availability over the next few weeks

Day of the meeting

- Create a `design meeting YYYY.MM.DD` topic
 - Ping `@t-compiler/meeting`, ideally 1h or more before to remind people
- At the time of the meeting, return to the topic
 - Ping `@t-compiler/meeting` to let people know the meeting is starting
- We typically begin with a 5min announcement
- Visit the [compiler-team](#) repository to get a list of topics

To actually make the final selection, we recommend the following process:

- First, try to identify topics that are clear non-candidates
 - for example, sometimes more investigation is needed
 - try to identify people to do those tasks
 - other issues may be out of date, or clearly not needed
- Next tackle technical design meetings, then non-technical
 - Typical ratio is 2 technical, 1 non-technical
 - It's ok to have fewer than 3 meetings

Announce the meetings

For each scheduled meeting, create a calendar event

- invite key participants to the meeting
- set the location to `#t-compiler, Zulip`
- include a link to the design meeting issue in the

In the relevant issues, add the `meeting-scheduled`

```
In today's [planning meeting], we decided to
**DATE**.
```

```
[Calendar event]
```

```
[planning meeting]: XXX link to Zulip topic
[Calendar event]: XXX link to calendar event
```

You can get the link to the calendar event by clicking
selecting “publish”.

Publish a blog post

Add a blog post to the Inside Rust blog using the [tern](#)
[repository](#).

How to run the design meeting

Week of the meeting

- Announce the meeting in the triage meeting
- Skim over the list of proposals and ping people about their availability over the next few weeks
- Make sure that a write-up is available and nag

Day of the meeting

- Create a `design meeting YYYY.MM.DD` topic
 - Ping `@t-compiler/meeting`, ideally 1h or more before the meeting to remind people
 - Include a link to the design meeting write-up
- At the time of the meeting, return to the topic
 - Ping `@t-compiler/meeting` to let people know the meeting is starting
 - Include a link to the design meeting write-up
- We typically begin with a 5min announcement

To guide the meeting, create a shared hackmd document (or use an existing one, if there is a write-up). Use this to help guide the meeting, reach consensus, and take live notes. Try to ensure that the meeting produces a consensus statement, even if that consensus is just a "we don't have a space of solutions and their pros/cons, but we don't know what to take".

After the meeting

- Post the final contents of the summary hackmd document to the `meeting` directory in the `compiler-team` repository
- (Optional) Create an Inside Rust blog post pointing to the meeting write-up, maybe giving a few notes

crates.io

This section documents the processes of the crates.

Crate removal procedure

If we get a DMCA takedown notice, here's what needs to be done:

Contact Legal

Before removing the crates, get in touch with legal services to discuss the request and whether we have any legal obligations.

Remove relevant version(s) and/or crate from crates.io

- Remove it from the database:

```
heroku run -a crates-io -- target/release  
[crate-name]
```

or

```
heroku run -a crates-io -- target/release  
[crate-name] [version-number]
```

- Remove the crate or version from the index. To remove a crate, remove the entire crate file. For a version, remove the line for the version.
- Remove the crate archive(s) and readme file(s)
- Invalidate the CloudFront cache:

```
aws cloudfront create-invalidation --dis  
paths '/*'
```

Remove entire crates from docs

The docs.rs application supports deleting all the docs by running a CLI command. The people who currently run the server and run it are:

- docs.rs Team:
 - [@pietroalbini](#)
 - [@jyn514](#)
- Infrastructure Team:
 - [@Mark-Simulacrum](#)
- People with elevated 1password access

You can find the documentation on how to run the

Database maintenance

There are times when Heroku needs to perform a maintenance on its instances, for example to apply system updates or update the database.

We must **not** let Heroku run maintenances during production hours, as this is disrupting production users (move the maintenance window to a time when the application is not in use). The maintenance window contains the instructions on how to perform the maintenance and the expected duration of disruption.

Primary database

Performing maintenance on the primary database requires the application to be in read-only mode. Heroku performs maintenance on the primary database follower and switching over to it, so we need to let the follower catch up.

Maintenance should take less than 5 minutes of read-only mode. We will announce it ahead of time on our status page. This is the expected maintenance window.

The crates.io team will perform a database maintenance on the primary database on 2025-02-22 from 14:00 to 14:05 UTC.

We expect this to take less than 5 minutes to complete. During this time, crates.io will only be available in read-only mode. The website will still work, but logging in, publishing new crates, and changing owners will not work.

Primary database checklist

1 hour before the maintenance

1. Go into the Heroku Scheduler and disable the database updater. You can “disable” it by changing its schedule to a time outside the maintenance window. The job uses a lot of database connections and running it during maintenance will cause issues.

5 minutes before the maintenance

2. Scale the background worker to 0 instances:

```
heroku ps:scale -a crates-io background_
```

At the start of the maintenance

3. Update the status page with this message:

Scheduled maintenance on our database is

We expect this to take less than 5 minutes t
crates.io will only be available in read-only m
visiting the website will still work, but login
crates, or changing owners will not work.

4. Configure the application to be in read-only m

```
heroku config:set -a crates-io READ_ONLY
```

The follower is removed because while Heroku
primary database from failing during mainten
not apply to the follower database, and there
follower is not available.

5. Wait for the application to be redeployed with

```
heroku ps:wait -a crates-io
```

6. Run the database maintenance:

```
heroku pg:maintenance:run --force -a cra
```

7. Wait for the maintenance to finish:

```
heroku pg:wait -a crates-io
```

8. Confirm all the databases are online:

```
heroku pg:info -a crates-io
```

9. Confirm the primary database fully recovered

```
echo "SELECT pg_is_in_recovery();" | heroku pg:psql -a crates-io DATABASE
```

10. Switch off read-only mode:

```
heroku config:unset -a crates-io READ_ONLY
```

WARNING: the Heroku Dashboard's UI is misleading. A red badge with a "-" (minus) indicates a configuration environment variable. A red badge with a "-" (minus) indicates a configuration environment variable has been *successfully removed*, it doesn't mean removing a configuration environment variable is indicated with a red badge with a "x" (cross) in the background.

11. Wait for the application to be redeployed with

```
heroku ps:wait -a crates-io
```

12. Update the status page and mark the maintenance finished

```
heroku status:page:set "Scheduled maintenance finished successfully"
```

The message is posted right now and not at the time of the maintenance. Production users are not impacted by the maintenance.

13. Scale the background worker up again:

```
heroku ps:scale -a crates-io background_worker=1
```

14. Confirm the follower database is available:

```
echo "SELECT 1;" | heroku pg:psql -a crates-io DATABASE
```

15. Enable connections to the follower:

```
heroku config:unset -a crates-io DB_OFFFL
```

16. Re-enable the background job disabled during

Follower database

Performing maintenance on the follower database of crates.io requires stopping application communication nor putting the application in read-only mode to prevent the loss of the follower's traffic to the primary database. It's a good idea to schedule these periods though, as we'll increase the primary database size in the future.

Follower database checklist

At the start of the maintenance

1. Configure the application to operate without the follower database:

```
heroku config:set -a crates-io DB_OFFFLIN
```

2. Wait for the application to be redeployed with the new configuration:

```
heroku ps:wait -a crates-io
```

3. Start the database maintenance:

```
heroku pg:maintenance:run --force -a cra
```

4. Wait for the maintenance to finish:

```
heroku pg:wait -a crates-io READ_ONLY_RE
```

5. Confirm the follower database is ready:

```
heroku pg:info -a crates-io
```

6. Confirm the follower database is responding to queries:

```
echo "SELECT 1;" | heroku pg:psql -a crates-io
```

7. Enable connections to the follower:

```
heroku config:unset -a crates-io DB_OFFLINE
```

8. Wait for the application to be redeployed with the new configuration:

```
heroku ps:wait -a crates-io
```

docs.rs

[docs.rs](#) is a website that hosts documentation for crates.

External Links

- Source code: [rust-lang/docs.rs](#)
- Hosted on: [docsrs.infra.rust-lang.org](#) (belonging to the Rust team)
- Maintainers: [docs.rs team](#)
- [Instance metrics](#) (only available to infra team members)
- [Application metrics](#) (only available to infra team members)

Add a dependency to the environment

Rustwide internally uses `rust-lang/crates-build-env` crate. If you want to add a system package for crate `foo`, for.

Preconditions

Docker and docker-compose must be installed. For

```
sudo apt-get install docker.io docker-compose
```

Getting started

First, clone the crates-build-env and the docs.rs repository.

```
git clone https://github.com/rust-lang/crates-build-env
git clone https://github.com/rust-lang/docs.rs
```

Set the path to the directory of your crate. This must be an absolute path! On platforms with coreutils, you can instead use `pwd` (relative to the docs.rs directory).

```
YOUR_CRATE=/path/to/your/crate
```

Add package

Next, add the package to `crates-build-env/linux/dependencies` in alphabetical order. This should be the name of a package in the Rust Repositories. See [the package home page](#) for a full list.

locally.

Building the image

Now build the image. This will take a very long time,

```
cd crates-build-env/linux
docker build --tag build-env .
```

Testing the image

Use the image to build your crate.

```
cd ../../docs.rs
cp .env.sample .env
docker-compose build
# avoid docker-compose creating the volume if it exists
if [ -e "$YOUR_CRATE" ]; then
    docker-compose run -e DOCSRS_DOCKER_IMAGE=build-env \
                        -e RUST_BACKTRACE=1 \
                        -v "$YOUR_CRATE":/opt/rustwide/work
    web build crate --local /opt/rustwide/work
else
    echo "$YOUR_CRATE does not exist";
fi
```

Making multiple changes

If your build fails even after your changes, it will be a waste of time to start from scratch just to add a single package. Instead, you can use the `docker-compose` Dockerfile so that the existing packages are cached. Change the `FROM` line from the Dockerfile to `packages.txt` once you are ready to build.

On line 7 of the Dockerfile, add this line: `RUN apt-get install -y`. Rerun the build and start the container; it should take a much shorter time.

```
cd ../crates-build-env/linux
docker build --tag build-env .
cd ../../docs.rs
docker-compose run -e DOCSRS_DOCKER_IMAGE=bu
                    -e RUST_BACKTRACE=1 \
                    -v "$YOUR_CRATE":/opt/rustwide/work
web build crate --local /opt/rustwide/work
```

Run the lint script

Before you make a PR, run the shell script `lint.sh`. `packages.txt` is in order and will tell you exactly what

```
cd ../crates-build-env
./lint.sh
```

Make a pull request

Once you are sure your package builds, you can make a pull request upstream for docs.rs and crater. Go to <https://github.com/rust-lang/crates-build-env> and click 'Fork' in the top right. Locally, add your fork as a remote and make changes:

```
git remote add personal https://github.com/<your_username>/crates-build-env
git add -u
git commit -m 'add packages necessary for <your_package>'
git push personal
```

Back on github, make a pull request:

1. Go to <https://github.com/rust-lang/crates-build-env>
2. Click 'compare across forks'
3. Click 'head repository' -> <your_username>/crates-build-env
4. Click 'Create pull request'
5. Add a description of what packages you added
6. Click 'Create pull request' again in the bottom right

Hopefully your changes will be merged quickly! After
release (rebuilds your docs immediately) or request
schedule a new build (may take a while depending on

Self hosting a docs.rs ins

These are instructions for deploying the server in a
instructions on developing locally without docker-co
[docker-compose](#).

Here is a breakdown of what it takes to turn a regul
docs.rs.

Beware: This process is rather rough! Attempts at cl
components, etc, would be greatly appreciated!

Requirements

The commands and package names on this page wi
systemd, but hopefully the explanatory text should
other systems. Note that docs.rs depends on the ho

Docs.rs has a few basic requirements:

- Rust (preferably via `rustup`)
- Git
- CMake, GCC, G++, and `pkg-config` (to build d
itself)
- OpenSSL, zlib, curl, and `libmagic` (to link agai
- PostgreSQL
- LXC tools (doc builds run inside an LXC contain

```
$ curl --proto '=https' --tlsv1.2 -sSf https:  
default-toolchain nightly  
$ source $HOME/.cargo/env  
# apt install build-essential git curl cmake  
libssl-dev zlib1g-dev postgresql lxc-utils
```

The cratesfyi user

To help things out later on, we can create a new user and process. This user will own all the files required by the process to be able to run `lxc-attach` through `sudo` to be able to edit the `sudoers` file at the same time:

```
# adduser --disabled-login --disabled-password --disabled-password --shell /bin/false --skeleton /etc/skel --system cratesfyi
# echo 'cratesfyi ALL=(ALL) NOPASSWD: /usr/bin/lxc-attach' > /etc/sudoers.d/cratesfyi
```

(The name `cratesfyi` is a historical one: Before the introduction of the “crates.fyi” instead. If you want to update the name, you can do so, but that the name `cratesfyi` will be used throughout the documentation.)

The “prefix” directory

In addition to the LXC container, docs.rs also stores the `crates` directory. This directory can be stored anywhere, but we need to be able to access it:

```
# mkdir /cratesfyi-prefix
# chown cratesfyi:cratesfyi /cratesfyi-prefix
```

Now we can set up some required folders. To make it easier, we can run them all as `cratesfyi`:

```
$ sudo -u cratesfyi mkdir -vp /cratesfyi-prefix/public_html /cratesfyi-prefix/sources
$ sudo -u cratesfyi git clone https://github.com/rust-lang/crates-index.git /cratesfyi-prefix/crates.io-index
$ sudo -u cratesfyi git --git-dir=/cratesfyi-prefix/crates.io-index --bare init
$ sudo -u cratesfyi git --git-dir=/cratesfyi-prefix/crates.io-index --bare branch crates-index-diff_last-seen
```

(That last command is used to set up the `crates-index-diff` monitoring new crate releases.)

LXC container

To help contain what crates' build scripts can access, we'll create an LXC container. To create one inside the prefix directory:

```
# LANG=C lxc-create -n cratesfyi-container --template ubuntu  
- --dist ubuntu --release bionic --arch amd64  
# ln -s /cratesfyi-prefix/cratesfyi-container /etc/default/lxc-net  
# chmod 755 /cratesfyi-prefix/cratesfyi-container  
# chmod 755 /var/lib/lxc
```

(To make deployment simpler, it's important that the container's OS is the same as the host! In this case, the host is assumed to be Ubuntu. You can make the container use a different release or distribution if you make the container use a different release or distribution separately inside the container when deploying.)

You'll also need to configure networking for the container. Edit `/etc/default/lxc-net` that enables NAT networking:

```
USE_LXC_BRIDGE="true"  
LXC_BRIDGE="lxcbr0"  
LXC_ADDR="10.0.3.1"  
LXC_NETMASK="255.255.255.0"  
LXC_NETWORK="10.0.3.0/24"  
LXC_DHCP_RANGE="10.0.3.2,10.0.3.254"  
LXC_DHCP_MAX="253"  
LXC_DHCP_CONFILE=""  
LXC_DOMAIN=""
```

In addition, you'll need to set the container's configuration file to `/cratesfyi-prefix/cratesfyi-container/etc/passwd`:

```
lxc.net.0.type = veth  
lxc.net.0.link = lxcbr0
```

Now you can reload the LXC network configuration, and enable the service to auto-start when the host boots:

```
# systemctl restart lxc-net  
# systemctl enable lxc@cratesfyi-container.service  
# systemctl start lxc@cratesfyi-container.service
```

Now we need to do some setup *inside* this container. Run `lxc exec cratesfyi-container -- /bin/bash` to open a root shell inside the container.

prefix.

```
# lxc-attach -n cratesfyi-container -- apt up
# lxc-attach -n cratesfyi-container -- apt up
# lxc-attach -n cratesfyi-container -- apt in
binutils gcc libc6-dev libmagic1 pkg-config
```

Inside the container, we also need to set up a `cratesfyi` user. In addition to the base Rust installation, we also need to install the `rust-docs` package so we can build docs for all the Tier 1 platforms. The `cratesfyi` container is the one that builds all the docs, so if you want to update this is the compiler to update.

```
lxc-attach -n cratesfyi-container -- adduser cratesfyi
lxc-attach -n cratesfyi-container -- adduser --password '' --gecos '' cratesfyi
lxc-attach -n cratesfyi-container -- su - cratesfyi -- curl -sSf https://sh.rustup.rs | sh -s -
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup update nightly
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup target add i686-apple-darwin
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup target add i686-pc-windows-msvc
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup target add i686-unknown-linux-gnu
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup target add x86_64-apple-darwin
lxc-attach -n cratesfyi-container -- su - cratesfyi -- rustup target add x86_64-pc-windows-msvc
```

Now that we have Rust installed inside the container, we need to add the `cratesfyi` user on the *host* the same Rust compiler. To add the following directories into its user directory, we don't need to

```
for directory in .cargo .rustup .multirust; do
  sudo cp -r /home/cratesfyi/$directory || sudo -u cratesfyi cp -r
  /var/lib/lxc/cratesfyi-container/rootfs/home/cratesfyi/$directory
done
```

Environment for the `cratesfyi` container

To ensure that the `docs.rs` server is configured properly, we need to

environment variables. The primary ones are going to be the ones that we can load them into the systemd service that will

Write the following into `/home/cratesfyi/.cratesfyi` token that the site can use to collect repository info. We can leave it blank. The variables need to exist, but they don't

```
CRATESFYI_PREFIX=/cratesfyi-prefix
CRATESFYI_DATABASE_URL=postgresql://cratesfyi:
CRATESFYI_CONTAINER_NAME=cratesfyi-container
CRATESFYI_GITHUB_USERNAME=
CRATESFYI_GITHUB_ACESSTOKEN=
RUST_LOG=cratesfyi
```

Now add the following to `/home/cratesfyi/.profile`

```
export $(cat $HOME/.cratesfyi.env | xargs -d\n)
export PATH="$HOME/.cargo/bin:$PATH"
export PATH="$PATH:$HOME/docs.rs/target/release"
```

Docs.rs build

Now we can actually clone and build the docs.rs source code. We can do much, but again, we want it to be owned by `cratesfyi` and be executable. In addition, we copy the built `cratesfyi` binary to `/var/lib/lxc/cratesfyi-container/rootfs/usr/bin` so it can be used to arrange builds on the inside.

```
sudo -u cratesfyi git clone https://github.com/cratesfyi/docs.rs
sudo su - cratesfyi -c 'cd ~/docs.rs && cargo build --release'
cp -v /home/cratesfyi/docs.rs/target/release/cratesfyi /var/lib/lxc/cratesfyi-container/rootfs/usr/bin
```

PostgreSQL

Now that we have the repository built, we can use it to create a Postgres database to store information about crates.

one up, we first need to ask Postgres to create the database. Then, we can run the command to create the initial tables and content:

```
sudo -u postgres sh -c "psql -c \"CREATE USER cratesfyi WITH PASSWORD 'password';\""
sudo -u postgres sh -c "psql -c \"CREATE DATABASE cratesfyi;\""
sudo su - cratesfyi -c "cd ~/docs.rs && cargo build"
sudo su - cratesfyi -c "cd ~/docs.rs && cargo build --release --features essential-files"
sudo su - cratesfyi -c "cd ~/docs.rs && cargo run --release --features rand 0.5.5"
sudo su - cratesfyi -c "cd ~/docs.rs && cargo run --release --features update-search-index"
sudo su - cratesfyi -c "cd ~/docs.rs && cargo run --release --features update-release-activity"
```

Server configuration

We're almost there! At this point, we've got all the pieces in place. We can set up a systemd service that will run the daemon, manage, orchestrate builds, and serve the website. The following is the content of the file placed in `/etc/systemd/system/cratesfyi.service`:

```
[Unit]
Description=Cratesfyi daemon
After=network.target postgresql.service

[Service]
User=cratesfyi
Group=cratesfyi
Type=forking
PIDFile=/cratesfyi-prefix/cratesfyi.pid
EnvironmentFile=/home/cratesfyi/.cratesfyi.env
ExecStart=/home/cratesfyi/docs.rs/target/release/cratesfyi
WorkingDirectory=/home/cratesfyi/docs.rs

[Install]
WantedBy=multi-user.target
```

Enabling and running that will serve the website on the local network. If you want to route public traffic to it, you'll need to set up a reverse proxy and connections to it.

Updating Rust

If you want to update the Rust compiler used to build (and run with it), you need to make sure you don't interrupt a build (it waits for 60 seconds between checking for new crates, so you can catch it during that window. Since we hooked the daemon to the available in its journal. Running `journalctl -efu cratesfyi` (as root if nothing appears) will show the latest log output. You're looking for a message like "Finished building" or "sleep" or "Queue is empty, going back to sleep", which indicates the thread is waiting.

To prevent the queue from building more crates, run:

```
sudo su - cratesfyi -c "cd ~/docs.rs && cargo lock --update
```

This will create a lock file in the prefix directory that prevents further builds. At this point, you can update the rustc inside the container and static files to the database:

```
lxc-attach -n cratesfyi-container -- su - cratesfyi -c "cd ~/docs.rs && cargo update --essential-files"
```

Once this is done, you can unlock the queue to allow builds to resume:

```
sudo su - cratesfyi -c "cd ~/docs.rs && cargo lock --update
```

And we're done! New crates will start being built with the new rustdoc, you need to manually rebuild any existing docs with the new rustdoc, you need to find an automated way to rebuild failed docs or docs from a previous version.

Updating docs.rs

To update the code for docs.rs itself, you can follow the instructions in the logs so you can stop the daemon from building more crates. You can stop the daemon with a `systemctl stop cratesfyi` if you want to be down while you build.)

```
# journalctl -efu cratesfyi  
(wait for build daemon to sleep)  
$ sudo su - cratesfyi -c "cd ~/docs.rs && cat
```

Once the daemon has stopped, you can start updating the docs.

```
$ sudo su - cratesfyi -c "cd ~/docs.rs && git pull  
$ sudo su - cratesfyi -c "cd ~/docs.rs && cargo
```

Now that we have a shiny new build, we need to make sure it's available to the public.

```
# cp -v /home/cratesfyi/docs.rs/target/release/cratesfyi  
/var/lib/lxc/cratesfyi-container/rootfs/usr/bin/cratesfyi  
# systemctl restart cratesfyi
```

Next, we can unlock the builder so it can start checking for updates.

```
$ sudo su - cratesfyi -c "cd ~/docs.rs && cargo build --release  
unlock"
```

And we're done! Changes to the site or the build will be reflected in the next update.

Common maintenance p

Temporarily remove a crate from

It might happen that a crate fails to build repeatedly in the queue and preventing other crates to build. In this case you can temporarily remove the crate from the queue until the docs.rs's build machine and open a PostgreSQL shell with:

```
$ psql
```

Then you can run this SQL query to remove the crate from the queue:

```
UPDATE queue SET attempt = 100 WHERE name = 'CrateName';
```

To add the crate back in the queue you can run in the PostgreSQL shell:

```
UPDATE queue SET attempt = 0 WHERE name = 'CrateName';
```

Pinning a version of nightly

Sometimes the latest nightly might be broken, causing builds to fail. It's possible to tell docs.rs to stop updating to the latest nightly release. To do that you need to edit the `/home/crate-builder/.env` file, changing this environment variable:

```
CRATESFYI_TOOLCHAIN=nightly-YYYY-MM-DD
```

Once the file changed docs.rs needs to be restarted:

```
systemctl restart docs.rs
```

To return to the latest nightly simply remove the environment variable again.

Rebuild a specific crate

If a bug was recently fixed, you may want to rebuild version. From the docs.rs machine:

```
cratesfyi queue add <crate> <version>
```

This will add the crate with a lower priority than new the priority with the `-p` option.

Raise the limits for a specific crate

Occasionally crates will ask for their build limits to be docs.rs machine with `psql`.

Raising a memory limit to 8 GB:

```
# memory is measured in bytes
cratesfyi=> INSERT INTO sandbox_overrides (crate_name, limit)
VALUES ('crate name', 8589934592);
```

Raising a timeout to 15 minutes:

```
cratesfyi=> INSERT INTO sandbox_overrides (crate_name, timeout)
VALUES ('crate name', 900);
```

Raising limits for multiple crates at once:

```
cratesfyi=> INSERT INTO sandbox_overrides (crate_name, limit)
VALUES ('stm32f4', 8589934592), ('stm32h7', 8589934592);
```

Set a group of crates to be auto

When many crates from the same project are published to space in the queue. You can de-prioritize groups of

```
cratesfyi=> INSERT INTO crate_priorities (pat
VALUES ('group-%', 1);
```

The `pattern` should be a `LIKE` pattern as documente
<https://www.postgresql.org/docs/current/functions>

Note that this only sets the default priority for crate
already in the queue, you'll have to update those m

```
cratesfyi=> UPDATE queue SET priority = 1 WH
```

Adding all the crates failed after queue

After an outage you might want to add all the failed
log into the machine and open a PostgreSQL shell w

```
psql
```

Then you can run this SQL query to add all the crate
back in the queue:

```
UPDATE queue SET attempt = 0 WHERE attempt >=
HH:MM:SS';
```

Removing a crate from the webs

Sometimes it might be needed to remove all the cor
(for example after receiving a DMCA). To do that, log

```
cratesfyi database delete-crate CRATE_NAME
```

The command will remove all the data from the dat
S3.

Blacklisting crates

Occasionally it might be needed to prevent a crate from being built, for example if we can't legally host the content of those crates. When preventing new builds for it, you can run:

```
cratesfyi database blacklist add <CRATE_NAME>
```

Other operations (such as `list` and `remove`) are also available.

Warning: blacklisting a crate doesn't remove existing builds, it just prevents new versions from being built!

Governance

Leadership Council

The Leadership Council is a representative group of the Rust community, tasked with coordinating between teams and to ensure the success of the Project.

The policies governing the Leadership Council are specified in [this](#) chapter.

Moderation

The Moderation team is responsible for dealing with issues related to [Community Conduct](#).

The policies governing the Moderation team are specified in [this](#) chapter.

Leadership Council

This document defines the authority¹ and policies of the Rust Foundation (“Council”) to ensure successful operation of the Rust Foundation.

This document serves as a living document defining the authority governing the Council. The basis of this document is the authority established the Council, and may be updated via the Council’s decision-making process.

The Council delegates much of this authority to teams (e.g., working groups, etc.²) who autonomously make decisions coordinated by the Council retains some decision-making authority over the Rust Foundation document.

The Council maintains a separate home site at <https://council.rust-lang.org> where they document their internal processes.

The Council is composed of representatives delegated by the Rust Foundation team.

The Council is charged with the success of the Rust Foundation, identifies work that needs to be done but does not directly manage teams to accomplish this work, holds existing teams accountable, purview, and coordinates and adjusts the organization’s strategy.

Outline

- [Motivation](#)
- [Duties, expectations, and constraints on the Council](#)
- [Structure of the Council](#)
 - [Top-level teams](#)
 - [The Launching Pad top-level team](#)
 - [Removing top-level teams](#)
 - [Alternates and forgoing representation](#)
 - [Term limits](#)
 - [Limits on representatives from a single company](#)
 - [Candidate criteria](#)
 - [Credentials](#)
 - [Relationship to the Rust Foundation](#)

- The Council's decision-making process
 - Operational vs policy decisions
 - Repetition and exceptions
 - The consent decision-making process
 - Approval criteria
 - Modifying and tuning the decision-making process
 - Agenda and backlog
 - Deadlock resolution
 - Feedback and evaluation
- Transparency and oversight for decision making
 - Decisions that the Council may make internally
 - Decisions that the Council must necessarily make
 - Decisions that the Council must make via consensus
 - Conflicts of interest
 - Determining and changing team purview
- Mechanisms for oversight and accountability
 - Ensuring the Council is accountable
 - Ensuring Council representatives are accountable
 - Ensuring teams are accountable
- Footnotes

Motivation

The Rust project consists of hundreds of globally distributed teams with various purviews. However, a great deal of work remains to be done, and still needs to get done.

The Council focuses on identifying and prioritizing work. The Council primarily delegates that work, rather than doing it themselves. The Council also serves as a coordination, organization, and accountability mechanism for cross-team efforts, roadmaps, and the long-term vision.

Duties, expectations, and the Council

At a high-level, the Council is *only* in charge of the following:

- Identifying, prioritizing, and tracking work that has no clear ownership (and not due to the owners' explicit backlog, etc.).
- Delegating this work, potentially establishing new teams to own this work.
- Making decisions on *urgent* matters that do not have a clear owner
 - This should only be done in exceptional circumstances and cannot be delegated either to existing teams or to the Council.
- Coordinating Project-wide changes to teams, such as reorganizing or creating new teams.
- Ensuring top-level teams are accountable to the Project and the Project's goals.
- Ensuring where possible that teams have the power to accomplish their work.
- Establishing the official position, opinion, or will of the Project
 - This helps reduce the need for Project-wide decisions, long public polling and consensus-building, and reduces the example, when communicating with third parties, of the understanding of what the Rust Project actually is.

In addition to these duties, the Council has additional responsibilities that help determine if the Council is functioning properly:

- *Delegate work:* The Council should not take on work that is not explicitly assigned to it; it must delegate to existing teams or the Council. Such teams may include Council representatives, but are not part of the duties of a Council representative.
- *Ensure the Project runs smoothly in the long term:* The Council should ensure that urgent Project management work is prioritized and that there is a regularity that the Project does not accumulate work.
- *Be Accountable:* As the Council wields broad power, its representatives must be accountable for their actions, listen to feedback, and actively reflect on whether they are meeting the expectations of the position they hold.
- *Be representational:* Council representatives should represent all of Project concerns but also the diversity of the community, as possible (demographics, technical background, etc.).
- *Share burden:* All Council representatives must share the burden of the Council's work.
- *Respect others' purviews:* The Council must respect the purview of other teams. The Council should consult with and work together on overlapping issues, and should almost never make decisions that are purely within the purview of another team.

given team.

- *Act in good faith:* Council representatives should act on behalf of the Rust Project *as a whole* even if those decisions affect individual teams, their employers, or other organizations.
- *Be transparent:* While not all decisions (or all actions) need to be public, the Council should be as open and transparent as possible. The Council should also ensure that the Project is clear and transparent.
- *Respect privacy:* The Council must never compromise privacy for the sake of transparency, including but not limited to unintentionally disclose privileged information.
- *Foster a healthy working environment:* The Council should ensure that members are satisfied with the amount and nature of their work. If a member's presence on the Council is merely out of obligation, they are not actively participating in a meaningful way.
- *Evolve:* The Council is expected to evolve over time as teams, the Project, and the community change.

Council representatives, moderation team members, and other leaders should serve as examples for those around them and the broader community. They represent positions of responsibility and leadership and should exert great force within the community, and should recognize that choosing to serve in these roles should thus recognize that they are held to a correspondingly high standard.

Structure of the Council

The Council consists of a set of team representatives from each top-level [team](#) and its subteams.

Each top-level team designates exactly one representative to the Council.

Any member of the top-level team or a member of a subteam can be designated the representative. Teams should provide members with the opportunity for input and feedback on potential candidates.

Each representative represents at most one top-level team and any subteams of other teams. The primary responsibility of representatives is to act on behalf of the top-level team they fall under.

All teams in the Rust Project must ultimately fall under the [Launching Pad team](#) serves as a temporary home for teams without a parent team. This ensures that all teams have representation.

Top-level teams

The Council establishes top-level teams via public proposal. All top-level teams should meet the following criteria:

- Have a purview that is foundational to the Rust Project
- Be the ultimate decision-makers on all aspects of the team's work
- Have a purview that is not a subset of another team's (e.g., a subteam or similar governance structure)
- Have an open-ended purview that's expected to last for the long term
- Be a currently active part of the Rust Project

There must be between 4 and 9 top-level teams (including the Launching Pad team). This number balances the desire for a diverse and rich set of teams while being practical for productive conversation and coordination.

When the Council creates a new top-level team, that team must be representative.⁵ When creating a new top-level team, the Council must provide justification for why it should not be a subteam or a subcommittee.

The set of top-level teams is:

- Compiler
- Dev tools
- Infrastructure
- Language
- Launching Pad
- Library
- Moderation

The Launching Pad top-level team

The Launching Pad team *temporarily* accepts subteams and subcommittees.

level team to slot underneath of. This ensures that a Council, while more permanent parent teams are for

The Launching Pad team is an umbrella team: it has representatives.

The Council should work to find or create a more appropriate team for the Launching Pad, and subsequently move those subteams

In some cases, an appropriate parent team may exist; in others, subteams; the Launching Pad can serve as an interim

The Launching Pad also serves as a default home for subteams or reorganized away, if that removal or reorganization is not subteams somewhere else in the organization.

The Council must review subteam membership in the Launching Pad to ensure that proper progress is being made on finding a home for them with other top-level teams, the Launching Pad team must have representation within the Council removed) if the Council decides it is necessary. The process for retiring the Launching Pad team is the same as for top-level teams. Alternatively, the Council is free to give the Launching Pad team purview.

Removing top-level teams

Any decision to remove a team's top-level designation (from the Council) requires the consent of all Council representatives. A representative of the top-level team being removed must be present. A representative of the team under consideration must also be present. Concerning the team's removal, and the Council should consider objections in extreme cases.

The Council cannot remove the moderation team. The Council cannot remove the moderation team's purview without the agreement of the moderation team.

Alternates and forgoing representation

A representative may end their term early if necessary due to availability or circumstances. The respective top-level team may then select a new representative. The role of representative is a voluntary position to fill that role, and no team is permitted to make selection a requirement or obligation of membership in a team. However, a representative is expected to fulfill the duties of the position of representative, or resign the position if they cannot.

A top-level team may decide to temporarily relinquish their representative if the team is temporarily understaffed and they have no one else to designate. If a team does not designate a Council representative, they are not permitted to participate in decision-making at a Project-wide level. However, the Council decision-making should not be blocked due to this. The Council must continue to consider new information and objections from all teams. The Council is not obligated to block decisions to special interest groups or a represented team's feedback.

Sending a representative to the Council is considered a commitment. If a team is being able to regularly do so means the team is not neglecting their Council representative does not relinquish their role. Absences due to temporary illness, vacation, etc.

A top-level team can designate an alternate representative if their primary representative is unavailable. This alternate representative must be designated before the primary representative returns. The alternate representative until the return of the primary representative. The alternate representative do not regularly attend meetings when the primary representative is absent (doubling the number of attendees).

If a team's representative *and* any alternates fail to participate in Council meetings for 3 consecutive weeks, the team's representative cannot make quorum requirements of the Council until they return. The team is not able to participate. The Council must notify the team if they fail to participate. The team wishes to ensure the Council does not make decisions without an ability for objections to be made on their behalf. The team must have an alternate representative available.

A top-level team may change their representative if necessary. However, as maintaining continuity is important, changing their representatives more than necessary is discouraged. The team is responsible for briefing their representative and a team representative is responsible for positions they wish to handle on an ongoing basis. The team is responsible for maintaining continuity for ongoing projects.

providing context to alternates and other new repre

For private matters, the Council should exercise discretion to avoid spreading private information unnecessarily; it may need to step in.

Term limits

Council representatives' terms are one year in length (up to two of three consecutive full terms for any given representative from a particular top-level team). A representative may be re-elected if the Council receives explicit confirmation from the relevant team to produce a different team member as a representative (or a willing alternative candidate, or due to team member unavailability or other candidate).

Beyond this, there is no hard limit on the number of representatives from other top-level teams or non-consecutive terms for any individual. We strive for a balance between continuity of experience and fresh perspectives, provide multiple people with such experience.⁶

Half of the representative appointments shall happen at the end of September. This avoids changing the composition at the same time. For the initial Council, and anytime the size of the Council and top-level teams should work together to ensure appointments are divided between March and September. However, emergency appointments of 6 months (temporary imbalance is acceptable to maintain balance).

If the Council and top-level teams cannot agree on a balanced distribution, representatives are randomly assigned to one or the other (or both out) to maintain balance.

Limits on representatives from a particular team

Council representatives must not disproportionately represent a single entity, or closely related set of legal entities, to avoid conflict of interest or impropriety. If the Council has 5 or fewer represent

may have any given affiliation; if the Council has 6 or more representatives, 2 representatives may have any given affiliation.

Closely related legal entities include branches/divisions/subsidiaries/entities connected through substantial ownership in order to make a judgment call in unusual cases, taking care to avoid a conflict of decision.

A Council representative is affiliated with a company if they receive a substantive fraction of their income from that entity (or are a major sponsor). Representatives must promptly disclose any such affiliation.

If this constraint does not hold, whether by a representative's own choice, teams appointing new representatives, or the Council's decision, the constraint as follows:

- Representatives with the same affiliation may not be appointed amongst themselves, such that a representative's team appoints someone else.
 - This must be a decision by the representative's team; it is not considered improper for the affiliated entity to influence the decision.
 - Representatives have equal standing in the Council regardless of seniority in the Project or the Council membership.
- If the representatives with that affiliation cannot be removed at random. (If the constraint still does not hold, representatives may again attempt to resolve the conflict by repeating this.) This is likely to produce suboptimal results; typically be preferable.
- While a team should immediately begin the process of appointing a new team's existing representative may continue to serve the remaining term.
- The existing representative should coordinate the appointment of a new representative but it is the team's choice which representative to appoint during the up to 3 month window. There is only one top-level team.

Candidate criteria

The following are criteria for deciding ideal candidates for the Council.

same as the criteria for an effective team lead or co-ordinator. To make a good Council representative, serving as a team lead or representative both require a substantial time investment. These roles among different people. The criteria are used for determining who is best positioned to be a Council representative should have:

- sufficient time and energy to dedicate to the role.
- an interest in helping with the topics of Project Rust.
- broad awareness of the needs of the Project and the ability to make an active contribution.
- a keen sense of the needs of their team.
- the temperament and ability to represent and not just their own personal agenda.
- ability and willingness to represent all viewpoints, not just those they agree with.

While some teams may not currently have an abundance of these criteria, the Council should actively foster such skills. This is helpful not only for Council membership but across the project.

Credentials

The Council does not have privileged access to administrative tools. This access solely resides with the infrastructure team. The team's responsibilities include ensuring teams have the tools they need to work effectively, while balancing against security and privacy concerns in infrastructure. The Council can help coordinate which tools are used and policy.

Relationship to the Rust Foundation

The Council is responsible for establishing the process for the Rust Project directors are the mechanism by which the Rust Foundation board.

The Council delegates a purview to the Project directed by the interests on the Foundation Board and to make certain matters. The exact boundaries of that purview are not

The Council's decision-making

The Council make decisions of two different types: operational and policy decisions. Certain considerations may be placed on a specific classification. However, by default, the Council uses the same process for all decisions regardless of classification.

Operational vs policy decisions

Operational decisions are made on a daily basis by the Council, including regular actions taking place outside of meetings. Policy decisions provide general reusable patterns of operations and support operations. In particular, policy decisions are made on operational decisions or other aspects of operations. The Council uses the same decision making process for all decisions unless otherwise specified.

It is not defined precisely which decisions are operational or policy, somewhere along a continuum. The purpose of this classification is to constrain the council's decision-making procedures, provide guidance to the Council, and clarifies how the Council can refine its decisions over time. For the purposes of a specific decision, associated with the operational/policy classification, the Council is operational or policy in this or future policy defaults.

Repetition and exceptions

Policy decisions often systematically address what is not operational decisions. The Council should strive to make decisions indicate the need for a policy decision, or the Council should avoid allowing repeated operational decisions to become policy.

Exceptions to existing policy cannot be made via an exceptions are explicitly allowed in said policy. Avoid “normalization of deviance”.

The consent decision-making process

Consent means that no representative’s requirement (team and subteams they represent) can be disregarded. It takes everyone’s input and sets a good foundation for working together equally.

The Council uses consent decision-making where in representatives are asked “do you object?”. This eliminates the need to have fully reviewed a proposal but decide against approval. Feedback as to the reason. Concerns, feedback, pre-emptive objections of feedback do not prevent making a decision, but suggest incorporation earlier in drafting and discussion. Objections are a requirement or need, *must* be considered and resolved.

Approval criteria

The consent decision-making process has the following criteria:

- Posting the proposal in one of the Council’s decision-making channels (meeting or a specific channel).
- Having confirmation that at least N-2 Council representatives (a minimum number of Council representatives) have fully given their consent.
- Having no outstanding explicit objections from Council representatives.
- Providing a minimum 10 days for feedback.

The approval criteria provides a quorum mechanism for Council representatives to have seen the proposal. Allowing for the acknowledgement of the volunteer nature of the Process while maintaining the speed of decisions with the amount of confirmation required. An objection; this assumes that those representatives have the time to do so. (This is modeled after the process used to

The decision-making process can end at any time if a representative decides to retract their proposal. Another representative can also choose to keep it alive.

If conflicts of interest result in the Council being unable to make a decision, the Council cannot make that decision until the conflict is resolved. See [in the “Conflicts of interest” section for how a decision is documented](#). In such a case, the Council should consider policies to avoid future recurrences of a similar conflict.

Modifying and tuning the decision-making process

Using the public policy process, the Council can establish policies and processes for classes of decisions.

When deciding on which decision-making process to use for a particular decision, the Council balances the need for quick decision-making with the need for confidence in full alignment. Consent decision-making falls on the middle of the spectrum:

- Consensus decision making (prioritizes confidence in alignment over quick decision making): team members must agree on all points, and all others, any team members may raise a blocking objection
- Consent decision making (default for the Council): team members must agree on all points, but one team member may raise a blocking objection
- One second and no objections (prioritizes quick decision making over confidence in alignment): one team member may raise a blocking objection

Any policy that defines decision-making processes must specify how a proposal may be posted, quorum requirements, number of members required for a minimum time delay for feedback. A lack of objection is required for all decision-making processes.

If conflicts of interest prevent more than a third of the Council from making a decision, the Council cannot make that decision until the conflict is resolved. See [in the “Conflicts of interest” section for how a decision is documented](#). (This is true regardless of any other quorum requirements.)

making process in use.) In such a case, the Council should review its processes and policies to avoid future recurrences of such a situation.

The Council may also delegate subsets of its own decision-making or policy decision, to teams, other governance structures, or the Council, such as operational lead, meeting facilitator, etc.

Note that the Council may delegate the drafting of a proposal, while delegating the decision to approve that proposal. The Council should avoid Project-wide policy that intersects the purviews of multiple teams or purview of any team. This may also help when bootstrapping a new team.

Agenda and backlog

The Council's agenda and backlog are the primary mechanisms for tracking and gives progress updates on issues raised by the Project.

To aid in the fairness and effectiveness of the agenda and backlog:

- Use a tool that allows Project members to submit requests and receive updates on those requests.
- Use a transparent and inclusive process for deciding what to put in the upcoming period. This must involve regular communication with representatives.
- Strive to maintain a balance between long-term needs in the backlog and on the agenda.
- Be flexible and adaptable and be willing to adjust the agenda as needed in response to changing circumstances.
- Regularly review and update the backlog to ensure it reflects current priorities and goals of the Council.
- Follow a clear and consistent process for moving items from the backlog to the agenda, such as delegating responsibility to a team (or scribe), and consenting to the agenda at the start of the meeting. Items rejected during the consent process must have been published meeting minutes of the Council.

Deadlock resolution

In some situations the Council might need to make a decision that everyone can construct a proposal in that time that everyone can agree to. If everyone agrees that a timely decision is better than no timely decision at all, the Council may use a voting system to attempt to resolve the deadlock. The alternative is that the Council members must still re-affirm their consent to the ongoing decision-making process. Council members may still raise objections.

For example, the Council can consent to a vote, then the Council members would consent to whatever decision is made. The Council should strive to document the perceived advantages of a particular alternative decision-making model.

There is, by design, no mandatory mechanism for decision-making. If representatives do not all consent to making a decision, or if the outcome of that decision, or if any representative feels that the decision is not in the proposal that will garner the Council's consent, they can raise objections.

If a representative withdraws an objection, or consents to a decision, or agrees with (whether as a result of an alternative decision-making model) the Council should schedule an evaluation or consider the already scheduled evaluation, and should establish a mechanism to address concerns voiced. The results of this review are intended to be used by the Council should consider changing its prior decision.

Feedback and evaluation

All policy decisions should have an evaluation date and a review period. The periods should be shorter in duration than subsequent evaluation periods should be adjusted based on the results of the evaluation. If a policy seems to be working well and require few changes should be reviewed. Policies that have been spent on unnecessary reviews. Policies that have been questioned should have shortened evaluation periods to ensure stability more quickly. The Council should establish a policy to use as defaults when determining periods

could have an evaluation date of 3 months initially to the policy could default to 6 months initially and 2 years

- New policy decisions can always modify or replace
- Policy decisions must be published in a central
- Modifications to the active policy docs should be made to the policy decision, rather than expecting people

Transparency and oversight in decision making

Decisions made by the Council will necessarily require some level of oversight based on the kind of decision being made. The Council will seek oversight for its decisions, and this can be in private or in public.

This RFC places certain decisions into each category. Decisions enumerated must use the public policy process. The categorization through the [public policy process](#).

Decisions made by the Council fall into one of three categories of oversight possible and necessary:

- Decisions that the Council may make internally
- Decisions that the Council must necessarily make via public
- Decisions that the Council must make via public

Decisions that the Council may make internally

Some types of operational decisions can be made internally with the provision that the Council has a mechanism for coming back and reviewing after it has been made.

Adding a new decision to the list of decisions the Council must make via public policy decision. Any decisions that impact the Council's oversight of the Council itself should not be added to the list.

The Council should also strive to avoid establishing

internal decisions in an effort to avoid public proposals for more details.

This list exhaustively enumerates the set of decisions made internally:

- Deciding to start a process that itself will play a role in “developing and posting the survey”, “let’s draft a decision”).
- Expressing and communicating an official position.
- Expressing and communicating the position of the Rust entity, such as the Rust Foundation.
- Communicating via Rust Project communication channels.
- Making most operational decisions about the Rust Project (including how the Council coordinates, the plan of action and when it meets, templates used for making decisions, requirements elsewhere in this document).
- Appointing officers or temporary roles within the Council (leading/facilitating meetings, recording and publishing, collating feedback from various parties, etc.⁸ Note that past and current holders) must be publicly disclosed.
- Inviting specific attendees other than Council members to meetings or discussions, or holding a meeting or discussion. In particular, the Council is encouraged to invite specific people to meetings or discussions where said decisions are made.
- Making decisions requested by one or more teams within the purviews of those teams to make without a public proposal or ask for Council input without requesting a Council decision.
- Making one-off judgment calls in areas where the rules are ambiguous (though *changing* the purviews of the Council is a decision).
- Any decision that this document or future Council documents make a decision.

See the [accountability section](#) for details on the feedback on decisions.

Decisions that the Council must privately

Some decisions necessarily involve private details or making these details public would have a negative impact on entities (e.g. safety) and on the Project (eroding trust).

This additional constraint should be considered an exception to making [decisions that would require a public proposal](#). The Council does permit decisions that the Council makes internally, with information provided for public oversight.

The Council may also decline to make a decision private if the matter is outside their purview (and chooses to decide the matter should be handled publicly. However, even if the Council publicly reveal information shared with it in confidence, the Council would not be trusted to receive such information). Confidentiality threats to safety.

Private decisions must not establish policy. The Council must not be establishing de facto unwritten policy via repeated private decisions or public proposal. See [“Repetition and exceptions”](#) for more.

This list exhaustively enumerates the set of decisions that the Council may make partly or entirely in private:

- Determining relationships with new industry / academia / community / confidentiality before launching.
- Discussing the personal aspects of a dispute between team members / interpersonal dynamics/conflicts.
- Participating in contract negotiations on behalf of the Project (accepting resources provided to the Project).
- Decisions touching on Project-relevant controversy, safety, or other topics in which people may not want to be involved.
- Discussing whether and why a team or individual may touch on personal matters.
- Any decision that this document or future Council decisions may be affected by.

The Council may pull in members of other teams for

a private or public decision, unless doing so would require information disclosed to the Council without permission. The Council should attempt to pull in people or teams affected by the decision for additional oversight.

Some matters may not be fit for full public disclosure but may be fit for smaller, more trusted circles (such as with all Project decisions involving/affected parties). The Council should strive to provide appropriate audiences for that information.

The Council may decide to withhold new decisions or information until it is unclear whether the information is sensitive. However, as it becomes clearer who the appropriate audience is or if the audience is expanded, the council should revisit its information.

The Council should always loop in the moderation team in cases of interpersonal conflict/dispute, both because such matters require moderation team, and to again provide additional oversight.

The council should evaluate which portions of a decision are not necessarily need to be private, and should consider making sensitive portions public, rather than keeping an entire decision private if a portion of it needs to be. This may include the existing decision on a topic, if those details are not themselves sensitive.

Private matters may potentially be able to become public if they're no longer sensitive. However, some matters may never become public, which means they will never become public. Thus, the Council must exercise caution and oversight in making a decision.

The Council should make every effort to not make private decisions public. They should have appropriate additional processes in place to ensure that they can collectively review such decisions and consider their impact.

Decisions that the Council must review before proposal

Decisions in this category require the Council to publicly review the proposal before it is made.

Rust Project *in advance* of the decision being made. decided via the appropriate public decision process the Council may adopt a different public proposal p decision process must require the consent of repres non-objection), must allow for blocking objections b provide reasonable time for public evaluation and o path for public feedback to the Council.

Following the existing RFC process, public proposals feedback before the decision takes effect. Any repre feedback period for a particular decision is extended may make an internal operational decision to extend days. The time-delay for feedback starts only when i is otherwise met, including there not being any raise and resolved during the time-delay, the waiting per

The Council is expected to evolve over time to meet Rust Project, and the community. Such evolutionary scope and require corresponding amounts of overs the shape of the Council would need to be part of a

As an exception to the above, modifications or remo than the moderation team) may occur with the una absent the representative delegated by that top-level

The Council is permitted to have private *discussions* ends up as a public proposal or a publicly disclosed wish to do this if the discussions are sensitive to allo more frankly and freely. Additionally, in some cases disclosed may impact an otherwise public decision/ be as transparent and non-misleading as possible a where all rationale is private.

Note that all decisions fall into this category unless (document or future public proposals) to fall into any those in the other two categories) is intentionally va guidance on what likely should belong in this category prescriptive.

- Any decision that has the effect of modifying the Council or the decision-making process of the

- Changing this list (or this document in general).
- Modifying the publication and approval process for proposals. Such a proposal must use the proposed process.
- Adding, modifying, or removing policies and representatives.
- Adding, modifying, or removing one or more
 - modifying the purview of a top-level team so that it meaningfully becomes a different team.
 - reorganizing the Project such that teams are merged into other teams.
- Adding other types of Council representatives to the top-level teams.
- Adding, modifying, or removing policies and locations in which binding decisions can be made.
- Any policy decision, as opposed to a one-off operational [making section](#) for details on policy decisions, includes any decision that binds the decisions of other teams or individuals), effectively serving the purviews of all teams. Some examples of policy decisions include:
 - Modifying or extending existing policies, such as the RFC.
 - A legal/licensing policy affecting Rust Projects or the Rust Project.
 - A change to the Code of Conduct.
 - A policy affecting eligibility for membership in the Rust Project.
 - A change to how the moderation team moderates the Council as a whole. Such decisions must be made by the moderation team.
 - An agreement with another project or organization on commitments on behalf of the Rust Project or Rust Projects that have agreed to those commitments.
 - Creating or substantially modifying legal entities (e.g., Foundations, changing relationship with other legal entities).
 - Making policy decisions requested by one or more of the normal purviews of those teams. (Not a one-off decision without requesting a Council decision).

- Deciding that a class of future decisions a rather than being delegated to any other
- Any decision that this document or future Cou decision.

Conflicts of interest

A Council representative must not take part in or in a conflict of interest.

Potential sources of conflicts of interest include, but

- Personal: a decision about themselves
- Financial: a decision with any substantive finan
- Employment or equivalent: a decision involves company, or would benefit/harm that compan others
- Professional or other affiliation: a decision inv representative is associated with, such as an industry/professional/standards/governmenta
- Familial/Friendship: a decision about a person expected to be impartial about, including a cor through that person (such as a family member

Council representatives must promptly disclose con themselves from affected decisions. Council repre disclose likely sources of potential conflict annually moderation team.

Note that conflicts of interest can arise even if a pro Council representatives cannot, for instance, use th proposal to disproportionately benefit their employ

A proposal favored widely across the Rust commun conflict of interest for a representative merely beca equivalent also favors the general area of that prop favor any particular entities. For example, a proposa particular Rust component is not a conflict of intere their employers generally care about Rust security;

specific developers or security experts, or one's conflict of interest in a proposal, might still raise a conflict.

The Council may not waive a conflict of interest if or considers it minor. However, the Council may evaluate. Council representatives must raise potential conflict determination.

The Council may request specific information from a conflicted or recused representative may provide that information.

Where possible and practical, the Council should seek to avoid a conflict of interest. For instance, the Council could limit access to a class of hardware (without setting specific rules) from the decision of which exact hardware to purchase, so made a conflict of interest only apply to the latter.

A representative simultaneously considering the interests of any Project team is not necessarily a conflict of interest. Council representatives are *expected* to regularly take part in decisions and delegates from those teams.

In the unlikely event that a proposed decision produces a conflict of interest for representatives that the remainder cannot meet a quorum requirement, and the decision must still be made, the Council may provide alternate representatives for the purposes of making decisions only) the Council may elect to proceed with the decision while documenting all conflicts of interest. (Note that proceeding with conflicts documented, does not actually eliminate the conflict, influencing the decision; it only allows the public to see that it has influenced the decision. Eliminating the conflict of interest in such a case, the Council should consider appropriate measures to prevent future recurrences of a similar conflict.

Determining and changing team structure

The Council can move an area or activity between teams, or create already existing or newly created (other than the main team) of a given top-level team may be further sub-divided.

moves or adjusts top-level purviews. If a sub-divided work with the involved teams to coordinate the app should be used when the Council believes the exist that it is not feasible to expect the team to fulfill the structure. However, this should not happen when a perform part of its duties.

The Council also must approve expansions of a top-notified of reductions in a top-level team's purview. team self-determines that they wish to expand or re happen as part of top-level teams agreeing to adjust Council awareness of changes to a purview is neces purview can be re-assigned elsewhere or intentiona

However, teams (individually or jointly) may further without approval from the Council. Top-level teams purviews assigned to them, even if they delegate (in for ensuring the delegation is successful).

The Council should favor working with teams on alt purviews between teams, as this is a relatively heavy that one of the use cases for this mechanism is shift a team that functionally no longer exists (for instanc time), potentially on a relatively temporary basis un ability to re-create that team. This section intentiona Council for exactly how (or whether) this consultatio

Mechanisms for oversight accountability

The following are various mechanisms that the Cou accountable.

Ensuring the Council is accounta

The Council must publicly ensure that the wider Pro the Council are consistently being met. This should

policies, procedures, and outcomes of the Council and the community when their expectations are not aligned.

To achieve this, in addition to rotating representative orientation, the Council must regularly (at least on a quarterly basis) have a widely available public communication on their activities and how well the Council is functioning using the list of duties and criteria for this evaluation.

Each year, the Council must solicit feedback on whether it is effectively from all willing and able Project members. This can be a forum that allows and encourages active participation. Additionally, so, the Council and other Project members consult with the community and constraints listed in this document and any subsequent updates to determine if the Council is meeting its duties and obligations.

In addition, it is every representative's *individual* responsibility to refuse to go along with failures to follow this document's policies, procedures, or any other aspects of Council accountability. To actively avoid “diffusion of responsibility”, the phenomenon where collectively fail to do something because each individual (often subconsciously) believes that someone else will do so, the Council should designate a specific role with the responsibility of handling procedural matters, and in particular raising procedural points. All representatives should still do so as well.

If any part of the above process comes to the conclusion that the Council is not meeting its obligations, then a plan for how the Council will correct these obligations must be presented as soon as possible. This plan, whether in a charter or similar, a rotation of representatives, or other mechanism, should have concrete measures for how the Council will evolve in light of the previous year's experience.

Ensuring Council representative

Council representatives should participate in regular evaluations of their respective top-level team (the nature of which is defined in the document) to reflect on how well they are fulfilling their duties. The goal of the feedback session is to help representatives

better serve the Project. This feedback must be shared with all members of the representative's top-level team, and the feedback should ask for both what representatives have done better.

Separately, representatives should also be open to feedback from fellow representatives at any time, and should regularly report on their role and efficacy on the Council.

Artifacts from these feedback processes must never be used in an open process. The Council should also reflect on any negative results do not lead to positive change.

If other members of the Council feel that a Council representative is not well with the rest of the Council, they should talk to that representative's team. Council representatives should use moderation/mediation resources as needed to facilitate discussion, can help resolve the issue, and/or determine if the issue requires some level of escalation.

While it is out of scope for this document to specify how representatives are held accountable, we encourage teams to use this as inspiration for their own policies and procedures.

Ensuring teams are accountable

Teams regularly coordinate and cooperate with each other about their needs; under normal circumstances the needs of individual teams.

However, the Council serves as a means for teams to coordinate with one another and to the Project as a whole. The Council should:

- Ask a team to reconsider a decision that failed to take into account the needs of teams or the Project as a whole into consideration.
- Encourage teams to establish processes that require mutual consideration.
- Ensure a shared understanding of teams' purview and responsibilities.
- Ensure teams are willing and able to fulfill those responsibilities.

- Establish new teams that split a team's purview

The accountability process must not be punitive, and active collaboration of the teams in question.

In extreme circumstances where teams are willfully regards to the wider Project, the Council has the authority to move some subset of a team's purview to another team. This is done through the Council's regular decision making process (see the moderation team; see the next section for accountability and moderation team.)

Footnotes

¹ The term 'authority' here refers to the powers and responsibilities for the success of the Rust Project. This document lays out the limits of authority that will be delegated to teams responsible for various aspects of the Project. Concerns may include - but are not limited to - product vision, decisions, mentoring, and marketing.

² Throughout this document, "teams" includes subteams, initiatives, and all other forms of official collaboration structures. This includes all forms of collaboration structures that report to the Council.

³ Subteams or individuals that fall under multiple top-level teams should have representation by having multiple representatives speaking for them. A "diamond" structure like this exists anywhere in the organization. The organization should strive to avoid ambiguity or diffusion of responsibility. Teams should know what paths they should use to raise issues and concerns.

⁵ The Council consists only of the representatives provided by the teams. It can appoint new ad hoc members to itself. However, if the Council decides to create a new top-level team. In particular, the Council can address a problem for which the Project doesn't currently have a solution and for which the Council doesn't know the right solution. In that case, the Council could bring together a team whose purpose is to solve that problem, determine the right solution, and to report back to the Council. That team would then provide a representative to the Council on aspects of that problem and solution.

⁴ This also effectively constrains the number of Council representatives. That this constraint is independently important.

⁶ Being a Council representative is ultimately a position of responsibility for the Project as a whole. While we hope that the position is fulfilled

also hope that it is not viewed as a position of status to vi

⁸ The Council is not required to assign such roles exclusiv
may appoint any willing Project member. Such roles do n
for purposes such as decision-making.

⁷ In practice the infrastructure team as a whole does not
internally strives to meet the principle of least privilege.

Moderation, disagreement

This section describes the roles of the Leadership Council in helping resolve disagreements and conflicts, as well as the roles of the teams.

Disagreements and conflicts fall on a spectrum of intensity. Disagreements are more factual and/or technical matters, while conflicts are more social or relational roadblocks to collaboration. Many situations are a mix of both disagreement and conflict. The Council can help with disagreements, while aspects of conflict are the purview of the moderation team.

This document does not specify moderation policy in detail, but it is necessary to specify interactions with the Council and the moderation team. General moderation policy is in the Council and the moderation team. General moderation policy is in the document.

Much of the work of the Rust Project involves collaboration with people whom care deeply about their work. It's normal for people to be strongly about that disagreement. Disagreement can be resolved by discussing and addressing issues, and ideally, people who disagree can amicably explore those disagreements without escalation.

Situations where disagreements and conflicts arise can sometimes escalate into conflicts, and conflicts can de-escalate back into disagreements. The line between a disagreement and a conflict is not clear in many cases. If you disagree, assume the situation is a conflict.

In the event of a conflict, involved parties should reach out to the Council for help resolve the conflict as soon as possible. Time is of the essence to resolve a conflict before it gets worse or causes more damage.

Disagreements among teams

Where possible, teams should attempt to resolve disagreements on their own. If assistance from the Council is needed, the Council will provide it. The Council will help resolve disagreements, but teams need to maintain good working relationships to avoid persistent disagreements or escalations into conflicts.

Potential resolution paths for disagreements between previously discussed option, devising a new option, falls in, or deciding that the decision is outside the p to the Council to find a new home for that work.

Conflicts involving teams or Project

Conflicts involving teams or Project members should as soon as possible. The Council can help mitigate the pending/urgent decisions, but the moderation team conflicts and interpersonal issues, across teams or c

Individuals or teams may also voluntarily engage in or interpersonal issues, such as non-binding external should keep the moderation team in the loop when from the moderation team regarding appropriate re Individuals or teams must not use resources that w

Contingent moderators

The moderation team must at all times maintain a p moderators", who must be approved by both the m internal consent decision. The moderation team and both consist of at least three members each. The co

- Not part of the current moderation team *or* th
- Widely trusted by Rust Project members as joi moderation team; this will often mean they're capacity.
- Qualified to do moderation work and [audits](#) as and moderation team. More detailed criteria a moderation policy, which is out of scope for th
- Willing to serve as contingent moderators: will interim moderation work if the moderation tea until they can appoint new full moderators. (Th expected to be willing to do moderation work

- Willing to stay familiar with moderation policy expected of a moderation team member (including contingent moderators). Contingent moderators should receive the same moderation team where possible.

The need for contingent moderators arises in a high-pressure situation and Council must be prepared to trust them to step in. A moderation team known and trusted by the rest of the Project helps limit the damage.

Moderation is a high-burnout activity, and individuals may find themselves wishing to step away from that work. Moderators may always choose to step down, in which case the Council must identify and bring in new moderators to fill any gaps. If a moderator asks a contingent moderator to become a full moderator, the Council must appoint a new contingent moderator. An individual moderator may also become a contingent moderator. If the moderation team becomes unavailable (as determined jointly by the Council and the moderators in a consent decision), or chooses to step down simultaneously, the Council must become the interim moderation team and must provide support to contingent moderators and start seeking new full moderators.

As the contingent moderator role does not have any special powers, in exceptional situations, those appointed to that role must be able to act as a moderation team, to reconfirm that they're still willing to act as a moderation team in a circumstance in which the contingent moderators are unavailable.

Moderation team policies and procedures

The moderation team has a duty to have robust policies and procedures. The Council provides oversight and assistance to ensure that the moderation team has policies and procedures and that they are sufficient for the role.

The Council may provide feedback to the moderation team. The Council is required to consider all feedback received. If the Council determines that the moderation team has not followed moderation policies and procedures, the Council may remove the contingent moderators. However, the Council must not remove the moderation team on a decision or policy.

Audits

If any Council member believes a moderation decision did not follow the moderation team's policies and procedures, they should discuss this with the moderation team. The Council and moderation team should, where possible, discuss and understand these concerns, and

One of the mechanisms this document provides for addressing concerns about moderation actions in a privacy-preserving manner is an audit mechanism. If a Council member believes moderation team actions do not follow the moderation team's policies or procedures, the Council member may decide to request an audit. (For example, in a particular moderation situation, they might do this in response to a report of a moderation situation.) This happens *in addition* to the moderation team's ongoing conversation; it is not a replacement for direct communication with the moderation team.

In an audit, the contingent moderation team works with the Council to establish whether the moderation team followed documented policies and procedures. This mechanism necessarily involves the contingent moderation team's judgment to evaluate moderation policy, specific events, and the corresponding moderation actions or proposed actions. The contingent moderation team is intended to second-guess the actions themselves; the contingent moderation team is not establishing whether the moderation team is acting in accordance with its policies and procedures, as well as highlighting unintended consequences and procedures themselves.

The contingent moderators also reach out to the Council to understand the context they might need.

Moderation processes and audits both take time, and require coordination. However, the Council, contingent moderators, and moderation team should communicate their concerns and expectations to each other in a transparent fashion and maintain open lines of communication.

Contingent moderators must not take part in decisions where they have a conflict of interest. Contingent moderators must not be involved in moderation before the contingent moderation team is established; the contingent moderation team; this gives people the opportunity to evaluate potential concerns or conflicts of interest.

The discussions with the Council and the contingent

the moderation team had to make an exception in p was an unexpected condition in policies or that they couldn't be incorporated in policy. This is an expected scrutiny by the contingent moderation team on the and the process for deciding the necessity to make violation of moderation team responsibilities.

As the audit process and the Council/moderation di team may decide to alter moderation policies and/o moderation decisions or proposed decisions. This is team to make.

The contingent moderation team must report the re team and the Council for their review. This must not private information, either directly or indirectly. Tog moderation team, this should aim to address the co

Last-resort accountability

The Leadership Council and moderation team each Rust Project. This document provides many tools by This section outlines the last-resort mechanisms by other accountable. This section is written in the hope that teams will make every possible effort to resolve

If the Council believes there is a systemic problem v based on an audit report from the contingent mode Council and moderation team cannot voluntarily co the situation, then as a **last resort**, the Council (by u simultaneously dissolve itself and the moderation t appoint new representatives to the Council, and the becomes the new interim moderation team.

Conversely, if the moderation team believes the Co Council and moderation team cannot voluntarily co the situation, then as a **last resort**, the moderation simultaneously dissolve itself and the Council. This p are at least three moderation team members. The t new representatives to the Council, and the conting

new interim moderation team.

The moderation team's representative is recused from the Council and moderation team to avoid conflicts of interest. The representative must still step down as well.

The removed representatives and moderators may not be on the moderation team for at least one year.

By default, the new Council and interim moderation team must be clearly communicating the transition.

This mechanism is an absolute last resort. It will almost never be used, but outcomes, to say the least. If situations escalate to the point where it is *horribly* wrong, and those cleaning up the aftermath are not *ever* happening again. The indication (by either the Council or the moderation team) that the situation *might* escalate to this point should be on the table and find a way to do "Something Else which

Moderation actions involving Projects

The moderation team, in the course of doing moderation, has the ability to take action not just against members of the Rust Project, but also against members of the Rust Project. Those actions may span from a conversation to removal from the Project. This document seeks to clarify the position of power and trust. This document seeks to clarify the position of power and cross-checks for the moderation team, as well as

If the moderation team plans to enact externally visible actions against the Rust Project (anything that would create a conflict of interest from a role, or exclusion from participation in a Project), any party may request that an [audit](#) take place by requesting contingent moderators, and that audit will be automatically

Until June 2024, audits are automatically performed. After that time, the Council will review the process is functional. After that time, the Council will review and decide whether to renew this provision.

When the moderation team sends a warning to a Project member, or of moderation action regarding a Project member, the

of requesting an audit.

Conflicts regarding Project members should be brought as possible.

Conflicts involving Council representatives

Conflicts involving Council representatives, or alternates, are treated as conflicts involving Project members. The moderation team may moderate representatives or alternates as any other Project member. A required [audit](#) by the contingent moderators for any moderation remains subject to the same accountability mechanism as moderation by the moderation team.

In addition to the range of moderation actions already described, the moderation team may take the following additional actions for representatives in resort, as a lesser step on the ladder of escalation than removal from the Project entirely. These actions are not generally specific to Rust teams as well.

- The moderation team may decide to remove a representative from a top-level team represented by that representative, and appoint a new representative to serve the remainder of the term.
- The moderation team may decide to prevent a representative from becoming a Council representative.
- The moderation team and Council (excluding the representative) may decide (as a private operational consent decision) to allow or disallow a representative's involvement in the Council. (In the case of disallowance, not excluded if they have a conflict of interest, and they agree to cooperate to make the sanctions effective. If they do not agree to cooperate, applying these partial sanctions, the moderation team may apply sanctions such as removal.)

All of these also trigger a required audit. The Council may also take moderation actions involving representatives or alternates to prevent people from becoming representatives.

Conflicts involving moderation team

Conflicts involving a member of the moderation team or members of the moderation team (minus any with a conflict of interest) should be handled by the moderation team or contingent moderation team to provide additional context and moderation or contingent moderation team should be involved in more systemic issue within the moderation team. The team must make this decision and must provide an audit report to the community.

Policies

These chapters contain policies covering the Rust project

Rust crate ownership policy

Introduction

This document covers the policy for crates published by the Rust project, adopted via [RFC 3119](#).

Categories

Rust crates published by the Rust project fall into one of three categories:

- **Intentional artifacts:** These are crates which are maintained by a team (usually libs), are actively maintained, are used by many users, and intentionally have an air of officialness.
- **Internal use:** These are crates which are used internally by the Rust project, e.g. crates.io, docs.rs, etc. Their primary purpose is for internal use, though the teams that maintain them (typically the Rust Project) may wish for the crate to have wider adoption and “intentional artifacts” and ultimately depend on them. Example: [conduit](#), [measureme](#). There are two sub-categories:
 - **Transitively intentional:** These are dependencies of intentional artifacts, and will show up in users’ dependency trees. They are intended to be *directly* used. The Rust Project will address security issues in these crates *as if* they are “intentional artifacts”.
 - **Not transitively intentional:** These are dependencies of internal use crates, tooling, the stdlib, or are otherwise not expected to show up in users’ dependency trees. The Rust Project may use these crates *internally*, but does not necessarily need to be public about security issues in these crates. If a security issue in these crates affects a published binary (or crate), it will be handled as a bug in the binary or website.
- **Experiment:** This was an experiment by a team to better inform API design (or whatever), with no intention of long-term maintainership. Example: [failure](#).

- **Deprecated:** This used to be an “intentional artifact” but isn’t anymore. Example: [rustc-serialize](#)
- **Placeholder:** Not a functional crate, used for placeholder tool, etc. Example: [rustup](#)
- **Expatriated:** This may have been an “intentional artifact” used by external users, but is no longer intended. The crate is no longer owned/managed by the Rust

Policy

Every crate in the organization must be owned by a team. Teams should use `rust-lang/foo` teams for this. Non-expatriated crates should use GitHub accounts as owners; if a crate needs additional owners, the team should create a project group. Note that this does not prevent (project group) users from having maintainer access to the repository. *publishing*.

Currently it is not possible for a crate to be owned by a GitHub account (or a similar account to be decided by the organization) in such cases. We should try to phase this account out. We should make sure it is clear who is responsible for each crate. The `rust-lang/publish-bots` team (or individual bot accounts) should use these accounts to publish crates.

Each crate in the organization, and any future crates, should be categorized, which to which category they belong in according to their purpose. We are not sure what the category should be when registering a crate. If a decision just yet, pick “Experimental”.

Each published crate must contain a README. At a minimum, it should contain the primary owning team. Based on their categories, crates should contain the following information in their READMEs and documentation:

Intentional artifact

“Intentional artifact” crates can choose their commitment level and what they are in their messaging. If and when a team

be mentioned in the charter as an intentional artifact. Such messaging should not be taken lightly and will require an RFC.

An example of such messaging would be text like:

```
This crate is maintained by The Rust [team] Team
This crate is post-1.0 and follows semver compatibility
```

Security issues in these crates should be handled with the same messaging by the Security Response WG, and should follow the [project's security policy](#).

Internal use

“Internal use” crates should contain the following text in their README or readme/documentation:

```
This crate is maintained by [team], primarily for use internally. It is not
intended for external use (except as a transitive dependency). It will not
undergo major changes to its APIs or be deprecated without a RFC.
```

The “except as a transitive dependency” text should only be used for the transitive dependency of an intentional-artifact library (“transitively intentional”).

Security issues in transitively intentional libraries should be handled with the same messaging as intentional artifacts.

Experiment

“Experiment” crates should mention they are experimental and are not intended to be used in a scoped sort of way; so if they are used, they should be clear about what they are guaranteeing.

An example of such messaging would be text like:

This crate is maintained by [team] as a part of an encourage people to try to use this crate in their through [method], but do not guarantee long ter

or, for experiments that are not intended to be used

This crate is maintained by [team] and is an inter guarantee stability or long term maintenance, us

Ideally, experimental crates that are published for f document to link to that lists out the purpose, rough experiment.

Deprecated

“Deprecated” crates should contain the following te readme/documentation:

This crate is deprecated and not intended to be u

Placeholder

“Placeholder” crates should contain the following te readme/documentation:

This crate is a functionally empty crate that exists [tool]. It should not be used.

In general it is better to have an empty placeholder the crate via yanking, so that there is a readme that crate is unavailable.

Expatriated

It's unclear if any action should be taken on these based on their officialness (including rust-lang/foo team owners). We should discuss (`rand`).

These should by and large not be considered to be "Intentional Artifacts" is in this RFC for completeness to be able to talk about them.

Transitions and new crates

Teams should feel free to create new crates in any of the "Intentional Artifact" crates must be accompanied with a team charter. Having team charters, this can transition to being a "Intentional Artifact" RFC or use its own process). Teams should notify `council@rust-lang.org` if they created such crates so that the Leadership Council can ensure the policy is applied.

From time to time a team's plan for a crate may change. Crates may need to be deprecated, or the team may decide to move to wider usage.

In general, teams should notify `council@rust-lang.org` if a transition is made.

Any transition *away* from "Intentional Artifact" requires Leadership Council approval.

Any transition to "Intentional Artifact" should ideally be accompanied by an update to the team charter if there is one.

Expatriation should basically *never* occur anymore, but if it does, it requires Leadership Council approval in case it is really necessary. If a team is working on a crate, they should deprecate it and encourage users to build their own thing. The repository may be transferred to a new group, the name is kept by the Rust project and the new group chooses a new crate name.

If "transitively intentional" crates are being deprecated, the security issues will still be handled.

Transitions between the other types can be made and state their lack of a strong stability/maintenance guarantee.

Applying this to existing crates

An audit should be performed on all existing potential crates, creating a list and roughly determining what their team and goals are.

Once we have this list, we can approach teams with the categorization and verify that the categorization is accurate. In the case of a mismatch, it takes time as the team may need to work out what their intention is.

Then, working with the teams, we make these changes. We make sure all crates have the appropriate `rust-lang` badge and remove personal accounts from the owners list.

For crates that are in direct use by a lot of the wider community, we treat them as anything other than “intentional artifact”, they are official. We announce this “change” to the community. While this is a change, in case of these crates, the vague sense of officialness was there, and we should at least try to rectify this so it is not misleading. Whether or not this needs to be done, and how, is up to individual teams.

A large part of this work can be parallelized; and it can be done.

Infrastructure

This section documents Rust's infrastructure, and how it is managed.

External Links

- [rust-toolstate](#) records build and test status of the Rust repository.

Other Rust Installation

- [Which installer should you use?](#)
- [Other ways to install `rustup`](#)
- [Standalone installers](#)
- [Source code](#)

Which installer should you use?

Rust runs on many platforms, and there are many ways to install Rust in the most straightforward, recommended way on the main [installation page](#).

That page describes installation via `rustup`, a tool that installs Rust in a consistent way across all platforms Rust supports. But what if you want to use those instructions?

- Offline installation. `rustup` downloads components from the internet. If you need to install Rust without access to the internet, you can use the `rustup-init` script.
- Preference for the system package manager. On Linux, you can use `dnf`, `apt`, or `pacman`. On macOS with [Homebrew](#), [MacPorts](#) or [pkgsrc](#), and on Windows with [Scoop](#), developers sometimes prefer to install Rust through the system package manager.
- Preference against `curl | sh`. On Unix, we usually install Rust via a shell script via `curl`. Some have concerns about the security of this method and would prefer to download and run the installer directly.
- Validating signatures. Although `rustup` performs signature verification, the only way to verify the signatures of Rust installers is to use the standalone installers.
- GUI installation and integration with “Add/Remove Programs”. `rustup` runs in the console and does not register its installed programs. If you prefer a more typical GUI installer, you can use standalone `.msi` installers. In the future, `rustup` will support Windows.

Rust’s platform support is defined in [three tiers](#), which categorizes the installation methods available: in general, the Rust project

tier 1 and tier 2 platforms, and they are all installable, though have only the standard library available, not cross-compilation targets only; Rust code can run on the compiler itself. Such targets can be installed with

Other ways to install rustup

The way to install `rustup` differs by platform:

- On Unix, run `curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs` in your shell. This downloads and runs `rustup` and runs the correct version of the `rustup-init`
- On Windows, download and run `rustup-init`

`rustup-init` can be configured interactively, and also controlled by command-line arguments, which can be found in the `rustup-init` help. Pass `--help` to `rustup-init` as follows to display the

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs
```

If you prefer not to use the shell script, you may directly install the Rust toolchain for your platform of your choice:

- `aarch64-apple-darwin`
- `aarch64-linux-android`
- `aarch64-pc-windows-msvc`
- `aarch64-unknown-linux-gnu`
- `aarch64-unknown-linux-musl`
- `arm-linux-androideabi`
- `arm-unknown-linux-gnueabi`
- `arm-unknown-linux-gnueabihf`
- `armv7-linux-androideabi`
- `armv7-unknown-linux-gnueabi`
- `armv7-unknown-linux-gnueabi`
- `i686-apple-darwin`
- `i686-linux-android`
- `i686-pc-windows-gnu`
- `i686-pc-windows-msvc`
- `i686-unknown-linux-gnu`

- [loongarch64-unknown-linux-gnu](#)
- [loongarch64-unknown-linux-musl](#)
- [mips-unknown-linux-gnu](#)
- [mips64-unknown-linux-gnuabi64](#)
- [mips64el-unknown-linux-gnuabi64](#)
- [mipsel-unknown-linux-gnu](#)
- [powerpc-unknown-linux-gnu](#)
- [powerpc64-unknown-linux-gnu](#)
- [powerpc64le-unknown-linux-gnu](#)
- [s390x-unknown-linux-gnu](#)
- [x86_64-apple-darwin](#)
- [x86_64-linux-android](#)
- [x86_64-pc-windows-gnu](#)
- [x86_64-pc-windows-msvc](#)
- [x86_64-unknown-freebsd](#)
- [x86_64-unknown-illumos](#)
- [x86_64-unknown-linux-gnu](#)
- [x86_64-unknown-linux-musl](#)
- [x86_64-unknown-netbsd](#)

Standalone installers

The official Rust standalone installers contain a single binary for an [offline installation](#). They come in three forms: tarballs for any Unix-like environment, Windows installers (`.msi`), and macOS installers (`.dmg`). The standalone installers come with `rustc` , `cargo` , `rustdoc` , the standard library, and the Rust documentation, but do not provide access to additional tools or features.

The most common reasons to use these are:

- Offline installation
- Preferring a more platform-integrated, graphical installer

Each of these binaries is signed with the [Rust signing keybase.io](#), by the Rust build infrastructure, with GPG signatures. The signatures are available in the `rustc` , `cargo` , and `rustdoc` binaries.

Past releases can be found in [the archive](#).

	platform	stab (1.85
	aarch64-apple-darwin	pkg pkg.asc tar.xz tar.xz.as
	aarch64-pc-windows-msvc	msi msi.asc tar.xz tar.xz.as
	aarch64-unknown-linux-gnu	tar.xz tar.xz.as
	aarch64-unknown-linux-musl	tar.xz tar.xz.as
	arm-unknown-linux-gnueabi	tar.xz tar.xz.as
	arm-unknown-linux-gnueabihf	tar.xz tar.xz.as
	armv7-unknown-linux-gnueabihf	tar.xz tar.xz.as
	i686-pc-windows-gnu	msi msi.asc tar.xz tar.xz.as
	i686-pc-windows-msvc	msi msi.asc tar.xz tar.xz.as
	i686-unknown-linux-gnu	tar.xz tar.xz.as
	loongarch64-unknown-linux-gnu	tar.xz tar.xz.as
	loongarch64-unknown-linux-musl	tar.xz tar.xz.as
	powerpc-unknown-linux-gnu	tar.xz tar.xz.as
	powerpc64-unknown-linux-gnu	tar.xz

	tar.xz.asc
powerpc64le-unknown-linux-gnu	tar.xz tar.xz.asc
powerpc64le-unknown-linux-musl	tar.xz tar.xz.asc
riscv64gc-unknown-linux-gnu	tar.xz tar.xz.asc
s390x-unknown-linux-gnu	tar.xz tar.xz.asc
x86_64-apple-darwin	pkg pkg.asc tar.xz tar.xz.asc
x86_64-pc-windows-gnu	msi msi.asc tar.xz tar.xz.asc
x86_64-pc-windows-msvc	msi msi.asc tar.xz tar.xz.asc
x86_64-unknown-freebsd	tar.xz tar.xz.asc
x86_64-unknown-illumos	tar.xz tar.xz.asc
x86_64-unknown-linux-gnu	tar.xz tar.xz.asc
x86_64-unknown-linux-musl	tar.xz tar.xz.asc
x86_64-unknown-netbsd	tar.xz tar.xz.asc

Source code

If you want to build the Rust toolchain from source

to download source code tarballs.

Channel	Archives
stable (1.85.0)	tar.xz tar.xz.asc
beta	tar.xz tar.xz.asc
nightly	tar.xz tar.xz.asc

If you want to make sure that the published source code is available in a public git repository, you can use the following script as a template.

► [Script for reproducing source tarball contents](#)

Archive of Rust Stable St Installers

Note: The Rust project only supports the latest s
patches. Generally speaking these archives shou
mechanisms to provide for patching.

The official Rust standalone installers contain a sing
offline installation. They come in three forms: tarba
any Unix-like environment, Windows installers (`.ms`
installers come with `rustc` , `cargo` , `rustdoc` , the s
documentation, but do not provide access to additio

The most common reasons to use these are:

- Offline installation
- Preferring a more platform-integrated, graphic

Each of these binaries is signed with the [Rust signing](#)
[keybase.io](#), by the Rust build infrastructure, with [GP](#)
are the signatures.

Stable (1.84.1)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``loongarch64-unknown-linux-musl``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.84.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``loongarch64-unknown-linux-musl``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.83.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

loongarch64-unknown-linux-musl

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.82.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

loongarch64-unknown-linux-musl

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.81.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``loongarch64-unknown-linux-musl``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu`

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.80.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

`i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.80.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.79.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.78.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

`aarch64-unknown-linux-gnu``aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.77.2)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.77.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.77.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-pc-windows-gnu
i686-pc-windows-msvc

`i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl`

x86_64-unknown-netbsd

Stable (1.76.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.75.0)

	platform
	aarch64-apple-darwin
	aarch64-pc-windows-msvc
	aarch64-unknown-linux-gnu
	aarch64-unknown-linux-musl
	arm-unknown-linux-gnueabi
	arm-unknown-linux-gnueabihf
	armv7-unknown-linux-gnueabihf
	i686-pc-windows-gnu
	i686-pc-windows-msvc
	i686-unknown-linux-gnu
	loongarch64-unknown-linux-gnu
	powerpc-unknown-linux-gnu
	powerpc64-unknown-linux-gnu
	powerpc64le-unknown-linux-gnu

`riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.74.1)

platform
<code>aarch64-apple-darwin</code>

`aarch64-pc-windows-msvc``aarch64-unknown-linux-gnu``aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.74.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.73.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

`x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.72.1)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>
<code>aarch64-unknown-linux-musl</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos`

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.72.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-pc-windows-gnu
i686-pc-windows-msvc

`i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl`

x86_64-unknown-netbsd

Stable (1.71.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

loongarch64-unknown-linux-gnu

`mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu`

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.71.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

`i686-unknown-linux-gnu``loongarch64-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd`

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.70.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd`

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.69.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.68.2)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

`x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.68.1)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>
<code>aarch64-unknown-linux-musl</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-pc-windows-gnu</code>

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.68.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu
aarch64-unknown-linux-musl
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.67.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.67.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

`armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.66.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabi

armv7-unknown-linux-gnueabi

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.66.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.65.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

`arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.64.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

`arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.63.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.62.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.62.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.61.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.60.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.59.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.58.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.58.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.57.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.56.1)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.56.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.55.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.54.0)

platform

aarch64-apple-darwin

aarch64-pc-windows-msvc

aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.53.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

riscv64gc-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.52.1)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.52.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.51.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.50.0)

platform
aarch64-apple-darwin
aarch64-pc-windows-msvc
aarch64-unknown-linux-gnu

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

`x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.49.0)

platform
<code>aarch64-apple-darwin</code>
<code>aarch64-pc-windows-msvc</code>
<code>aarch64-unknown-linux-gnu</code>

`aarch64-unknown-linux-musl``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-illumos

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.48.0)

platform

aarch64-unknown-linux-gnu

aarch64-unknown-linux-musl

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

`x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.47.0)

platform
<code>aarch64-unknown-linux-gnu</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``riscv64gc-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-illumos`

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.46.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.45.2)

platform

aarch64-unknown-linux-gnu

`arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

`x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.45.1)

platform
<code>aarch64-unknown-linux-gnu</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl`

x86_64-unknown-netbsd

Stable (1.45.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.44.1)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.44.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.43.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
mips-unknown-linux-gnu
mips64-unknown-linux-gnuabi64
mips64el-unknown-linux-gnuabi64
mipsel-unknown-linux-gnu
powerpc-unknown-linux-gnu
powerpc64-unknown-linux-gnu
powerpc64le-unknown-linux-gnu
s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.43.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.42.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.41.1)

platform

`aarch64-unknown-linux-gnu``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu`

	s390x-unknown-linux-gnu
	x86_64-apple-darwin
	x86_64-pc-windows-gnu
	x86_64-pc-windows-msvc
	x86_64-unknown-freebsd
	x86_64-unknown-linux-gnu
	x86_64-unknown-linux-musl
	x86_64-unknown-netbsd

Stable (1.41.0)

	platform
	aarch64-unknown-linux-gnu
	arm-unknown-linux-gnueabi
	arm-unknown-linux-gnueabihf
	armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.40.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl`

x86_64-unknown-netbsd

Stable (1.39.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
mips-unknown-linux-gnu
mips64-unknown-linux-gnuabi64
mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.38.0)

platform

`aarch64-unknown-linux-gnu``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu`

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.37.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

`x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl``x86_64-unknown-netbsd`

Stable (1.36.0)

platform
<code>aarch64-unknown-linux-gnu</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-apple-darwin</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-linux-musl`

x86_64-unknown-netbsd

Stable (1.35.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-linux-musl

x86_64-unknown-netbsd

Stable (1.34.2)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.34.1)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.34.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
mips-unknown-linux-gnu

`mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.33.0)

platform	
	aarch64-unknown-linux-gnu
	arm-unknown-linux-gnueabi
	arm-unknown-linux-gnueabihf
	armv7-unknown-linux-gnueabihf
	i686-apple-darwin
	i686-pc-windows-gnu
	i686-pc-windows-msvc
	i686-unknown-linux-gnu
	mips-unknown-linux-gnu
	mips64-unknown-linux-gnuabi64
	mips64el-unknown-linux-gnuabi64
	mipsel-unknown-linux-gnu
	powerpc-unknown-linux-gnu
	powerpc64-unknown-linux-gnu
	powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.32.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.31.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu

`mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.31.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.30.1)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.30.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.29.2)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.29.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.29.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.28.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.27.2)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.27.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.27.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.26.2)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.26.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu`

x86_64-unknown-netbsd

Stable (1.26.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
mips-unknown-linux-gnu
mips64-unknown-linux-gnuabi64
mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.25.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.24.1)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc`

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.24.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.23.0)

platform

`aarch64-unknown-linux-gnu``arm-unknown-linux-gnueabi``arm-unknown-linux-gnueabihf``armv7-unknown-linux-gnueabihf``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu`

`s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.22.1)

platform
<code>aarch64-unknown-linux-gnu</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-apple-darwin</code>

`i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.22.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

`mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.21.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.20.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu`

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.19.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

`i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`

Stable (1.18.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.17.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf

`i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin`

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.16.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu

`i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu`

`x86_64-unknown-netbsd`

Stable (1.15.1)

platform
<code>aarch64-unknown-linux-gnu</code>
<code>arm-unknown-linux-gnueabi</code>
<code>arm-unknown-linux-gnueabihf</code>
<code>armv7-unknown-linux-gnueabihf</code>
<code>i686-apple-darwin</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>
<code>i686-unknown-linux-gnu</code>
<code>mips-unknown-linux-gnu</code>
<code>mips64-unknown-linux-gnuabi64</code>
<code>mips64el-unknown-linux-gnuabi64</code>

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.15.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi

`arm-unknown-linux-gnueabi``armv7-unknown-linux-gnueabi``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``mips-unknown-linux-gnu``mips64-unknown-linux-gnuabi64``mips64el-unknown-linux-gnuabi64``mipsel-unknown-linux-gnu``powerpc-unknown-linux-gnu``powerpc64-unknown-linux-gnu``powerpc64le-unknown-linux-gnu``s390x-unknown-linux-gnu`

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.14.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

mips-unknown-linux-gnu

mips64-unknown-linux-gnuabi64

mips64el-unknown-linux-gnuabi64

mipsel-unknown-linux-gnu

powerpc-unknown-linux-gnu

powerpc64-unknown-linux-gnu

powerpc64le-unknown-linux-gnu

s390x-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.13.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.12.1)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.12.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

`armv7-unknown-linux-gnueabi``i686-apple-darwin``i686-pc-windows-gnu``i686-pc-windows-msvc``i686-unknown-linux-gnu``x86_64-apple-darwin``x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-freebsd``x86_64-unknown-linux-gnu``x86_64-unknown-netbsd`**Stable (1.11.0)**

	platform
	aarch64-unknown-linux-gnu
	arm-unknown-linux-gnueabi
	arm-unknown-linux-gnueabihf
	armv7-unknown-linux-gnueabihf
	i686-apple-darwin
	i686-pc-windows-gnu
	i686-pc-windows-msvc
	i686-unknown-linux-gnu
	x86_64-apple-darwin
	x86_64-pc-windows-gnu
	x86_64-pc-windows-msvc
	x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.10.0)

platform
aarch64-unknown-linux-gnu
arm-unknown-linux-gnueabi
arm-unknown-linux-gnueabihf
armv7-unknown-linux-gnueabihf
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.9.0)

platform

aarch64-unknown-linux-gnu

arm-unknown-linux-gnueabi

arm-unknown-linux-gnueabihf

armv7-unknown-linux-gnueabihf

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-freebsd

x86_64-unknown-linux-gnu

x86_64-unknown-netbsd

Stable (1.8.0)

platform

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-linux-gnu

Stable (1.7.0)

platform

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-linux-gnu

Stable (1.6.0)

platform
i686-apple-darwin
i686-pc-windows-gnu
i686-pc-windows-msvc
i686-unknown-linux-gnu
x86_64-apple-darwin

`x86_64-pc-windows-gnu``x86_64-pc-windows-msvc``x86_64-unknown-linux-gnu`

Stable (1.5.0)

platform
<code>i686-apple-darwin</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>
<code>i686-unknown-linux-gnu</code>
<code>x86_64-apple-darwin</code>
<code>x86_64-pc-windows-gnu</code>

`x86_64-pc-windows-msvc``x86_64-unknown-linux-gnu`

Stable (1.4.0)

platform
<code>i686-apple-darwin</code>
<code>i686-pc-windows-gnu</code>
<code>i686-pc-windows-msvc</code>
<code>i686-unknown-linux-gnu</code>
<code>x86_64-apple-darwin</code>
<code>x86_64-pc-windows-gnu</code>
<code>x86_64-pc-windows-msvc</code>

x86_64-unknown-linux-gnu

Stable (1.3.0)

platform

i686-apple-darwin

i686-pc-windows-gnu

i686-pc-windows-msvc

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-pc-windows-msvc

x86_64-unknown-linux-gnu

Stable (1.2.0)

platform
i686-apple-darwin
i686-pc-windows-gnu
i686-unknown-linux-gnu
x86_64-apple-darwin
x86_64-pc-windows-gnu
x86_64-pc-windows-msvc
x86_64-unknown-linux-gnu

Stable (1.1.0)

platform
i686-apple-darwin

i686-pc-windows-gnu

i686-unknown-linux-gnu

x86_64-apple-darwin

x86_64-pc-windows-gnu

x86_64-unknown-linux-gnu

Stable (1.0.0)

platform
i686-apple-darwin
i686-pc-windows-gnu
i686-unknown-linux-gnu
x86_64-apple-darwin

`x86_64-pc-windows-gnu``x86_64-unknown-linux-gnu`

The Rust Release Channel

NOTE This document should be considered incomplete and non-normative. Do not rely on anything described herein as to how things *should* be done.

A lot of the content herein is derived from [a posting by Brian Anderson back in 2016](#).

Rust releases are deployed onto `static.rust-lang.org`. There are several parts to a release channel, all key off a manifest file and then go from there.

Channel manifests

There is a top level directory `/dist/` which contains manifests. Manifests are named `channel-rust-[channelname]` accompanied by a `.sha256` file which is a checksum to check integrity of the downloaded data. In addition, each manifest is accompanied by a `.asc` file which is a detached GPG signature. This check not only the integrity but also the authenticity of the release.

In addition to the `stable`, `beta`, and `nightly` channels, there is a release which will be called `channel-rust-x.yy.z.t` accompanied by `.asc` files.

To support date-based channels, there is an archive directory `MM-DD` which contains copies of the requisite channels. If you installed `nightly-2019-02-16` then the channel manifest is at <https://static.rust-lang.org/dist/2019-02-16/channel-rust-nightly.toml>.

Content of channel manifests

Channel manifests are `toml` files. These are known as `channel-rust-x.yy.z.t.toml`. They are simply lists of the files associated with a release and

all of the time. Currently it is recommended to work are the topic of this section.

The top level of the `.toml` file consists of two important fields: `manifest-version` which is, at this time, `"2"`, and `date` whose value is of the form `"YYYY-MM-DD"`.

There are then a number of top level sections (table

- `pkg` - This contains the bulk of the manifest and the release. Typically this will be things like `rustc` package is semi-special and currently is used to install which will be installed by default.

Within packages are `components` and `extensions` installed by default by `rustup`, `extensions` are available via `rustup component add` and friends.

- `renames` - This contains a set of package renames to the correct package to fetch when the user enters a name.

Typically renames are used when a package is not yet considered to be release quality. For example, a package called `rustfmt-preview` but since its release table whose `to` field is `rustfmt-preview`. When you run `add rustfmt` the name is automatically translated to the user runs `rustup component list` then `rustfmt` is renamed back to `rustfmt` for display to the user.

- `profiles` - This is part of the future setup for installing to install. Instead of choosing the `components` to honor one of the entries in the `profiles` table entry which *essentially* (though not exactly) boils down to `"rust-std", "rust-docs", "rustfmt", "clippy"`.

Other profiles include `minimal` (`["rustc", "cargo"]`) which adds in additional things such as a copy of `src`), `miri`, `lldb`, `llvm-tools`, and `rust-analyzer`.

Package entries in the channel manifest

For example, a package might be:

Here you can see that this is for the `cargo` package `linux-gnu` target. The `url / hash` combo is for a `.tar.gz` for the same tarball compressed with `xz`. Either package may be present, but it is not useful for neither. Setting `url` to `false` to indicate that that particular combination is not available in this channel at this time.

In addition, there will be a single entry providing the

Here `version` will be effectively the `$tool --version`

Targets

Targets are the same triples you might use when building. You can specify a target with `-target=$target` and you can add them to your installation with `rustup target add $target`. When you do that, what `rustup` actually does is look up the target in question and installs that. Essential components are added with `rustup component add rust-std.$target`.

If a `rust-std` package for a target is not available installed via `rustup`. This can happen for lower tier

Since components and extensions are target-specific

to see that `rust-std` for every target is specified in
allows for cross-compilation by installation of any `r`

Service Infrastructure

Most services in the Rust Infrastructure are deployed about infrastructure, including current status, should

Our stability guarantees: many of our services rely on APIs, but not all of these are intended for public consumption. **resources behind `static.rust-lang.org` are considered stable** and these resources will not change without (at least) prior notice from the Rust project infrastructure for your own work to know.

Rust Log Analyzer

The [Rust Log Analyzer](#) analyzes CI build logs to extract information from the pull request. It is run by [TimNN](#).

Homu / bors

[Homu](#) is a bot which manages pull requests. It is often the name of its [bot user account](#). Approved pull requests have their tests run.

Documentation on homu commands can be found [here](#).

Please contact [Alex Crichton](#) if something goes wrong.

rfcbot

[rfcbot](#) is a bot ([bot user account](#)) which helps manage pull requests and PRs (typically RFCs). Team members can view a [RFC dashboard](#).

Documentation on rfcbot commands can be found [here](#).

rustbot

[rustbot](#) is a bot ([bot user account](#)) to assist with managing labels and to label and assign without GitHub permissions. See [here](#) for more details.

DXR

[DXR](#) is a cross-referenced source index for Rust, allowing you to navigate and search with ease. It is generated by [rustdoc](#).

perf / rust-timer

[perf](#) offers information about the performance of Rust. It is used for demand benchmarking.

It is split into a data [collector](#) and a [web frontend](#) + [API](#) available [here](#) and can be browsed on the [perf web](#).

One-off performance runs can be done by addressing the [@rust-timer](#) bot. You can trigger the necessary try-build and queue a [benchmark](#).

```
@bors try @rust-timer queue
```

(Technically, the requirement is that the [queue](#) command is a try build completing successfully.)

See the [documentation](#) for further bot commands.

Rust Playground

[Rust Playground](#) allows you to experiment with Rust. It is useful in other cases where you might not have the compiler or access to a [compiler](#) accessed [here](#).

Crater

[Crater](#) is a tool to run experiments across the whole of Rust. Its primary purpose is to detect regressions in the Rust compiler, and it does this by building crates, running their test suites and comparing the results with the Rust compiler.

Crates comes with a [bot to trigger experiments](#).

docs.rs

[docs.rs](#) builds and serves the rustdoc documentation for crates. Any issues may be filed on the [docs.rs repository](#). See the [#docs.rs](#) discussion or urgent issues.

Toolstate

The state of tools included with Rust are tracked on [toolstate](#). When merged via CI, the status of each tool is recorded in [toolstate](#). For further information, see the [toolstate system](#).

Rustup components history

The [rustup components history](#) tracks the status of rustup components on each platform over time. See [the repository](#) for more information.

CI Timing Tracker

The [CI Timing Tracker](#) tracks and compares how long CI takes on different platforms. [Alex Crichton](#).

Highfive (retired)

Highfive is a bot ([bot user account](#)) which was previously used to assign reviewers to pull requests and assigned reviewers. This service has been replaced by the Rust Forge bot.

Team Maintenance

The roster of the Rust teams is always in flux. From but also people sometimes opt to into “alumni status” but currently an active part of the decision-making process. A person is added or someone goes into alumni status and places that need to be updated.

Team repo

Membership of teams is primarily driven by the con See the README of that respository for the systems

Rules for changes to tea

Pull requests to the repository are merged by the [team](#) to merge PRs:

people and teams directories

If a change is related to an individual and does not e individual’s approval is required. If the change has a team repo (e.g., GitHub username change) then it is non-exhaustively includes:

- Changing team membership to alumni or full m
- Changing email address
- Adding zulip-id

If a change will grant additional permissions, then a change. Any team lead in the “parent team” chain m

- Adding new subteams under an existing team
- Changing other metadata (website description

repo directory

The `repo` directory differs slightly from the other directories, as it manages access to repositories, but also configures the repository settings.

The following changes must be approved and merged by the `infra-admins` team:

- Changing access to repositories owned by the `team-repo-admins` team
 - For repositories where ownership is not currently set, the `team-repo-admins` are expected to set the ownership to the team which owns the repository, when in the process of making changes, codifying in a comment in the relevant repository.

On the other hand, changes to the configuration of the repository must be approved and merged by `infra-admins`:

- Changing the [general repository settings](#)
 - This includes granting bots access to the repository.
- Changing the [repository branch protections](#)

Source code changes

The `team-repo` repository additionally contains code to track changes to the repository's source code files. This is owned by the Infrastructure team and is used to track changes.

Who belongs to team-repo-admins?

This group of people is nominated & approved by the `infra-admins` team. They are selected through any formal criteria. Eventually, we expect this group to diminish as additional automation is added.

Note also that the `infra-admins` team maintains "root" access to the repository, including the `team-repo`, in order to make changes to the repository when it is not operational. Those rights should only be exercised when necessary, with the `team-repo-admins` being the first point of contact for changes to the repository (for both teams).

Extra steps for changes

Full team membership

To make a full team member, the following places need to be updated:

- the [team repo](#)
- if the member is going to join the review rotation:
 - `[assign.owners]` section of `triagebot.toml` in the repo they will be reviewing

Team member departure

Remove the team member from any and all places:

- [team repo](#)
- `triagebot.toml` files of all repos they were in
- 1password

Handling of tools embedded in the rust-lang/rust repo (“toolstate”)

The Rust repository contains several external git submodules (see [Submodule Reference](#)). The [toolstate system](#) is used to allow the Rust repository to be in a failing state, except for beta releases.

This is necessary because the documentation is tested on the CI of the documentation repo. If there is a failing state in the Rust repository, it would not be possible to update the documentation, as a merged version of rustc that breaks it doesn't exist, and the documentation is in a failing state in both repos.

The toolstate system solves this problem by temporarily allowing the Rust repository to be in a “failing” state on `rust-lang/rust`. When the Rust repository is in a failing state, the submodule will be notified. They will then be responsible for resolving the failing state.

The three possible states of a “tool” are: `test-pass`, `test-fail`, and `test-unknown`.

This page gives a rough overview how the toolstate system works and for when which tools are (not) allowed to break.

Note: Historically, the toolstate system was used for tools that were closely coupled with the compiler (like `rustfmt` or `clippy`). These tools have been transitioned to use git subtrees instead, so they are not part of the toolstate system. Their tests, and any failures must be resolved with the Rust repository.

This document uses the term “tool”, but as of this writing, it only applies to external documentation.

Toolstate Rules

- For all tools, if a PR changes that tool (if it changes the tool's source code or submodule), the tool has to be in `test-pass` state before the PR can be merged.
- For all tools except for “nightly only” tools, the tool has to be in `test-pass` state before a new beta release can be made.

- If a PR lands on the `beta` or `stable` branch
- If a PR lands on `master` in the week before a new beta cut-off, regresses the tool (if it makes the state “vulnerable”) and ensure all these tools become `test-pass` status in the Rust `Forge index` for when the next beta cut-off occurs.

At the time of writing, the following tools are “vulnerable”:

Updating the toolstate repository

Updating the `toolstate repository` happens in two steps. First, a new branch (where bors moves a PR to test if it is good for release) is created for the individual platforms (at the time of writing, Linux, macOS, and Windows). A file is added to the repository recording the state of each tool. Later, if that commit actually entirely passed CI and all the tests, the “current tool status” in the toolstate repository is updated.

These scripts also automatically ping some people about the update.

For further details, see the comments in the involved scripts, specifically `publish_toolstate.py` as well as the other files mentioned in the comments.

Updating tools

Tools can be updated by updating the submodule to the latest version.

Run `git submodule update --remote path/to/submodule` to update the submodule. If the tests pass, commit, and send a pull request. The submodule is in the `src/doc` repository, so for example, the reference is `src/doc`.

While not required, `subup` may assist you with this.

Adding a tool

NOTE: We are trying to switch away from submodule to a more modern approach.

adding a subtree instead of a submodule: [#70651](#)

To add a new tool to be tracked, the following steps

1. Create a PR to rust-lang/rust that adds the sub build system / bootstrap updates. Be careful to use `--no-fail-fast` to avoid [issues like this](#).
2. Include changes to `checktools.sh`:
 - Build the tool at the top. This is the step to get the status for the tool. When `save-toolstate` is run, the build system will write a JSON file with the tool's status.
 - Add the tool to `status_check` with what to check.
3. Update `publish_toolstate.py` to add the tool to the list if the tool is broken, and its source repo. (Note that only users must have permissions to be assignable).
4. Submit a PR to the [toolstate repository](#) to merge the `latest.json` file.

Policies of the infrastruc

This section documents the policies created by the i

Policy on broken nightly

Sometimes the nightlies released automatically by CI are broken for some people or even everyone. This policy defines what to do in those cases.

Which nightly will be rolled back

A nightly can only be rolled back in the following cases:

- If it contains destructive code, for example if it modifies users files.
- If an infra problem caused it to be broken for at least 1 platform. Issues affecting only lower tier platforms are not rolled back since we don't guarantee working builds for those.

A nightly will **not** be rolled back if it's broken by a compiler bug. This is supposed to be caught by CI, and nightly can have compiler bugs. There are no exceptions, even if big projects are broken by a compiler bug.

What are we going to fix

Once any member of the infra team decides to roll back, they will roll back to the most recent working nightly. The roll back is done with rustup:

```
$ rustup toolchain install nightly
```

It's not required to roll back other things like the documentation or downloadable artifacts. After the nightly is rolled back, announce it on the [@rustlang](#) twitter account and on the status page.

Infrastructure guideline

This section contains the guidelines written by the infrastructure team who want to use the project's infrastructure.

Rust Infrastructure host websites

The Rust Infrastructure team provides hosting for st teams. This document explains the requirements a setup one.

Requirements for hosting websi

- **The website must be managed by a Rust te the project.**
The infrastructure team has finite resources a community projects.
- **The website's content and build tooling mu in either the [rust-lang](#) or [rust-lang-nursery](#)**
The infrastructure team must be able to rebuild example if we need to switch hosting), and hav inside infra-managed organizations is the best though we'd prefer for all the repositories to b
- **The website must be built and deployed wi**
We have custom tooling built around hosting s moment they work with Travis CI and Azure Pi services ask us in advance and we'll adapt the
- **The website must reach an A+ grade on the**
Browsers have multiple security features togg headers, and those features enhance users' pr working. An A+ grade on the Observatory indic correctly set.
- **The website must be hosted on platforms v**
We recommend either GitHub Pages or Amazo as the hosting and CloudFront as the CDN, but good as long as we consider them secure and

Static websites configuration

To avoid limitations of some hosting providers we have added additional, custom behaviors. These behaviors are configured in `website_config.json` at the root of the generated

Adding custom headers

One of the requirements for having a static website to reach an A+ grade on the [Mozilla Observatory](#), are custom headers. To setup custom headers you need to add an `headers` section to `website_config.json`. This example content includes headers that get a grade B on the Observatory (to reach grade A+ a Content Security Policy is required).

```
{
  "headers": {
    "Strict-Transport-Security": "max-age=31536000; includeSubDomains",
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "X-XSS-Protection": "1; mode=block",
    "Referrer-Policy": "no-referrer, strict-origin-when-cross-origin"
  }
}
```

Fixing GitHub Pages redirects

GitHub Pages behaves weirdly when it sits behind Cloudflare redirects: since it doesn't know the real domain name, it uses `name.github.io/repo-name` as the base of the redirect. To prevent this behavior the `github_pages` section in `website_config.json` with the origin base url as the

```
{
  "github_pages_origin": "org-name.github.io"
}
```

Deployment guide

These deployment steps are meant to be executed on a Linux machine.

team since they require access to our AWS account.

Configuring AWS

Create a CloudFront web distribution and set the following:

- **Origin Domain Name:** rust-lang.github.io/repo
- **Origin Protocol Policy:** HTTPS Only
- **Viewer Protocol Policy:** Redirect HTTP to HTTPS
- **Lambda Function Association:**
 - **Viewer Response:** arn:aws:lambda:us-east-1:123456789012:function:static-websites:4
- **Alternate Domain Names:** your-subdomain-name.rust-lang.org
- **SSL Certificate:** Custom SSL Certificate
 - You will need to request the certificate for your-subdomain-name.rust-lang.org (please use the DNS challenge to validate)
- **Comment:** your-subdomain-name.rust-lang.org

Wait until the distribution is propagated and take note of the distribution ID.

Head over to the domain's Route 53 hosted zone and create a new record:

- **Name:** your-subdomain-name
- **Type:** CNAME
- **Value:** the `.cloudfront.net` domain name you created

Create an AWS IAM user to allow the CI provider user to perform whitelisted automatic actions. Use `ci--ORC-rust-lang--rust` as the user name, allow programmatic access to the `ci-static-websites` IAM group. Then take note of the access key since you'll need those later.

Adding deploy keys

To deploy websites we don't use GitHub tokens (since they require fine-grained scoping) but a deploy key with write access unique for each repository. To add a deploy key you need to be an administrator on the repository and run this command:

```
$ cargo run --bin setup-deploy-keys rust-lang
```

The command requires the `GITHUB_TOKEN` (you can see yours [here](#)) to be present. You can also set the `TRAVIS_TOKEN` (you can see yours [here](#)) to be present. You can upload it to GitHub and configure Travis CI to use it.

Configuring Travis CI

To actually deploy the website, this snippet needs to be updated (please replace the contents of `RUSTINFRA_DEPLOY_DIR` and `RUSTINFRA_CLOUDFRONT_DISTRIBUTION`):

```
env:
  RUSTINFRA_DEPLOY_DIR: path/to/be/deployed
  RUSTINFRA_CLOUDFRONT_DISTRIBUTION: ABCDEFGH
import:
  - rust-lang/simpleinfra:travis-configs/status
```

You will also need to set the contents of the `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` environment variables on Travis CI to the credentials of the IAM user you created earlier. The `AWS_ACCESS_KEY_ID` should be taken from the build log, while the access key id should be taken from the IAM user's console page.

Configuring Azure Pipelines

To actually deploy the website, this snippet needs to be updated in the pipeline's YAML file:

```
resources:
  repositories:
    - repository: rustinfra
      type: github
      name: rust-lang/simpleinfra
      endpoint: rust-lang
```

Then you can add this steps when you want to execute the contents of `deploy_dir` and `cloudfront_distribution`:

```
- template: azure-configs/static-websites.yml
parameters:
  deploy_dir: path/to/output
  # Optional, only needed if GitHub pages
  cloudfront_distribution: AAAAAAAAAAAAAA
```

You will also need to set the following environment

- `GITHUB_DEPLOY_KEY` : value outputted when ac
- `AWS_ACCESS_KEY_ID` : access key ID of the IAM (public)
- `AWS_SECRET_ACCESS_KEY` : access key of the IAM CloudFront (**secret**)

Infrastructure team doc

This section contains the documentation about the Rust Infrastructure Team. Most of the linked resources are to infra team members though.

AWS access for team members

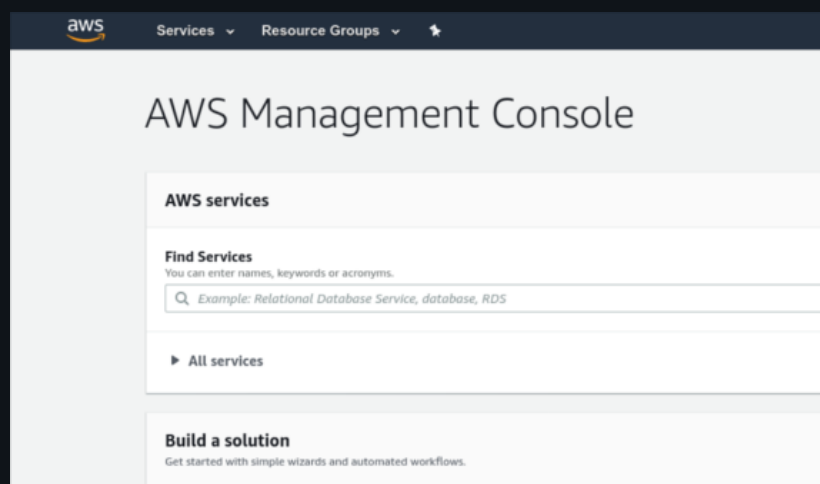
Selected members of the Rust Team have access to AWS. This page includes both members of the Infrastructure Team and members who are hosted on AWS.

This document explains how to access our AWS account if you're an infrastructure team member and you need access. If you're not, read the ["AWS access management"](#) page.

Setting up your user after receiving credentials

The first thing you need to do after receiving your credentials is to enable 2-factor authentication: until you do this, you'll be automatically granted just the permissions needed to complete the setup.

[Sign into the console](#) with the temporary credentials of the team member who created the user. You'll be prompted to change your password: change it and log in again. Then, go to the [MFA settings](#) located in the dropdown at the top:



Scroll down and click the "Assign MFA device" button (this is classic TOTP) and configure it with your authenticator app. Then, log out of the console and log in again to gain access to the rest of the console.

Do not choose "U2F security key", even if you own one, as this would prevent you from using the CLI, restricting access to the console only.

Using the AWS console

The AWS console provides a visual interface to most

[Sign into the console](#)

Using the AWS CLI

The [AWS CLI](#) allows you to interact with our AWS account. To set it up the first time, follow Amazon's documentation on [credentials](#). The CLI doesn't use your console password, but you'll need to create an access key from the "My Security Credentials" page.

2-factor authentication

To ensure the security of our AWS account, 2-factor authentication is required when using the CLI. The Infrastructure Team developed a script to automate this process by creating a temporary session validated with a 2FA code. The session expires in 12 hours, and it's valid for an unlimited number of CLI commands.

To use the script, clone the [rust-lang/simpleinfra](#) repository. At any time you need to use the AWS CLI run this command:

```
eval $(~/PATH/TO/SIMPLEINFRA/aws-creds.py)
```

That command will prompt you for your 2FA code, and set environment variables in the current shell with the temporary credentials. You'll need to run the command again after 12 hours, or if you want the credentials to persist.

Plaintext credentials

By default, AWS CLI stores your credentials (including your access key ID and secret access key) in the `~/.aws/credentials` file, without any kind of encryption. While it's possible to store plaintext credentials in your home directory, for security requirements, it'd be best not to store them anyway.

If you use a password manager with a CLI interface, the problem is to store your credentials in the password manager to [call your password manager](#) to fetch the credentials.

AWS access management

This document explains how to setup and manage AWS access. If you're a team member and you need to access AWS, you will have received your credentials for the first time, check the ["AWS access for team members"](#) page.

Granting access

To grant access to a person, go to `team-members-access` configuration and add the new user to it, specifying the group they are a member of. The user will be created as soon as you save the configuration.

By default, there will be no credentials attached to the user. To enable access to the [IAM console](#), open the security credentials page for the user and enable a console password. Let AWS generate a random password that will be changed on first login.

Finally communicate to the user that they can join the team. They should follow the ["AWS access for team members"](#) page to get their access to their account.

Revoking access

To revoke access from a person, log into the [IAM console](#) and go to the page of the user you want to delete, and:

- Disable console access by clicking "Manage" on the console access row
- Disable 2-factor authentication by clicking "Manage" on the 2FA row
- Remove all the access keys, including inactive ones

Once all the access was removed from the console, remove the `access/_users.tf` in the Terraform configuration, and run `terraform destroy` to remove the configuration.

Selection of AWS Region

The Rust project has deployed a lot of resources on `west-1`. As we are growing our footprint and expanding, we are reconsidering which regions we want to use.

Please note that this is mainly for new resources that we create on new accounts. Existing resources might get migrated, but it might not be worth it given our limited time.

Selection Criteria

We have two criteria that we use to make this decision:

- **Price** - Pricing differs between regions, and we want to use cheaper regions.
- **Location** - We want to host our services close to our users. Rust is used globally, we won't be able to satisfy everyone.

Price

Looking at the current distribution of our bill, outbound traffic is the most expensive item. This severely limits the price saving we can get from a cheaper region.

Even if we assume that we will be able to significantly reduce our AWS costs (e.g. by moving to Fastly), the difference between

Locations

Because most of our traffic comes from the US, we want to host our services here. The following regions are interesting to us:

- `us-east-1` or `us-east-2` (cheaper)

- `us-west-1` (already in use)

Services we want to distribute more globally, e.g. the ones we want to deploy to Europe. Here, the following regions seem

- `eu-west-1` (cheaper)
- `eu-central-1` (more central location)

Decision

We decided to use the following regions for new resources:

- `us-east-2` - Given that most of our infrastructure is in the US, we will use a cheaper region here to benefit at least a little.
- `eu-central-1` - Since we're not deploying that much to Europe, we will optimize for location here.

When deploying new resources, they should be deployed to the following regions: `us-east-2` for resources that need to be geographically distributed in the US, and `eu-central-1` for resources that need to be geographically distributed in Europe.

Bastion server

- FQDN: `bastion.infra.rust-lang.org`
- [Ansible playbook](#) to deploy this server.
- [Terraform configuration](#) to create AWS resources.
- [Instance metrics](#) (only available to infra team members).

Logging into servers through the bastion

To improve the security of our infrastructure it's not recommended to connect directly to a production server with SSH. Instead, all connections should go through a bastion server, called the "bastion", which only allows connections from specific IP addresses and logs any connection attempt.

To log into a server through the bastion, use one of the following methods:

- Use SSH's `-J` flag:

```
ssh -J <username>@bastion.infra.rust-lang.org  
-<username>@servername.infra.rust-lang.org
```

- Configure your SSH client to always jump through the bastion host:
 - Add this snippet to your SSH configuration file (`~/.ssh/config`):

```
Host servername.infra.rust-lang.org  
ProxyJump <username>@bastion.infra.rust-lang.org
```

- Use SSH:

```
ssh <username>@servername.infra.rust-lang.org
```

Please remember the bastion server only allows connections from specific IP addresses. Infra team members with AWS access can use the bastion server in practice to either have your own bastion server or a shared one.

The SSH keys authorized to log into each account are stored in the bastion's vault. Additionally, people with sensitive 1password access can access the vault to log into every account, provided their connection is authorized.

Common maintenance procedures

Adding a new user to the bastion server

To add a new user to the bastion you need to add it to the `unprivileged_users` list in `ansible/roles/common/files/ssh-keys`, and change the `allowed_users` list to include the new user. Please leave a comment in the code if a user will have access to.

Once that's done [apply the playbook](#) and [add a new user](#).

Editing a whitelisted IP

Due to privacy reasons, all the static IP addresses of the bastion are stored on [AWS SSM Parameter Store](#) instead of in the code. When running the following commands, replace `USERNAME` with the username you want to edit.

Adding a whitelisted IP

1. Run:

```
aws ssm put-parameter --type String --name /bastion/ips/USERNAME --value "IP_ADDRESS/32" --overwrite
```

2. Add the username to the list in `terraform/bastion/main.tf` (under `allowed_users`).
3. [apply the Terraform configuration](#).

Updating a whitelisted IP

1. Run:

```
aws ssm put-parameter --overwrite --type  
"/prod/bastion/allowed-ips/USERNAME" --v  
us-west-1
```

2. [apply the Terraform configuration.](#)

Removing a whitelisted IP

1. Run:

```
aws ssm delete-parameter --name "/prod/b  
region us-west-1
```

2. Remove the username from the list in [terraform](#) variable `allowed_users`).
3. [apply the Terraform configuration.](#)

Bors

The infrastructure team manages an instance of [Homu](#) repositories inside the `rust-lang` organization. The [lang.org](#), and is backed by the [@bors](#) GitHub account.

The service is configured [with Terraform](#), and it's auto-deployed to the `lang/homu` repository onto our [ECS cluster](#).

Maintenance procedures

Fixing inconsistencies in the queue

Homu is quite buggy, and it might happen that the queue gets stuck in the repositories.

See [Fixing inconsistencies in the bors queue](#) for instructions.

Adding a new repository to bors

There are multiple steps needed to add a repository to bors:

1. The [@bors](#) GitHub account needs to be granted write access to the repository.
2. Each CI provider needs to have a single GitHub Actions workflow provided by default on GitHub Actions, but it can be customized which will generate a `bors build finished` commit.

```

end-success:
  name: bors build finished
  if: success()
  runs-on: ubuntu-latest
  needs: [ALL, OTHER, JOBS]
  steps:
    - name: Mark the job as successful
      run: exit 0

end-failure:
  name: bors build finished
  if: "!success()"
  runs-on: ubuntu-latest
  needs: [ALL, OTHER, JOBS]
  steps:
    - name: Mark the job as a failure
      run: exit 1

```

Make sure to replace `[ALL, OTHER, JOBS]` with the jobs that you want to run on.

These jobs need to run on specific branches (e.g. `main`), so you need to add those branches to the list of branches tested by the workflow. This is done by adding a `branches` field to the `on` field of the workflow. This looks like this:

```

on:
  push:
    branches: [
      auto,    # Added for bors
      try      # Added for bors
    ]

```

3. Add the repository name to the bors permissions. You need to grant the `bors.REPOSITORY.review` permission to the workflow. You can see an example of adding bors permissions in the [bors workflow](#).
4. Add the repository to the `repositories` map in the `workflow`. This will create a webhook and inject its secret into the workflow environment.

5. Add the repository to the [Bors configuration](#), to the repositories. Note that the environment variable `BORS_CONFIG` is automatically set as long as you completed step 3.
6. Give it a test by commenting `@bors ping` in a PR. You can then try to approve the PR with `@bors r+`.

Fixing inconsistencies in

bors queue page: <https://bors.rust-lang.org/queue/>

i ****WARNING****: You should only do this if you are working on the rust-lang/rust repo. Please do not synchronize the bors queue permissions even if you have write access to the repository. You must perform the required cleanup steps.

This is a **destructive** operation. If someone syncs the bors queue for around 45 minutes: around 30 minutes before the queue is recollected, and 15 minutes after that to kick off the tree.

DO NOT CLICK THIS BUTTON IF YOU ARE NOT READY TO DO A FULL CLEANUP.

Sometimes you have to do a bors queue sync for various reasons. This operation requires you to be very careful, as otherwise we may end up with (or even `beta`) that should not have been merged or released.

Steps

Step 0: Announce your intention

Let T-infra (and other reviewers) know that you plan to sync the bors queue via [zulip thread](#) to let other contributors know about the plan.

Step 1: Close the tree

Find a PR that's currently being tested (or any open PR that is *really* messed up).

Issue `@bors treeclosed=1000` along with some brief explanation of the tree so other reviewers (especially people doing

Example:

```
Closing the tree due to a resync.  
cc <https://rust-lang.zulipchat.com/#narrow/c  
infra/topic/try.20jobs.20not.20kicking.20off?  
  
@bors treeclosed=1000
```

Step 2: Click “synchronize” button

As a courtesy, you can record which PRs had try jobs. The distinction between regular jobs and try jobs will be “pending” PRs.

Click the “synchronize” button in the [bors queue page](#) performing the next step.

Step 3: Kick out all actively tested PRs

Find *all* actively tested PRs. This includes both “auto”, “pending”, or “try” builds which show up as “pending” to the distinction and try jobs can show up as “pending” to

On each “pending” or “pending (try)” PR, write:

```
@bors retry r- (sync)
```

to suspend the current job and take it out of the queue (full CI jobs).

Step 4: Wait

Wait for around **30-45 minutes** to allow bors to recompute the tree beforehand, as it will cause bors to have an increased lead to unspecified behavior.

Step 5: Kick out ineligible PRs

Check “approved” PRs in the queue. Some of them will be rejected due to reasons such as:

- Merge conflicts
- Significant changes since last review
- Never has been approved

But sometimes bors forget these distinction. You’ll need to re-review “approved” PRs and check their eligibility.

For “approved” PRs that are not actually eligible, you can re-review via `@bors r-`. Prefer to be cautious if you are not sure to avoid ambiguity.

```
@bors r- (sync)
```

Step 6: Double-check approved PRs

Do another review pass of “approved” PRs in the queue to ensure they are actually eligible for merge.

Step 7: Re-open the tree on the same PR

Reopen the tree on the same PR that you issued the `@bors r-` command.

```
@bors treeclosed-
```

Closely monitor bors’ behavior for around 5 minutes after re-opening the tree. If testing a PR that’s eligible for merge. Update the relevant PRs.

Step 8: Re-queue try jobs

In Step 2, if you had to kick out try jobs, you can re-queue them. If you previously had try jobs started on them.

Use the normal try-job command:

```
@bors try
```

and not `@bors retry`.

Step 9: Edit your `treeclose` commands to prevent bors from reopening them up

Edit the `@bors treeclosed=xxxx` command and `@bors`

```
~~@/bors treeclosed=xxxx~~
```

```
EDIT(ferris): edited to prevent bors from reopening
```

AFTER the tree has been reopened to prevent bors from reopening

Content Delivery Network

Users of the Rust programming language interact with the project in various different ways. They access the project's website, crates index, and download Rust releases and crates. The Rust project and served through a [Content Delivery Network](#).

This document outlines why we use CDNs, for what purposes, and our objectives.

Objectives

We have three goals for our use of CDNs in our infrastructure:

1. Reduce costs of outbound traffic through cheaper providers.
2. Reduce load on origin servers to save compute resources.
3. Provide a way to rewrite legacy URLs for some resources.

Reducing Costs

As an open source project, we have to be very mindful of our costs. Outbound traffic is by far one of the most expensive parts of our infrastructure that will continue to increase as Rust gets more popular.

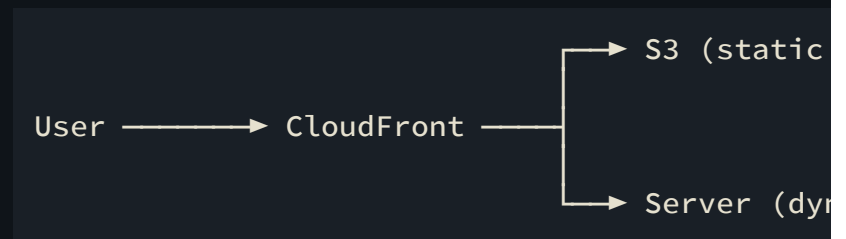
Cloud providers typically charge different rates for different services. For example, serving data straight from [Amazon S3](#) is cheaper than serving the same data through an [Amazon CloudFront](#) distribution. By default, even for services that can't make use of other services.

Infrastructure

Most of the project's resources are hosted on [AWS](#). Static content is served from [Amazon CloudFront](#), while dynamic content is loaded from a server. Both static and dynamic content is served through [Amazon CloudFront](#), the [Content Delivery Network](#).

When a user access a resource, e.g. they are trying to access a crate page, they are trying to access a resource that is served through a CDN.

the resource through the CDN. Different *distribution* configuration and a backend (called the *origin*). For `static.crates.io` goes through a *distribution* that and then caches it for future requests.



Distributions

There are many distributions, all of which are configured in the repository. However, their usage is very unevenly distributed. The following are the most important ones for the project, both in terms of traffic and ecosystem.

Rust Releases

Whenever a user installs or updates Rust, pre-compiled binaries are served from `static.rust-lang.org`. The same is true when Rust is updated. This is why this distribution has by far the highest traffic.

Rust binaries are static and are stored in [Amazon S3](#) and served by a [CloudFront](#) distribution.

The distribution for `static.rust-lang.org` has a [CloudFront](#) [Lambda](#) function. The router provides a way to list files and a legacy URL for `rustup.sh`.

The cache for Rust releases is invalidated nightly.

Crates

Similar to Rust releases, crates are served from as static content. While still being the second-largest distribution in our ecosystem, it is significantly smaller than Rust releases.

than the releases.

Crates are static and stored in [Amazon S3](#), and serv

Crater agents

- Source code: [rust-lang/crater](#)
- Hosted on:
 - [crater-aws-1.infra.rust-lang.org](#) (be
 - [crater-azure-1.infra.rust-lang.org](#) (
- Maintainers: [pietroalbini](#)
- [Application metrics](#) (only available to infra team)
- Instance metrics (only available to infra team)
 - [crater-aws-1.infra.rust-lang.org](#)
 - [crater-azure-1.infra.rust-lang.org](#)

Service configuration

Crater agents are servers with our standard configuration hosting the agent. A timer checks for updates every image is present the container will automatically be [managed with Ansible](#).

Common maintenance procedures

Starting and stopping the agent

The agent is managed by the `container-crater-agent` means it's possible to start, stop and restart it with the

```
systemctl stop container-crater-agent.service
systemctl start container-crater-agent.service
systemctl restart container-crater-agent.service
```

Inspecting the logs of the agent

Logs of the agents are forwarded and collected by j

journalctl:

```
journalctl -u container-crater-agent.service
```

Manually updating the container image

The container is updated automatically every 5 minutes (at the moment of writing this document, it is not present). If you need to update them sooner you can do so by running this command:

```
systemctl start docker-images-update.service
```

Dev Desktops

The dev desktops provide maintainers and contributors access to high-powered cloud compute. They are part of the [Rust Foundation](#).

Machine	Architecture	Permissions
<code>dev-desktop-eu-1</code>	<code>aarch64</code>	Yes
<code>dev-desktop-eu-2</code>	<code>amd64</code>	No
<code>dev-desktop-us-1</code>	<code>aarch64</code>	Yes
<code>dev-desktop-us-2</code>	<code>amd64</code>	No

How to apply to the program

At this time, access to the program and the compute is limited to [certain teams](#) and core contributors of the Rust Project. While the program is limited to [certain teams](#). If you are in one of these teams, you can request access.

If you feel like your work on the Rust project would benefit from a powerful build machine, reach out to infra@rust-lang.org for more information:

- Your GitHub handle
- A short description of how you would use and benefit from the dev desktops

How to connect to a dev desktop

Each user has their own account on the dev desktops. The account is named after the user's GitHub handle, with `gh-` as a prefix. For example, if your GitHub handle is `user`, you will have a user account with the name `gh-user`.

Users can connect to the dev desktop with SSH. The dev desktops will handle the authentication, and automatically fetch the user's public key from GitHub.

You can connect to the instance with the following command:

```
ssh <your-username>@<name>.infra.rust-lang.org
```

Replace `<name>` with the machine name from the table. For example, connect to `dev-desktop-eu-1` using the host `1.infra.rust-lang.org`.

If you don't have a public key on GitHub, read the following guide to create an SSH key and add it to your GitHub account. The key has been added before the dev desktops get provisioned.

- [Generating a new SSH key and adding it to the SSH agent](#)
- [Adding a new SSH key to your GitHub account](#)

To make the command easier, you can configure an alias:

```
Host rustvm
  User <your-username>
  HostName <name>.infra.rust-lang.org
```

Then you can connect with `ssh rustvm`.

How to set up your account

When connecting to the machine for the first time, there are a few things you need to do.

First, check that your Git username and email are correct:

```
git config -l --global
```

You can configure your username and email address like this:

```
git config --global user.name "Your name"
git config --global user.email "your-email"
```

How to customize your shell

You can set your default shell on the dev desktops by editing the `vars_user_config` file in the [rust-lang/simpleinfra](#) repository. Open `desktop/defaults/main.yml`, look for the variable `vars_user_config` to the list.

```
vars_user_config:
  - username: gh-jdno
    shell: /usr/bin/zsh
  - username: gh-WaffleLapkin
    shell: /usr/bin/fish
```

Open a pull request and request a review from [@rust-lang/simpleinfra](#) on Zulip).

After the pull request is merged, an infrastructure automation script will push the configuration to the dev desktops. Only after that will the new shell be available.

How to install a Rust toolchain

The dev desktops don't have Rust pre-installed, but you can install a specific toolchain from a local repository or worktree.

First, you want to run the following command to install the Rust toolchain.

```
/usr/local/bin/init.sh
```

If you don't want or need to work with your own version of Rust, you can skip the previous section and start working.

If you haven't done so yet, open the [rust-lang/rust](#) repository in your personal account. Then connect to the dev desktops.

```
/usr/local/bin/setup_rust.sh
```

The script will clone your personal fork to the dev desktops, build it from [rust-lang/rust](#), and compile it. Once that's done, you can work with them locally.

The directory contains more scripts to manage workspaces. Run `help.sh` to get a list and a short description of them.

How to interact with GitHub

The dev desktops are designed to work with repositories on GitHub. You need a GitHub user account. A GitHub App is used to protect your account and give you control over the repositories that the dev desktops can access.

First, go to <https://github.com/apps/rust-cloud-vm> and install the app on your repositories. It's recommended to only grant access to the repositories you use on the dev desktop, e.g. your fork of `rust-lang/rust`.

Then connect to the dev desktop and clone the repository using the HTTPS URL. From there, you can work with the repository as usual.

Under the hood, the GitHub App acts as a credential provider that generates temporary access tokens that are scoped to the permissions you granted to the application. If you get an error, review the permissions you granted to access your repository.

How to set up remote development with VS Code

Most modern code editors provide support for remote development. You can use them to write code locally, but execute it inside the dev desktop. The workflow will differ slightly, the following example for Visual Studio Code applies to other editors as well.

Setting up remote development with VS Code is pretty straightforward. See in detail in VS Code's documentation: [Remote Explorer](#).

1. SSH into the dev desktop and clone the repository to a local folder
2. Then open VS Code on your machine and install the [Remote Explorer Extension Pack](#)
3. Open the command palette and search for "Remote Explorer: Add New Remote"

4. Enter your username and the instance name (
5. Select the path for the cloned repository from
6. Install any extensions that you want to run on
7. Use VS Code to run or debug the code remote

How to request new packages and

If you need a specific tool or package that is not installed, you can request its installation by opening a pull request against the `dev-desktop` repository. The Ansible role `dev-desktop` contains a list of the system packages that are installed on the machine. To request a new package, open a pull request, and ping `@rust-lang/`

It helps to check <https://packages.ubuntu.com/> first to see if a package is available for both `arm64` and `amd64` architectures. If you are running Ubuntu 22.04 LTS.

Available disk space

Every user has a quota of how much disk space they can use, up to `150GB`.

Exceeding the quota will result in the error `Disk quota exceeded`.

The [remarks on disk space](#) section of the `rustc dev` guide explains how to clean your disk space.

How to give feedback and report

If you experience any problems with the dev desktop, please get in touch with the [infrastructure team](#):

[#t-infra](#) on Zulip

We might ask you to create an issue in the [rust-lang](#)

Github App for pushing to the dev-desktops

These instructions are for server-side setup and deployment of the app. The user only needs to be directed to the app and it will just work for them.

We're using the python github library for all github operations. <https://pygithub.readthedocs.io/en/latest/introduction.html>

How to setup an App

1. Go to <https://github.com/settings/apps>
2. New Github App
3. Fill out metadata (name and url)
4. disable WebHook checkbox
5. Set **Contents - Repository contents, commits, releases, and merges.** to read/write
6. Set **Workflows - Update GitHub Action workflows**
7. Set to "enable on any account"
8. Create App
9. Go to https://github.com/settings/apps/{your_app_id} and copy the **ID** into **app_id.txt** (same folder as **gen_temp**)

How to generate a .pem file for your App

1. Go to https://github.com/settings/apps/{your_app_id} and generate a private key
2. Download starts, save it to somewhere private
3. copy the .pem file into the same folder as the **app_id.txt** name it **dev-desktop.private-key.pem**

How to install the app for a user

1. direct the user to
`https://github.com/settings/apps/{your_app_name}`
2. let them install it on the org/user they want to use
want to use

How to generate a temporary access token

1. invoke `gen_temp_access_token.py <github_username>`

Integration into git command line

We're using `credential-helpers`. For debugging a credential helper and invoke it with

```
git -c credential.helper -c credential.UseHttpCredentials  
origin branch
```

Note that this does not work for remotes that are reachable via `https`!

The first command line argument is `get`, `store` or `erase` (`exit(0)`) for everything but `get`, as we regenerate the token anyway.

The actual arguments are passed via stdin and usually look like

```
protocol=https  
host=github.com  
path=your_repo.git
```

Discord moderation bot

- Source code: [rust-lang/discord-mods-bot](#)
- Hosted on: [rust-ecs-prod](#) ECS cluster
- Maintainers: [technetos](#)

The bot is hosted on the [rust-ecs-prod](#) ECS cluster with the [discord-mods-bot](#) service name. Its container is named [discord-mods-bot](#) with the same name, and its data is stored in the [sh](#)

Automatic deploys are setup from the [rust-lang/discord-mods-bot](#)

The Discord bot account is [rustbot#4299](#). [pietroalbini](#) and [aidanhs](#) have access to the developer portal.

Common maintenance procedures

[Instructions on how to manage ECS services are available](#)

Domain names and DNS

All the DNS records of the domains owned by the Rust Forge are managed by [AWS Route 53](#), and can be tweaked by members of the community. There are instructions for them on how to make changes.

- [Changing DNS records of a domain managed by AWS Route 53](#)
- [Managing DNS for a new domain with Terraform](#)
- [Adding subdomain redirects](#)
- [Transferring domain names to Rust Forge](#)

Changing DNS records of a domain with Terraform

Warning: not all domain names are yet managed by Terraform. If a domain's comment doesn't start with `[terraform]`, you'll need to manage it manually from the UI. Work is underway to migrate all domains though.

Warning: `terraform/services/dns` only contains resources pointing to resources managed outside of Terraform. If you use a resource it will automatically add the required providers to the configuration. See the documentation to learn where its Terraform configuration lives.

DNS records are managed in the `terraform/services/dns` configuration. A file named after the domain name, `domain.tf`, is created for each managed domain, and it contains some basic information about the domain.

The configuration supports adding A, CNAME, MX and TXT records. The definition contained in the domain's file, each record is defined by its name, while the values are defined in the `records` block.

For example, to add a `pages.rust-lang.org` CNAME record, you'll need to add this to `terraform/services/dns/domain.tf`:

```
module "rust_lang_org" {  
  # ...  
  
  CNAME = {  
    "pages.rust-lang.org." = ["rust-lang.github.io"]  
    # ...  
  }  
}
```

Once you made all the changes you can apply them

```
terraform apply
```

Managing DNS for a new domain

Setting up Terraform to manage the DNS records of a domain is a multi-step process. First of all you need to decide the identifier used for the domain. By convention, the identifier is the domain name itself. For example `rust-lang.org` becomes `rust_lang_org`.

Then you can create a file in `terraform/services/dns` ending in `.tf`, with this content (take care of replacing the placeholders):

```
module "<IDENTIFIER>" {  
  source = "./domain"  
  
  domain = "<DOMAIN-NAME>"  
  comment = "<COMMENT-FOR-THE-DOMAIN>"  
  ttl = 300  
}
```

Finally you need to output the ID of the Route53 zone created by Terraform configuration to add records. Add this snippet to `terraform/services/dns/outputs.tf`:

```
# ...  
  
output "zone_<IDENTIFIER>" {  
  value = module.<IDENTIFIER>.zone_id  
}
```

Once you're done you can apply the changes with:

```
terraform init
terraform apply
```

Adding subdomain redirects

Our Terraform configuration supports creating redirects from subdomains we control to an URL. Redirects are created as infrastructure:

- A S3 bucket for each set of redirects, named `rust-lang-redirects`. The bucket has website hosting enabled, configured to redirect requests to the chosen URL. This allows implementing redirects without an underlying server.
- An ACM certificate (plus the DNS records to validate it) for all the sources as alternate names. This is used for all redirects.
- A CloudFront distribution for each set of redirects, configured to use the previously generated ACM certificate and the S3 bucket.
- Route53 records for each redirect in the related module, using the ACM and ALIASEs for apex domains.

All the redirects are defined in `terraform/redirects`. You can create a new destination URL. Either create a new module if you need a new subdomain to an existing module. See an example of creating a new subdomain (replacing the placeholders):


```
module "redirect_<IDENTIFIER>" {
  source = "../modules/subdomain-redirect"
  providers = {
    aws      = "aws"
    aws.east1 = "aws.east1"
  }

  to = "<DESTINATION-URL>"
  from = {
    "<SUBDOMAIN-1>" = module.dns.zone_<DOMAIN>
    "<SUBDOMAIN-2>" = module.dns.zone_<DOMAIN>
  }
}
```

Once you made all the changes you can apply the c

```
terraform init
terraform apply
```

Note that each change is going to take around 15 m
distribution changes are really slow to propagate. A
resources being recreated when a domain is added
as the ACM certificate will need to be regenerated.

Transferring domain names to R

These are the steps a member of the infrastructure
domain name to the Rust project's registrar:

1. Ask inside the infrastructure team if this is a d
own. In some more complicated cases this will
Leadership Council.
2. If the domain name doesn't already use AWS R
current owner of the domain a list of all the DI
migrated. Then, add all the records to a new h
transfer of the domain. See the [section below](#)
information on this step.
3. Ask the current owner to unlock the domain n
code from them. The transfer code is key to tr

receiving it on public communication platform

4. Go to the [Transfer Domain](#) section of AWS Route 53. If it doesn't give an error (which should detail what the transfer code you received earlier, and choose the correct zone (it should auto-complete the right one). Use Pietro's details as the domain contacts. Finally, start the transfer process.
5. Tell the current owner to wait for an email from AWS on a link to confirm the domain name transfer.
6. The transfer process will take a while. Once approved, telling the domain has been transferred you're done.

Transferring DNS

Most domain names use their registrar as the DNS server. When a domain is transferred away the old registrar also stops managing it, so that we need to ensure all the DNS records are correct before actually starting the transfer process.

Explicitly ask the current domain owner for all the [DNS](#) records. Everything except the [MX](#) records needs to be in a [configuration](#) (create a new file for the domain name and all the domain names).

If you notice some of the records are referring to HTTP on the current registrar then those will have to wait until the transfer occurred, add a new [domain redirect](#) on the registrar before the transfer to be able to request the TLS certificate.

If the domain has [MX](#) records those will need to be updated with the new domain and add the domain name there. Ensure it's in the U.S. and has a 1024 bit DKIM key (the 2048 keys do not fit into the DNS). Copy all the records except the [CNAME](#) tracking one into the [configuration](#), and wait for the domain to be transferred back to Mailgun and verify the DNS settings for the domain in the [team repository's config.toml](#) and create the new DNS records as usual process.

docs.rs

- Source code: [rust-lang/docs.rs](#)
- Hosted on: [docsrs.infra.rust-lang.org](#) (bel
- Maintainers: [Joshua Nelson](#), [Pietro Albini](#)
- [Instance metrics](#) (only available to infra team r
- [Application metrics](#) (only available to infra team
- [Common maintenance procedures](#)

ECS services management

Some applications running on the project's infrastructure are hosted on our AWS account. This document explains the common tasks you should follow when operating them. Most of the actions require [AWS access](#).

Note: our ECS cluster is located in the Northern California region. Make sure it's the selected region when interacting with the AWS console.

Inspecting the logs

Logs for applications hosted on ECS are stored in CloudWatch. To view them in the AWS Console, [Open the console](#), go to CloudWatch, and click on the link called `/ecs/<service-name>`. There are two ways to inspect the logs:

- If you need to look at the application as a whole, click on the "View all log events" button (or, on the "Log Group" tab).
- If you need to debug a specific instance of a container, the logs for each running task are available. The streams are named after the task and the task ID.

Logs are periodically purged (retention varies based on the log group).

Restarting an application

To restart an application, you can force a new deployment of the new code beforehand. To do so, run this command:

```
aws ecs update-service --cluster rust-ecs-prod --service rust-ecs-prod --force-new-deployment
```

Rolling back a deployment

To rollback a bad deployment you can run the `aws-s` (simpleinfra repository) with your [AWS credentials](#) providing the name of the ECR container image repository as

```
./aws-rollback.py <image-repository-name>
```

The script will show the list of images available in the repository and you can enter a number to rollback to. Once that's inserted the script will deploy the image you chose, and if an ECS service with the same name exists, the service will be restarted too.

Deploying application changes

Each application stores its own Docker container image. You can deploy changes both manually and automatically.

For production applications it's recommended to set up a CI/CD pipeline.

Manual deployments

To manually deploy a local build you first need it to be tagged with the repository name:

```
docker tag <image-tag> 890664054962.dkr.ecr.us-west-1.amazonaws.com/<repository-name>:latest
```

Then you can authenticate with ECR and push it:

```
$(aws ecr get-login --no-include-email --region us-west-1)
docker push 890664054962.dkr.ecr.us-west-1.amazonaws.com/<repository-name>:latest
```

Finally, you need to force a new deployment of the service.

```
aws ecs update-service --cluster rust-ecs-prod  
force-new-deployment
```

Automatic deployments with GitHub Actions

The infrastructure team prepared an action for GitHub Actions to perform deployments from CI. To use it, ask a team member to create a repository, and then add this snippet to your workflow.

```
- name: Build the Docker image  
  run: docker build -t deploy-image .  
  
- name: Deploy to production  
  uses: rust-lang/simpleinfra/github-actions/deploy-ecs  
  with:  
    image: deploy-image  
    repository: <ecr-repository-name>  
    region: us-west-1  
    redeploy_ecs_cluster: rust-ecs-prod  
    redeploy_ecs_service: <service-name>  
    aws_access_key_id: "${{ secrets.AWS_ACCESS_KEY_ID }}"  
    aws_secret_access_key: "${{ secrets.AWS_SECRET_ACCESS_KEY }}"  
  if: github.ref == 'refs/heads/<deploy-branch>'
```

Be sure to replace `<ecr-repository-name>`, `<service-name>` with the correct values for your workflow. Once the workflow is set up on the branch you chose for deploys, any future commits pushed to that branch will trigger a new deployment to the ECS cluster.

Monitoring

- Hosted on: `monitoring.infra.rust-lang.org` (`connect`)
- Maintainers: [pietroalbini](#), infra team
- Public URL: [grafana.rust-lang.org](#)
- [Ansible playbook](#) to deploy this server.
- [Instance metrics](#) (only available to infra team members)

Service configuration

Our monitoring service is composed of three parts: Prometheus to monitor metrics, [Alertmanager](#) to dispatch the alerts, and [Grafana](#) to display the metrics. All the parts are configured in the `monitoring` directory.

The metrics are not backed up, as Prometheus purges old data. Grafana dashboards are stored in a PostgreSQL database. Backups of the database are stored in the `rust-backups` bucket (the `monitoring` subdirectory). The password for the database is in `1password`.

Common maintenance procedures

Scrape a new metrics source

Prometheus works by periodically scraping a list of targets in [its custom format](#). In our configuration the list is located in the `targets` section of the `ansible/playbooks/monitoring.yml` file.

To add a new metrics source, add your endpoint to the list. The targets for scraping are not related to any other job, a new one will be created from the monitoring instance. You can read the [Prometheus documentation](#) for the available options.

Create a new alert

Alerts are generated by Prometheus every time a rule evaluates to true. In our configuration the list of rules is defined in the `prometheus_rule_groups` section of the `ansible/p ansible/playbooks/simpleinfra` repository.

To add a new alert you need to create an alerting rule. The full list of options is available in the [Prometheus documentation](#).

Add permissions to a user

There are two steps needed to grant access to our Grafana instance.

First of all, to enable the user to log into the instance, the user must be a member of a team authorized to log in. The `grafana_github_teams` section of the `ansible/playbooks/simpleinfra` repository, and it contains a list of GitHub teams. To run this command:

```
curl -H "Authorization: token $GITHUB_TOKEN" https://api.github.com/orgs/<ORG>/teams/<NAME>
```

Once the user is a member of a team authorized to access Grafana, the user must be added to the main Grafana organization with “viewer” permissions. If the user is a team member that needs to be changed to “admin” or “server admin” otherwise leave it as viewer.

By default a viewer only has access to the unrestricted dashboards. For other dashboards you’ll need to add them to a team (see the [Grafana documentation](#) page). It’s also possible to grant admin privileges to a user. To do this, go to “Server Admin” -> “Users” -> “<username>” page. Do this for all trusted infra team members.

Additional resources

- [Prometheus documentation](#)
- [Grafana documentation](#)

rust-bots

- FQDN: `bots.infra.rust-lang.org` (behind the proxy)
- [Instance metrics](#) (only available to infra team members)

Common maintenance procedures

Adding a new domain

First, edit `sudo vim /etc/nginx/nginx.conf` to edit the domain.

```
server {
    listen 443 ssl;
    listen [::]:443 ssl;
    server_name <domain>.infra.rust-lang.org;

    location /.well-known/acme-challenge {
        root /home/ssl-renew/challenges;
    }

    location / {
        # configure the domain here
    }
}
```

Then run `sudo -i -u ssl-renew vim renew.sh`. Add the domain you're adding.

Then, run the script: `sudo -i -u ssl-renew ./renew.sh`

How the Rust CI works

Continuous integration (CI) workflows on the `rust-`
`master` branch is always in a valid state.

The CI infrastructure is described in detail in the [rus](#)

Sentry

The infrastructure team manages a [Sentry](#) organization to use. The instance is generously sponsored by Sentry to use it.

Log into the instance

Every member of the `rust-lang` GitHub organization can log into the instance, using their GitHub credentials. Visit [the authentication page](#) and click the “Sign-On” tab and enter the `rust-lang` Organization with your GitHub Account!

If this is the first time signing into our Sentry organization, you will see [access to the teams you're on](#). Once you request access to a team, the team will approve it.

Request a new project

If you're a member of a Rust Team and you want to request a new project, you need to follow these steps:

1. If the project is public facing (i.e. people outside the team can use it), you need to contact the Leadership Council to review the privacy policy, adding a note that your service complies with existing ones.
2. Once the privacy policy is sorted out (whenever possible), the infrastructure team will create a new project in the new Sentry team.
3. Finally, you can integrate the Sentry SDK with your application.

Creating a new project

This section documents how the infrastructure team can create a project when requested. You need to either have a personal Sentry account or the necessary permissions, or access to the Sensitive 1Password vault (if stored).

To create a project, authenticate in Sentry and visit the page for the technology stack the team is using, a relevant name for the project (you can create new teams by clicking the “+” icon). Finally, add the relevant people to it.

Language

This section documents meta processes by the lang

External Links

- The language team has communications chan

RFC Merge Procedure

Once an RFC has been accepted (i.e., the final comments have been raised), it must be merged. Right now this is done by the subteam members (if you're not a subteam member, you can request to be added rather than merge the RFC manually). Here is the current procedure. In some cases, not all the steps will be applicable.

Step 1: Open tracking issue

Open a tracking issue over on rust-lang/rust. Here is a template. You have to adjust the various places labeled XXX with something (the name of the RFC, or the most appropriate team).

```
This is a tracking issue for the RFC "XXX" (link to RFC)
```

****Steps:****

- [] Implement the RFC (cc @rust-lang/XXX -- see instructions?)
- [] Adjust documentation ([see instructions on rustc-dev-guide])
- [] Stabilization PR ([see instructions on rustc-dev-guide])

```
[stabilization-guide]: https://rustc-dev-guide.rust-lang.org/stabilization_guide.html#stabilization-guide
```

```
[doc-guide]: https://rustc-dev-guide.rust-lang.org/stabilization_guide.html#documentation-guide
```

****Unresolved questions:****

```
XXX --- list all the "unresolved questions" here to ensure they are not forgotten
```

Add the following labels to the issue:

- **B-rfc-approved**
- **C-tracking-issue**
- the appropriate **T-XXX** label

(If you don't have permissions to do so, leave a note in the issue.)

asking them to do so.)

Step 2: Merge the RFC PR itself

In your local git checkout:

- Merge the RFC PR into master in your fork
- Add a commit that moves the file name from C to D
- Edit the new file to include links to the RFC PR created in the header
- Open a PR or push directly to the master branch

Step 3: Leave a comment

Leave a final comment on the PR directing everyone to this, but feel free to add your own personal flavor (a

```
**Huzzah!** The @rust-lang/lang team has de  
To track further discussion, subscribe to the  
rust-lang/rust#41517
```

That's it, you're done!

Triage meeting procedure

This page documents how to run a lang team triage meeting. It is a bit of a misfortune of being forced to do so.

Attending a meeting

If you would just like to **attend** a lang-team triage meeting, join the zoom call (the URL is attached to the calendar invite).

Scheduling

Note that the scheduling for all meetings is recorded in the [lang-team](#) repository. It can be found on the [rust-lang/lang-team](#) repository.

Pre-triage

To start, we have a pre-triage meeting which occurs every week and is recorded. It is boring.

To execute this meeting you:

- Open the [Current Meeting](#) dropbox paper document
- Skim down the action items and look to see if they have been handled
 - they can be checked off and removed
- Skip down to the Triage section
- For each Triage section, click on the link and prepare to discuss
 - typically it is best to copy-and-paste the text to have it intact
- For each item, click in and try to add a few notes
 - look for things where there isn't much discussion
 - these can be handled quickly in the meeting

- items that require more discussion will n

Main meeting

- Ping the team on discord `@lang-team`
- Begin the recording on Zoom, if you have access
 - If nobody has access to the recording, oh
- Discuss item by item and take some notes on y
 - Add specific actions to the action items s
 - If a consensus arises, make sure to creat
 - The goal should be that we leave some c

After meeting

- Export the meeting file to markdown
 - you will need to cleanup “check boxes” –
`^(\\s*)[ ]` with `\\1* [ ]` or something l
which makes them valid markdown
- Upload video to youtube if applicable and get
- Add the file to the `minutes` directory of `rust-lan`
name like `YYYY-MM-DD.md`

Libs

This section documents meta processes by the Libs

Where to find us

The [rust-lang/libs-team](#) GitHub repository is the on current project groups, upcoming meetings, and

The Libs team hangs out primarily in [the rust-lang Zulip stream](#).

You can also find out more details about [Zulip and h](#)

Maintaining the standard

Everything I wish I knew before somebody gave me

This document is an effort to capture some of the c maintain the Rust standard library. It's goal is to hel process and experience they bring to working on th can benefit. It'll probably accumulate a lot of trivia t members of the wider Rust community.

This document doesn't attempt to discuss best prac [API Guidelines](#).

Contributing

If you spot anything that is outdated, under specifie feel free to open up a PR on the [rust-lang/rust-f](#)

Terms

- Libs. That's us! The team responsible for devel standard library (among other things).
- Pull request (PR). A regular GitHub pull reques
- Request for Comment (RFC). A formal docume introduces new features.
- Tracking Issue. A regular issue on GitHub that'
- Final Comment Period (FCP). Coordinated by chance to review RFCs and PRs.

If you're ever unsure...

Maintaining the standard library can feel like a daunt. Automated reviewer assignment via `triagebot`, you can review in new contexts.

Ping the `@rust-lang/libs` team on GitHub anytime.

If you don't think you're the best person to review a PR, ping somebody else.

Finding reviews waiting for your input

Please remember to regularly check <https://rfcbot.rs/fcp/>. If you have a GitHub nickname to go to a page like <https://rfcbot.rs/fcp/SimonSapich/> to see reviews that are waiting for your input.

Reviewing PRs

As a member of the Libs team you'll find yourself asked to review PRs and your input requested on issues in the Rust project.

When is an RFC needed?

New unstable features don't need an RFC before they are accepted. Small, and the design space is straightforward, stable features go through FCP. Sometimes however, you need an RFC for stabilizing.

Is there any unsafe?

Unsafe code blocks in the standard library need a code review. There's a `tidy` lint that checks this. The unsafe code blocks in the standard library need a code review.

The rules around what's sound and what's not can be found in the [Guidelines WG](#) for current thinking, and consider ping the team.

`lang/lang`, and/or somebody from the WG if you're soundness of unsafe code, and the more eyes on it

Is that `#[inline]` right?

Inlining is a trade-off between potential execution speed. There's some discussion about it in [this PR to the handbook](#).

`#[inline]` is very different than simply just an inlined item. There's no equivalent in C++ for what `#[inline]` basically ignores `#[inline]`, pretending you didn't. The compiler will, by default, codegen an `#[inline]` referencing codegen unit, and then it will also add you have 16 CGUs and they all reference an item's entire item's implementation inlined into it.

You can add `#[inline]` :

- To public, small, non-generic functions.

You shouldn't need `#[inline]` :

- On methods that have any generics in scope.
- On methods on traits that don't have a default implementation.

`#[inline]` can always be introduced later, so if you removed.

What about `#[inline(always)]`?

You should just about never need `#[inline(always)]` helper methods that are used in a limited number of micro benchmark should justify the attribute.

Is there any potential breakage?

Breaking changes should be avoided when possible

what constitutes a breaking change. Breakage may depend on its actual impact, which can be approximated with

There are strategies for mitigating breakage depending on

For changes where the value is high and the impact is

- Using compiler lints to try phase out broken behavior

If the impact isn't too high:

- Looping in maintainers of broken crates and suggesting

Is behavior changed?

Breaking changes aren't just limited to compilation time. Changes to function signatures generally can't be accepted. See [the home](#)

Are there new impls for stable traits?

A lot of PRs to the standard library are adding new implementations that can break consumers in many weird and wonderful ways. Here are some examples of breakage from new trait impls that have been made to the standard library.

Inference breaks when a second generic impl is added

Rust will use the fact that there's only a single impl for a trait. This breaks once a second impl makes the type of the

```
// in `std`  
impl From<&str> for Arc<str> { .. }
```

```
// in an external `lib`  
let b = Arc::from("a");
```

then we add:

```
impl From<&str> for Arc<str> { .. }
+ impl From<&str> for Arc<String> { .. }
```

then

```
let b = Arc::from("a");
```

will no longer compile, because we've previously been using `from` for the `T` in `Box<T>`.

This kind of breakage can be ok, but a `crater` run shows

Deref coercion breaks when a new impl is introduced

Rust will use deref coercion to find a valid trait implementation directly. This only seems to occur if there's a single implementation. If we break consumers relying on deref coercion. Say we

```
// in `std`
impl Add<&str> for String { .. }

impl Deref for String { type Target = str; .. }
```

```
// in an external `lib`
let a = String::from("a");
let b = String::from("b");

let c = a + &b;
```

then we add:

```
impl Add<&str> for String { .. }
+ impl Add<char> for String { .. }
```

then

```
let c = a + &b;
```

will no longer compile, because we won't attempt to dereference `&str`.

This kind of breakage can be ok, but a `crater` run s

Could an implementation use existing fun

Types like `String` are implemented in terms of `Vec` through deref coercion. `Vec<T>` can use methods of possible, methods on a wrapping type like `String` exist on their underlying storage or deref target.

Are there `#[fundamental]` items involv

Blanket trait impls can't be added to `#[fundamental]` coherence rules. See [RFC 1023](#) for details. That inclu

- `&T`
- `&mut T`
- `Box<T>`
- `Pin<T>`

Is specialization involved?

Specialization is currently unstable. You can track its

We try to avoid leaning on specialization too heavily implementations. These specialized optimizations u implementation, rather than specializing the public specialization that changes how methods are dispat carefully considered.

As an example of how to use specialization in the st creating an `Rc<[T]>` from a `&[T]`:

```
impl<T: Clone> From<&[T]> for Rc<[T]> {  
    #[inline]  
    fn from(v: &[T]) -> Rc<[T]> {  
        unsafe { Self::from_iter_exact(v.iter()) }  
    }  
}
```


It would be nice to have an optimized implementati

```
impl<T: Copy> From<&[T]> for Rc<[T]> {
    #[inline]
    fn from(v: &[T]) -> Rc<[T]> {
        unsafe { Self::copy_from_slice(v) }
    }
}
```

Unfortunately we couldn't have both of these impls. This is where private specialization can be used to c internally. In this case, we use a trait called `RcFromSlice` implementation:

```
impl<T: Clone> From<&[T]> for Rc<[T]> {
    #[inline]
    fn from(v: &[T]) -> Rc<[T]> {
        <Self as RcFromSlice<T>>::from_slice(v)
    }
}

/// Specialization trait used for `From<&[T]>`
trait RcFromSlice<T> {
    fn from_slice(slice: &[T]) -> Self;
}

impl<T: Clone> RcFromSlice<T> for Rc<[T]> {
    #[inline]
    default fn from_slice(v: &[T]) -> Self {
        unsafe { Self::from_iter_exact(v.iter()) }
    }
}

impl<T: Copy> RcFromSlice<T> for Rc<[T]> {
    #[inline]
    fn from_slice(v: &[T]) -> Self {
        unsafe { Self::copy_from_slice(v) }
    }
}
```

Only specialization using the `min_specialization` feature is known to be unsound.

Are there public enums?

Public enums should have a `#[non_exhaustive]` at variants being introduced, so that they can be added

Does this change drop order?

Changes to collection internals may affect the order of drops, but this has not been accepted in the past, but should be noted.

Is there a manual `Drop` implementation?

A generic `Type<T>` that manually implements `Drop` with the `[may_dangle]` attribute is appropriate on `T`. The `Drop` implementation is all about.

If a generic `Type<T>` has a manual drop implementation for `T` then `Drop` needs to know about it. If `Type<T>` is used through types that don't drop `T` themselves such as `MaybeUninit<T>` then `Type<T>` also needs a `PhantomData<T>` marker field. Types in the standard library that don't need a `PhantomData<T>` marker field. The `Unique<T>`.

As a real-world example of where this can go wrong, here's something that looks something like this:

```

struct OptionCell<T> {
    is_init: bool,
    value: MaybeUninit<T>,
}

impl<T> Drop for OptionCell<T> {
    fn drop(&mut self) {
        if self.is_init {
            // Safety: `value` is guaranteed
            // Safety: The cell is being dropped
            // `is_init` is true.
            // again.
            unsafe { self.value.assume_init_drop(); }
        }
    }
}

```

Adding a `#[may_dangle]` attribute to this `OptionCell` `PhantomData<T>` marker field opened up a [soundness hole](#) that could allow the `OptionCell<T>` to outlive the `OptionCell<T>`, and so could be accessed after its `Drop` implementation. The correct application of the `PhantomData<T>` field:

```

struct OptionCell<T> {
    is_init: bool,
    value: MaybeUninit<T>,
+   _marker: PhantomData<T>,
}

- impl<T> Drop for OptionCell<T> {
+ unsafe impl<#[may_dangle] T> Drop for OptionCell<T> {

```

How could `mem` break assumptions?

`mem::replace` and `mem::swap`

Any `Sized` value behind a `&mut` reference can be replaced by `mem::replace` or `mem::swap`, so code shouldn't assume that references can't have their internals changed by replacement.

`mem::forget`

Rust doesn't guarantee destructors will run when a value is dropped (e.g. with `mem::forget`), so code should avoid relying on destructors. Remember, *everyone poops*.

It's ok not to run a destructor when a value is leaked or repurposed. If the storage is initialized and is being used, destructors need to be run first, because *memory management* can still be exceptions for skipping destructors when there's never pinning involved.

How is performance impacted?

Changes to hot code might impact performance in production. Appropriate benchmarks should give an idea of how. For changes that affect `rustc` itself, you can also do

Is the commit log tidy?

PRs shouldn't have merge commits in them. If they do, they need to be rebased.

Merging PRs

PRs to `rust-lang/rust` aren't merged manually using `git` branches. Everything goes through `bors`.

When to rollup

For Libs PRs, rolling up is usually fine, in particular if it only touches docs.

See the [rollup guidelines](#) for more details on when to merge a number of PRs together and merge them all at once. This can be things merged faster, but might not be appropriate if it causes conflict, or have performance characteristics that w

When there's new public items

If the feature is new, then a tracking issue should be created. Check previous [tracking issues](#) to get an idea of what needs to be done. `[unstable]` attributes should be updated with the tracking issue.

Unstable features can be merged as normal through a PR.

When there's new trait impls

There's no way to make a trait impl for a stable trait. **impls for already stable traits must go through a tracking issue.** If the trait itself is unstable though, then the impl needs to be unstable.

When a feature is being stabilized

Features can be stabilized in a PR that replaces `#[unstable]` with `#[stable]` ones. The feature needs to have an accepted RFC before merging through a FCP before merging.

You can find the right version to use in the `#[stable]` attribute.

When a `const` function is being stabilized

Const functions can be stabilized in a PR that replaces `#[unstable]` attributes with `#[rustc_const_stable]` ones. The compiler is pinged for input on whether or not the `const`-ness is an intrinsic being exposed that is const-stabilized. It should be included in the FCP.

Check whether the function internally depends on `unsafe` through `#[allow_internal_unstable]` attributes and whether it's implemented if its internally unstable calls were removed. For more details on `#[allow_internal_unstable]`.

Where `unsafe` and `const` is involved, e.g., for operating on `const` safety argument for the usage also be documented.

additional determinism (e.g. run-time/compile-time function's output only depends on its inputs...) restrictions those should be argued when `unsafe` is used.

When a feature is being deprecated

To try reduce noise in the docs from deprecated items, move them to the bottom of the module or `impl` block so they're rendered last. The docs should then be cut down to focus on why it's deprecated and how you might use it.

Release

This section documents the process around creating releases as well as information on The Rust Programming Language.

External Links

- The [Homu/Bors](#) page provides links to the pull requests in the [rust-lang](#) GitHub organisation, as well as providing a way for users to use to interact with it.
- [Rustup Component History](#) documents when a component (or was available) for a specific platform on nightly.
- [PR Tracking](#) provides visualisations of pull requests in the repository.
- kennytm's [rustup-toolchain-install-master](#) project integrates generated artifacts from CI into [rustup](#).

Backporting

There's a steady trickle of patches that need to be ported back to the beta branch. Only a few people are even aware of the process, but you can do it.

Beta backporting in rust-lang

Backports of PRs to the beta branch are usually only done for security fixes. Backporting to the beta branch involves the following steps:

1. Add the label `beta-nominated` to the PR to bring it to the attention of the team. This indicates the state that it needs attention from the appropriate people to be backported. Anybody with triage access is free to backport it.
2. If the team thinks it should be backported, they will add the `accepted` label. Otherwise they should remove the label.
3. Occasionally someone will make a beta rollup, but it can be done by anyone. The process is as follows:
 1. Create a local branch off the `beta` branch.
 2. Cherry-pick all of the PRs that have both the `beta-nominated` and `accepted` labels. It is usually preferred to not include PRs that are not yet merged, in case there are any last minute changes, or if the PR is still under review.
 3. Run `./x.py run replace-version-placeholders` to update the version numbers. Changes, put them into a new commit.
 4. (Recommended) Run some tests locally. The changes may not apply cleanly, or the UI tests need to be updated to reflect differences in the output.
 5. Open a PR **against the beta branch** with the changes (this way reviewers can see its specialness).
 6. List all of the PRs being backported in the PR description.

7. Go through all of the PRs being backport

- Change the milestone to the correct
- Remove the `beta-nominated` label. been completed.

If there are a lot of PRs, this can be done `accepted` query, check all the PRs being b and “Label” drop-downs to modify multip

This last step can be done before or after though it can be easy to forget if you wait

4. A reviewer (typically from the release team) ne correct and that it's submitted to the beta bran `@bors r+ rollup=never` (to avoid it being roll the PR has r+ rights, and has not made signific can also self-approve the PR.

In summary, there are three states that a PR can go

1. `beta-nominated` : Needs the team's attention.
2. `beta-nominated` + `beta-accepted` : Waiting to
3. `beta-accepted` : Backport complete.

Stable backporting in rust-lang

Backports to the stable branch work exactly the same slightly different name: `stable-nominated` identifies and `stable-accepted` is a PR accepted for backport have the `stable-nominated` label removed.

The `T-release` will decide on a case by case basis if (.patch) release (f.e. release a `1.50.1` between `1.5`

Beta Backporting in rust-lang

The procedure for backporting fixes to Cargo is similar to the `rust-lang/rust` repo's procedure. Currently the Cargo team does not have a dedicated repository, but you'll initiate the backport process by requesting a backport. Once a Cargo team member approves your request, you're good to start sending PRs!

- First you'll send a PR to the `rust-1.21.0` branch (the current rustc beta version number). Like with `rust-lang/rust`, prefix the title of your PR with `[beta]` and ensure it's flagged as a beta change.
- Next a Cargo reviewer will `@bors: r+` the PR and bors will automatically merge the PR (when tested on the Cargo branch).
- Finally you'll send a PR to the `rust-lang/rust` repo to update the Cargo submodule. The Cargo submodule should be on the `rust-1.21.0` branch (the branch your Cargo PR was merged to). Ensure you've got `[beta]` in the PR title.

After that's all said and done the Cargo change is ready for release!

Preparing Release Notes

This document discusses the process for preparing and the blog post (high level changelog) announcing

Automation/community: Step 1: for release note consideration

This step happens for `relnotes`, `relnotes-perf`, and labels in rust-lang/rust and is managed by triagebot

Note that this step happens **on labeling**, not necessarily merged/closed. (FIXME: Should we move this to closed authors see the relnotes issue when it's warranted,

Automation/community: Step 2: note text is proposed

Once labeled, a new issue is automatically filed with title of `#xxxx: $original pr title`. This is labeled with relnotes

The Rust team responsible for the area and/or content use this issue to propose any release note text. The release note contents (to go into RELNOTES.md) and

Ideally, text is directly edited into the code block, but discussion can happen on the thread and will be able to

The release note text is automatically pulled in subsequent headers from [this list](#) if possible:

- Compatibility Notes
- Library
- Stabilized APIs
- Const Stabilized APIs

- Language
- Compiler
- Internal Changes
- Other

Stabilized APIs and Const Stabilized APIs should both

– [``std::ptr::null_mut``](https://doc.rust-lang.org/std/ptr/fn.null_mut.html)

Note that:

- this is not a PR link, but directly links the standard library
- the link is to stable docs (and so may not actually be the latest)
- the API is directly noted. Sometimes we compare integers) to avoid too much text.

Release team: Step 3: Confirm all relnotes are labeled relnotes

This steps should happen in the first 3 weeks of the release cycle and can be done with help from the wider Rust project team.

Search for `is:pr milestone:1.85.0 is:merged -label:finished-final-comment-period` in GitHub and label the milestone appropriately.

This should find all merged PRs that haven't already been labeled. There will be several hundred of them; the goal is to try to find all that *have been nominated* and nominate it by tagging with `relnotes` without clicking through and using the GitHub check strategy.

The goal here is mostly **catching obvious things**, not perfect. It's OK if we miss something. If there's a consistent pattern, triagebot's automatic issue filing.

FIXME: This step may also need to include an attempt to find all tagged relnotes and closed – those currently don't get tracked which means their corresponding relnotes tracking

(letting it get missed indefinitely).

FIXME: Process for Compatibility Notes?

Release team: Step 4: Release notes

This steps should happen in the first 3 weeks of the release team owner for the current release will run

```
cargo build
GITHUB_TOKEN=$(gh auth token) cargo run --bin relnotes.md
```

This produces console output (stderr) like this:

```
Did not use "Libraries" from Tracking issue ...
and undeprecate home_dir() <https://github.co
Did not use "Category (e.g. Language, Compile
notes, ...)" from Tracking issue for release
impls for tuples of arity 1 through 12 <https
lang/rust/issues/133975>
```

These lines typically mean that someone either has the list above, for example:

- Libraries should be Library
- Category ... needs updating.

The runner of the tool should go update these issue (renaming/categorizing). The **source of truth** is the **issue** this is important as we want to automate tool execution to minimize the amount of local edits done on the repository.

The tool's standard output is directed to `relnotes.md` formatted headers for each of the sections with content for ...” **or** standard content (PR/issue title + link) if the content is by the tool.

If an issue is missing, see Step 3 (tag it and re-run the tool) *shouldn't be*, the best thing is to tag it with relnotes (

tracking issue) and *close that tracking issue*. This will output.

Release team: Step 5: Publish release notes

See example from 1.84: <https://github.com/rust-lang/rust/blob/master/RELEASES.md>

Take the `relnotes.md` you have locally (typically with the `rust` world, you'll add them at a later point – we want the best possible), and insert it at the top of `RELEASES.md` in the PR with those contents. You can `@r?` the owner for actual review cycle and cc the release team.

Include a link to this document (<https://forge.rust-lang.org/edition-guide/2018-03-01-releasing-a-new-version.html>) in the PR description, pointing at step 6 (i.e., prefer suggesting a new version).

The next release team meeting should also discuss the release (see below for blog post process).

Pinging `relnotes-interest-group` for review and blog post

Contributors may be interested to help review the release notes (e.g. on behalf of their team). They can opt-in to being pinged by the `relnotes-interest-group` marker team.

When creating a relnotes PR and release blog post,

```
@rustbot ping relnotes-interest-group
```

All: Step 6: Incorporate edits from community

You'll typically get a lot (several dozen) of comments and suggestions for alternative text, etc. A good strategy is to create issues for issues/PRs (or file them and update them as they come in).

already done locally in step 4. The longer state stays in the PR, the more likely it is to incorporate updates from those into the PR (just rebase).

Pushing edits into the issues helps bring the right people into the loop on what is getting discussed.

Release team: Step 7: Close track

At some point, the release team owner should declare the release notes tracking issues associated with the current release. This is easiest in the GitHub UI.

FIXME: Ideally those would all get linked from the release notes and go forth between them.

Blog post process

After publishing the release notes PR, the release team owner should identify the blog post topics we want. Typically this is a pretty informal process; the goal is to provide input into the blog post and get perspectives.

The blog post author will then aim to post the blog post 3-7 days out from the release, though we've had some posts get reviewed, edited, and finally merged on the release day.

The Rust Release Process

Here's how Rust is currently released:

A note about the `start-release` script

Steps of the release process that require interacting with the CI are executed through the `start-release.py` script. The script sets up the programs and configure your local environment, and

The first time you run the script (or when the pre-release script fails) you have to invoke the script *multiple times* until everything is set up.

The `start-release.py` will always start a CI job in the background. You have to watch the logs. When the build finishes

```
Phase complete: UPLOAD_ARTIFACTS State: SUCCESS
```

Bump the stable version number (before)

Open a PR bumping the version number in `src/version.rs` (and approve it).

Mark it as `rollup=never`, because if it lands in a rollup, requests in that rollup will be incorrectly associated with this version.

This is effectively when the beta branch forks – when you create a new branch off of the PR that landed just before this version number.

Promote branches (Monday)

Both promotions should happen on Monday. You can check the status of the

but prioritize landing the stable promotion first (to reduce the risk of a failed promotion period).

Updating the base of the beta and stable branches

Run this command from the [rust-lang/release-team](#) repository:

```
./scripts/start-release.py update-rust-branch
```

Remember that `start-release.py` starts a job in the `rust-lang/release-team` repository *before* the branches are updated. Watch the logs to ensure the job completes successfully before proceeding.

stable PR

Send a PR to [rust-lang/rust](#) targeting the new `stable` branch with the following changes:

- Update release notes to the latest available commit.
 - If the release notes PR was merged:

```
git checkout origin/master -- RELEASES.md
```

- Otherwise, manually copy `RELEASES.md` from the `stable` branch.
- Update `src/ci/channel` to `stable`.

Self-approve the PR with `r+ rollout=never p=1000`.

Note that we need to merge this PR as soon as possible to reduce the testing time. If another PR is being tested by bors, and you (at your judgement here), you can “yield” priority to the other PR and typing this comment:

```
@bors retry  
Yield priority to the stable release.
```

beta PR

Send a PR to [rust-lang/rust](#) targeting the new `beta`

- Run this command and create a **separate commit**

```
./x.py run replace-version-placeholder
```

- Update `src/ci/channel` to `beta`

Self-approve the PR with `r+ rollout=never p=10`.

Publish the pre-release on the dev-stable

After the `stable` PR is merged you'll need to start the release from the [rust-lang/release-team](#) repository¹:

```
./scripts/start-release.py publish-rust-dev-stable
```

You need to replace `YYYY-MM-DD` with the date of the release.

Master bootstrap update (Tuesday)

This step can only be done after the new beta has been released. The beta happens automatically at 00:00 UTC every day, so you will have to wait another day. You can check when the next beta is installing it with `rustup`.

Send a PR to the master branch to:

- Cherry pick the commit that ran `replace-version` to the merged beta branch PR. Do not re-run the toolchain stabilizations on master which were not included in the beta. They will be attributed to the current release.
- Run this to update the bootstrap compiler to the new beta

```
./x.py run src/tools/bump-stage0
```

- Remove references to the `bootstrap` and `not` attributes. You can find all of them by installing

```
rg '#!?\[.*\.(bootstrap)' -t rust -t toml
```

The general guidelines (both for `#[ ]` and `#![ ]`)

- Remove any item annotated with `#[cfg(bootstrap)]`
- Remove any `#[cfg(not(bootstrap))]` and
- Remove any `#[cfg_attr(bootstrap, ...)]` item.
- Replace any `#[cfg_attr(not(bootstrap), ...)]` with the relevant documentation block (or in a new block)
- Replace any `#[cfg_attr(not(bootstrap), ...)]` with the relevant documentation block (or in a new block)

Note that if a PR adds `cfg(bootstrap)` and is the master bootstrap update, the `rg` invocation will have to be removed. The easiest way to handle this is to let CI show you the failure.

- Ensure there are no new warnings or Clippy lint errors

```
./x.py clippy ci
```

Release day (Thursday)

Decide on a time to do the release. You are fully in control of when it happens, pick the time that works best for you. The release must start and finish within the release day (in UTC)

Let the Social Media coordinator (currently Mara) know you are ready to post the release on the project's social media

As of September 2024 a release takes between 75 and 90 minutes. Start the release process earlier enough to hit the time you want

To start the release, Run this command in the [rust-l](#)

```
./scripts/start-release.py publish-rust-prod
```

The command will start a background job to invoke production environment, and it will show the instru

When the release process completes, merge the blo announce the release on social media. Finally, bask

Beta stage0 update (Friday)

Send a PR to the beta branch updating the stage0 to

```
./x run src/tools/bump-stage0
```

Appendix: Rebuilding stable pre-

If something goes wrong and we need to rebuild the [stable](#) branch of the [rust-lang/rust](#) repository. One [\[authenticate with AWS\]\[awscli\]](#) and run this comm repository:

```
./scripts/start-release.py publish-rust-dev-s
```

You'll also want to update the previously published blog and internals with the new information.

¹ Publishing releases require authentication, and only aut invoke it. The command will prompt you on how to setup authenticate with AWS the first time you execute it.

Rollup Procedure

Background

The Rust project has a policy that every pull request can be pushed to master. As PR volume increases this leads to the long (~3.5hr) current CI duration for Rust.

Enter rollups! Changes that are small, not performance dependent are marked with the `rollup` command. When creating a PR and mark as a rollup, `@bors rollup` to mark as a rollup, `rollup-` to un-mark as a rollup). 'Performing a Rollup' changes many changes into one PR and merging them all at once. The values `always`, `maybe`, `iffy`, and `never`. See [the PR](#) for guidance on what these different statuses mean.

You can see the list of rollup PRs on Rust's [Homu queue](#). The 'approved' queue with a priority of 'rollup' means they wait themselves until everything in front of them in the queue is merged.

Making a Rollup

1. Using the interface on [Homu queue](#), select pull request and click the 'rollup' button to make a rollup pull request. (The text 'rollup' is only visible if you are logged in.)
Important note: consider for additional PRs marked with `rollup=maybe` and `rollup=iffy`, based on the [rollup section](#). Be extra careful when deciding what to include in a `rollup=maybe` and `rollup=iffy` PRs. We should be careful not to risk regressions (bugs or perf). Also, don't forget to tag things with `rollup=never`, when they are not explicitly tagged with rollup, be extra careful.
2. Run the following command in the pull request comment:

```
@bors r+ rollup=never p=5
```

3. If the rollup fails, use the logs rust-log-analyzer to find the specific PR and do `@bors r-`. If the PR is running, otherwise, your rollup succeeded. If it did, proceed with the rollup and then let `rollup=never` and toolstate PRs.
4. Recreate the rollup without the offending PRs. Use the rollup PR's body to automatically prefill the rollup (minus any PRs that have been `r-` d)

Selecting Pull Requests

The queue is sorted by rollup status. In general, a good rollup should include `rollup=always` PRs (if available), a bunch of `maybe` (unmarked) PRs, and `rollup=never` PRs. A rollup should never include `rollup=never` PRs.

The actual absolute size of the rollup can depend on the number of PRs. Making rollups might start with including 1 `iffy`, 4 `maybe`, and 1 `rollup=always`. Experienced people might even make a rollup of 1-2 `rollup=always` PRs. Massive rollups are rarely needed, but as your intuition tells you, there is a risk when including PRs in a rollup.

Don't hesitate to downgrade the rollup status of a PR. A `rollup=always` PR has some chances for failures, not a `rollup=iffy`. A lot of the unmarked `maybe` PRs are `rollup=iffy`. A reviewer may not have considered rollupability, so it's worth a critical eye. Similarly, if a PR causes your rollup to fail, downgrade its rollup status.

Generally, PRs, that touch CI configuration or the bootstrap script, are `iffy` and should be handled with care. On the other hand, PRs that are usually `rollup=always`.

Avoid having too many PRs with large diffs or submitters who are new. Also avoid having PRs you suspect will have large performance regressions. Use `rollup=never`.

It's tempting to avoid including `iffy` PRs at all since they might fail. However, it's worth remembering that the rollup is the way to land them. As such, a rollup that fails because of

PR would have to be tested at *some point* anyway and a large amount of time to test if it never got included in a rollup. When it comes to `iffy` PRs is that a rollup is a way for a bors test the CI cycle that the `iffy` PR needs anyway. If rollups aren't happening is that these PRs tend to languish in the queue, not good.

Similarly, make sure to leave some spare CI cycles so that if you're the only person making rollups it's worth letting bors pay attention to the queue, but these days there are many contributors, so it's often best to just keep an eye on the rollup queue a couple CI cycles for `never` PRs, especially if they pass.

Try to be fair with rollups: Rollups are a way for things to get tested. For `rollup=maybe` PRs, try to include the oldest one (at least if PRs aren't jumping the queue over older PRs entirely). If you include older than PRs included in your rollup, but try to include the newest. From a perspective around `iffy`, it's useful to look at a rollup as a piggyback on the CI cycle of the oldest PR in queue.

Failed rollups

If the rollup has failed, run the `@bors retry` command (usually due to a network problem or a timeout). If it wasn't successful, throw it out by copying a link to the rust-logs-analyzer `<link_to_comment>`, `@bors r- <link_to_comment>`. Hopefully, the author will get the PR fixed or confirm that it's not at fault. The rollup will be removed.

Once you've removed the offending PR, re-create your rollup. Sometimes however, it is hard to find the offending PR. If you suspect the PRs that you suspect are the problem and recreate a rollup, raise the priority of the PRs you suspect, mark them as `iffy`, bors test them standalone to dismiss or confirm your suspicion.

If a rollup continues to fail you can run the `@bors r- <link_to_comment>` command to remove the rollup the PR in question.

Triage Procedure

Pull Request Triage

Status Tags

- [S-waiting-on-author](#) - Author needs to make changes, add comments, or merge conflicts/test failures are resolved (in obscure cases, like a PR being blocked on another PR in addition), or waiting for a [crater](#) run – it is then ready to be merged forward.

Also used for work-in-progress PRs, sometimes marked with [WIP](#) in GitHub.

- [S-waiting-on-review](#) - Review is incomplete
- [S-waiting-on-team](#) - A [T-](#) label is marked, and waiting for team feedback
- [S-waiting-on-bors](#) - Currently approved, waiting for bors to merge
- [S-waiting-on-crater](#) - Waiting to see what the impact is on the ecosystem
- [S-waiting-on-bikeshed](#) - Waiting on the consensus of the community
- [S-waiting-on-perf](#) - Waiting on the results of a performance benchmark
- [S-waiting-on-ACP](#) - Waiting on API change proposal
- [S-blocked](#) - Waiting for another PR to be merged before this can proceed
- [S-inactive](#) - Hasn't had activity in a while
- [S-experimental](#) - An experimental PR that should not be used for this, but [S-experimental](#) can be used to experiment to test out some changes.

Also: [PRs with no status tags](#). This is useful to find PRs that need triage.

didn't assign a reviewer and thus didn't assign [S-waiting-on-review](#) otherwise. (Note that you should likely **not** triage PRs with this label, as it means the author does not need a review yet.)

Procedure

We primarily triage three status labels: S-waiting-on-review, S-waiting-on-author (once in a while) S-blocked. Here is the procedure for each.

S-waiting-on-review

Click [this link](#) to see all PRs with the S-waiting-on-review label that have not been last updated 15 days or more ago (give or take a day).

For each PR:

1. If the PR has new conflicts, CI failed, or a new reviewer has been assigned, change the label to S-waiting-on-author and ping the author.
2. Add the PR to your [report](#).

S-waiting-on-author

Click [this link](#) to see all PRs with the S-waiting-on-author label that have not been last updated 15 days or more ago (give or take a day).

For each PR:

1. If the author did what the PR was waiting on the label, change the label to S-waiting-on-review.

Otherwise, if the author still needs to do some work, change the label to S-waiting-on-author **not** a member of a Rust team (does not include `T-compiler`, `T-lang`, `T-rustdoc`, etc.).

2. Add the PR to your [report](#).

S-blocked

You only need to check S-blocked PRs occasionally (once in a while).

see all PRs with the S-blocked label.

For each PR:

1. If it is still blocked then leave it as-is.

Otherwise, if it is no longer blocked, then remove the label (or change it to something like S-waiting-on-review if appropriate).

2. Add the PR to your [report](#).

Triage Report

You should record information about each PR you triage in a small document that looks like:

S-waiting-on-review

#12345 20 days - still waiting on review - author:

[...]

Your report can look different, just make sure you include:

1. The PR number (e.g., [#12345](#)). No need to manually autolink PR (and issue) numbers.
2. Number of days since last activity. "Activity" means:
 - author, reviewer, or team member commented;
 - someone commented about merge conflicts;
 - PR was pushed to;
 - etc.
3. Author, reviewer, and who or what (person, team, bot) commented on.
4. Current status and what the most recent activity was (e.g., commented).

Once you are done triaging PRs, post your report in the `#t-release/triage` Zulip stream. the topic should be `to YYYY-MM-DD`. Note that this uses a monday-sunday format.

If a topic does not exist, you can generate its title with the following command (requires GNU date):

```
echo "$(date -I --date="$(date +%a) = 'last monday')") to $(date -I --date="$(date +%a) = 'today' || echo 'next sunday')")"
```

Avoiding duplicate work

Since triaging is sometimes done by looking at oldest issues, `S-*` labels will update an issue/PR's last-modified time to indicate that it has already been taken care of.

Issue triaging

This page is about the `rust-lang/rust` repository. `lang` GitHub organization likely have different procedures.

Tracking issues (issues labeled `C-tracking-issue`) are treated differently.

Motivation

The `rust-lang/rust` repository has thousands of issues being worked on. It is impossible for all people to check. The goals of triaging are to:

- Improve **discoverability**:
 - Categorize which teams an issue may concern
 - Make sure regressions are properly tagged
 - Connect issues to the relevant people (especially the assignee)
- First-pass **coarse classification** of the issue nature
 - Is this issue a bug, or is this discussion, or is this a feature request?
- Identify if the issue is **sufficiently actionable**:
 - Does the report need further info, like reproducible steps, build/execution environment info?
 - Does the reproducer need minimization?
- Make it easier for contributors to **filter** issues

In practice, it is unrealistic for all issues to be solved. Through applications of labels we make the issue triage process more transparent, so that people in the future have an easier time finding issues they are interested in working on.

Triaging can be done by **everyone**, no matter if you are a contributor or not. We encourage everyone to help here, as triaging is a good way to get started with.

Some actions that require write access to `rust-lang/rust`:

- Using `@rustbot transfer` to transfer the issue to another team

`lang` GitHub organization.

- Closing the issue.
- Using `@rustbot` to apply certain labels that remove labels (see the “removing labels” section below).

If you don’t have write access but you assess that the issue should be closed, just open a new topic in `t-release/triage` zulip channel and link the issue in an existing topic in the `t-release/tria`

Initial triaging

When an issue is opened, it usually receives the `needs-triage` label. This is done so that untriaged issues are easily discovered. For new issues, a new issue receives an initial triaging pass (describe the issue, add a checklist, checkpoint. The effort needed to get an issue past this checkpoint).

To do the initial triage and remove the `needs-triage` label, the following should be fulfilled/considered. It’s okay if not all of these are met. This is a non-exhaustive list as a guideline, not a hard checklist.

- The issue should make sense, that is, it should be a question or a request.
 - For example, if an issue is a question about a feature, it should be closed and the user redirected to `URL` or `Discord`. If the question is about a bug, the question too, but make sure to mention `URL/Discord` next time.
- Check if this issue is a duplicate of earlier-reported issues.
 - If you are certain this is a duplicate, close the issue. Make sure this is obvious in the body of the issue with a link to the duplicate issue.
 - If you are not sure, you can still leave a comment saying “possibly a duplicate, similar, or related.”
- Add appropriate labels (`Labels`).
 - Specifically, team labels (`T-*`) and issue labels (`S-*`) are important, because team members will use these to filter what concerns the team (e.g. during a meeting).
- If the issue contains no reproduction but needs one, ask for one and add the `S-needs-repro` label.
- If the issue lacks information (e.g. system information), add the `S-needs-info` label.

then request that the reporter provide the info to reproduce the problem. Add the `S-needs-info` label.

- The issue tracker is the wrong place for some issues. Ask the author to proper channels:
 - Standard library API requests should follow the [Standard Library API Request Process](#).
 - Language changes should be redirected to the [Rust Language Team](#) (`lang`).
- If the issue could benefit from bisecting the regression, add `E-needs-bisection` (or do the bisection yourself with `bisect-rustc`).
 - If there is a bisection and the bisection output is useful, add `bisection`.
- If the reported example is large and/or complex, has many external dependencies, has unrelated warnings, or is otherwise a problem), add `E-needs-mcve` to indicate the need for a Minimal, Complete, and Verifiable Example.
 - If there is a Minimal, Complete and Verifiable Example, add `S-has-mcve`. Double-check that the MCVE is for the original problem and has the same behavior.
- Is the issue a regression? If so, apply the `regression` label (and what regression it is exactly). Generally speaking, a regression is if something used to work, but no longer does.
- If you happen to know people who this issue is affecting, add `cc` to the issue.
 - For example, write `cc @ThatPerson` if `ThatPerson` worked on the feature in question recently.
- Does this issue require nightly? Add `requires-nightly`.
- Does this issue require incomplete or internal features? Add `{incomplete,internal}-features`.

Applying and removing labels

Users without write access to `rust-lang/rust` can only add labels allowed by the `triagebot.toml` configuration.

Users with write access should change the labels displayed to everyone subscribed to the issue unnecessarily.

For example:

```
@rustbot label +T-compiler +C-bug +A-linkage
```

To see a list of all labels, check out the “labels” page tracker.

Note that some labels may only be applied by users [lang/rust](#). Refer to the [allow-unauthenticated](#) list in [triagebot.toml](#) to see what labels users without write access can apply.

Labels

There are many different labels that can be applied to an issue:

- **needs-triage** : Signals that an issue is new and needs attention.
- **T-*** : Specifies the team or teams that this issue is relevant to, such as **T-compiler**, **T-types** or **T-libs**.
- **WG-*** : Specifies the working groups that this issue is relevant to, such as **WG-debugging**.
- **PG-*** : Specifies the project groups that this issue is relevant to, such as **PG-exploit-mitigations**.
- **C-*** : Specifies the category of the label, for example **C-bug** for a bug report.
 - **C-optimization** : For missed compiler optimizations.
 - **C-defective-hardware** : For hardware bugs.
 - **C-gub** : If the problem is due to user error.
 - **C-discussion** : Not a bug, but still worth tracking.
 - **C-external-bug** : For software bugs that are outside of Rust's control over, but is worth tracking from our perspective.
- **O-*** : For target-specific issues, specifies the code that the issue affects (most notably the platform, i.e., the architecture). Examples include **O-macos**, **O-aarch64**, **O-windows**, **O-windows-gnu**.
 - Where possible, specify the most specific target that the issue affects, such as ***-windows-gnu** but not ***-windows** over the generic **O-windows**.
- **A-*** : The areas that the issue is relevant to, for example **A-diagnostics**. This is particularly helpful for issues that are not bugs.
 - **A-diagnostics** : Issues created from the compiler's diagnostic output.

- `A-diagnostic` and not `C-bug`.
- `B-*`: Issues which are blockers.
- `D-*`: Labels for diagnostic issues.
 - `D-diagnostic-infra`: This issue is about
- `L-*`: When the issue concerns a specific lint.
 - `L-false-positive` if the lint fires on a ca
 - `L-false-negative` if the lint misses a ca
- `F-*`: When the issue concerns a specific (usual
- feature.
- `-Z*`: When the issue concerns a specific unsta
- `requires-nightly`: This issue only affects the
- not affect the beta or stable channel compiler.
- `requires-{incomplete,internal}-features`:
 - internal feature. The latter often means that th
 - accordance with compiler [MCP 620](#).
- `regression-*`: Labels for tracking issues that
 - `regression-untriaged`: The nature and
 - further regression triage is necessary.
 - `regression-from-stable-to-stable`: So
 - stable release, which has already reached
 - earlier stable releases.
 - `regression-from-stable-to-{beta,nigh`
 - the a stable release to beta or nightly cha
- `I-*`: Different labels about the nature² of an i
 - `I-ICE`: Internal compiler error.
 - `I-prioritize`: Indicates that the issue s
 - [compiler prioritization working group \(W](#)
 - applicable to indicate the urgency.
 - `I-heavy`: Heavy code (binary size).
 - `I-slow`: Slow run-time performance.
 - `I-hang`: The compilation fails to complet
 - `I-crash`: The compiler or generated cod
 - ICE, e.g. SIGSEGV or access violations.
 - `I-unsound`: Library, compiler, type syste
 - `I-compilemem`: Excessive memory usage
 - `I-compiletime`: Slow compilation time.
 - `I-{team}-nominated`: Issue is nominated
 - [compiler-nominated](#).

- **I-lang-easy-decision** : The decision ne person applying the label) to be easy or p not imply **I-lang-nominated** ; the nomin simultaneously if the person apply the la lang discussion.
- **P-*** : Priority labels. Applied using the **compile**
- **E-*** : Calls for participation³, for example to m
 - **E-mentor** : A mentor is available to help v good first issue. Make sure that the inter available to mentor.
 - **E-needs-mcve** : This issue has a reproduc be minimized.
 - **E-needs-bisection** : This issue needs a b **bisect-rustc** .
 - **E-needs-investigation** : This issue need root causes and the nature of the issue.
 - **E-needs-design** : This issue will require s (exploration, prototyping, discussions, et
 - **E-needs-test** : The issue has been fixed, After someone adds a test, it can be close
 - **E-{easy,medium,hard}** : Someone has es This can help with finding good first issue
- **S-*** : The status of an issue, for example **S-ne**
 - **S-needs-info** : Needs more information
 - **S-needs-repro** : Needs reproducer (code
 - **S-bug-has-test** : Issue has a known-bug **tests/crashes** test suite or elsewhere.
 - **S-has-bisection** : A bisection has been convincing.
 - **S-waiting-on-LLVM** : Waiting on upstre
 - **S-tracking-forever** : The tracking issue
- **beta-nominated** : Tracks changes nominated f
- **beta-accepted** : Tracks changes that have bee **beta**. **T-release** will usually handle the backp
- **stable-nominated** : Tracks changes nominate anticipation of a point release.
- **stable-accepted** : Tracks changes that have b **stable**. **T-release** will usually handle the back

- **relnotes** : Changes that are proposed to be done in the next release
 - It makes triagebot create a new relnotes issue
 - It also marks relnotes issues, so it can be used in tooling
 - An FCP will also cause a relnotes issue to be created
- **metabug** : Tracks other bugs.

Relnotes issues

Release note issues will currently come with **needs-triage** label. relnotes is usually best done if you have sufficient context.

- Remove unrelated area **A-*** labels that conceal user-facing relnotes.
- Remove unrelated **T-*** or **WG-*** labels.
- Remove **needs-triage** if the relnotes wording is clear, PR author, reviewer or a relevant team member has approved.

Internal Compiler Error (ICE) issue triage

For issues that include an Internal Compiler Error (ICE),

- Make sure the issue is tagged with **I-ICE** , **T-ICE** or **E-ICE** .
- Check that the issue is actually an ICE, and not a **crash** or **I-hang** .
- If it is an older (like latest stable) version of rustc
 - Does the ICE reproduce on beta and latest stable?
 - If older stable, ask if the ICE reproduces on the latest stable.
- Check for duplicates, but don't close as duplicate if it's the same underlying issue. Prefer simply linking to related/duplicate.
- A good way is to search the ICE message, as the same ICE message may have different reproductions.
- If it does not have a reproduction, comment asking for a **repro** .
 - If there isn't one for around a month it should be closed.
- If the reproduction is not minimal, add **E-needs-minimal-repro** .

See the “Initial triaging” section for more detail.

- Add `A-*` labels based on the code that causes the issue and the nature of the repro (e.g. if the repro is a weird `rustc_trait_selection`, add `A-traits`).
- Add `T-*`, `WG-*`, `PG-*`, `F-*`, `requires-*`, and `requires-ice` labels.
 - Usually the ICE should be tagged `T-comp` (compiler system) (e.g. trait solver), also tag the issue with `requires-ice`.

Further triaging

For issues that have been through the initial triaging (no `triage` label anymore), there are usually still things you can do. Often there are often many more labels that could be applied (using `rust-lang/rust` access to `rust-lang/rust`).

Additionally, old (there is no clear definition of old yet, but usually more than 6 months) `S-needs-repro` issues can be closed if they have a reproduction. This requires write access to `rust-lang/rust`. If you can just link the issue on Zulip (for example in `rust-lang/rust`), someone with write access can close it for you.

Another useful thing to do is to go through `E-needs-bisect` issues and create minimizations or bisecting the issue. When you provide one, you can also remove the label `needs-bisect`.

¹ The `O` in `O-*` labels originally stood for *operating system*, but now it stood for *importance*. This makes the most sense for the `O` labels however it makes sense to interpret the `I` as *issue experience*.

Triaging Crater Runs

Running crater

We regularly run Crater runs, and this documents the process. This may also be applicable to non-release team runs (e.g. for documentation modifications).

First, file a new issue titled “Crater runs for 1.x” ([example](#)).

A crater run for beta should be started as soon as we have a new craterbot invocations.

\$BETA_VERSION is e.g. 1.40.0-1, increment the 1 if it is a new run. You can also use the auto-incremented counter on the beta channel.

\$STABLE is e.g. 1.39.0 (the stable release) \$BETA is beta. You can be looking at <https://static.rust-lang.org/manifests.txt> and <https://static.rust-lang.org/channel-rust-beta.toml>.

```
@craterbot run name=beta-$BETA_VERSION start=beta-$BETA_VERSION  
and-test cap-lints=warn p=10  
@craterbot run name=beta-rustdoc-$BETA_VERSION start=beta-$BETA_VERSION  
mode=rustdoc cap-lints=warn p=5
```

Once the runs complete, you want to triage them.

Triaging

These steps should generally be done for the normal triage of the rustdoc run. Ignore failures in rustdoc (e.g. duplicate failures).

There will usually be quite a few regressions – there are ways to reduce the amount of work that you need to do. It's a preference which is more helpful.

- <https://github.com/Mark-Simulacrum/crater-g>
 - This groups regressions by 'root' by parsing failed messages printed by Cargo
- <https://github.com/Centril/crater-cat-errors>
 - This groups regressions by the "error" message

If you've written a tool, feel free to add it here! We're open to this.

Regardless of the tool you've run, you ultimately need to and try to quickly determine if they're genuine failures. A compiler failure is genuine, and test failures are more of some level of guessing.

Once you've determined that something is a genuine failure (local file, HackMD, whatever) with the error "category" things such that the regressions in a single group are from the same commits, and different groups have different causes.

Once this is done, and you have all the regressions together, you want to file a new issue for each group. It should have the `beta` and `T-compiler` label by default, possibly `T-test` regression, but that's relatively rare. If you happen to find the failure, cc the PR author in a separate comment. The compiler team will triage the issue soon.

Leave a comment on the original issue with the crater run just opened, ideally with the issue titles as well.

You're done!

Re-running rustc on a crate

For the crates which we're not sure about, you can't run the crate directly (`cratesio-curl` can be helpful). Because you are running arbitrary code locally. It's also fine to file an issue about and let the triage process naturally categorize this for *all* the crates. Once you've triaged a crater run, you have a good sense of what is spurious and what isn't, too.

You can run crater on just a single crate by doing so. Note that this will download several gigabytes (on first running).

```
git clone https://github.com/rust-lang/crater
cd crater
cargo run -- prepare-local
CRATES="crates-io-crate-0.4.0,owner/repository"
cargo run -- define-ex --crate-select=list:$CRATES
beta # Edit the stable version.
cargo run -- run-graph --threads 4
cargo run -- gen-report work/ex/default/
# view report for this crate
```

It's also possible to re-queue a subset of crates onto the queue. Take a look at: <https://gist.github.com/ecstatic-morse/be799bfa4d3b3d6e163fa61a9c30706f>

Determining the root cause of the problem

It's not always apparent why a crate stopped building as part of crater triage – but can be a good followup. Felix's [minimization guide](#) are excellent tools to apply.

Edition Releases

This document gives an overview of how to manage Editions. If you are familiar with what Editions are (see the [Editions](#) page), you can skip this section.

Edition project group

[RFC 3501](#) established that the Leadership Council is the group who takes responsibility for managing the Editions.

Project group leads

It is recommended that the project group have 2-3 leads who can quickly make decisions and actions that will be needed on the timeline.

The other consideration is commitment and time availability. In Rust, Editions have a fixed timeline. This requires a commitment. Ordinarily our system is pretty tolerant of people coming and going. Unexpected delays. There is less room for that with Editions. You commit to meeting regularly and following up on action items. (Of course, things happen, people take vacations, whatever. But you get the idea.)

Another thing to consider is that the time requirement is high. There may be some months where there is a fairly high time requirement (10+ hours in a week) depending on how many changes are in an Edition.

Project group members

Additional members of the Edition Project Group can be added to help with specific aspects of the process (such as implementing migration lints, fixing bugs, authoring documentation, etc.).

Phases

Running an Edition involves many steps, coordinate

1. Preparation phase. This is the time approxima
ships that involves all the preparation work. Th
performed, the better.
 1. Preliminary support for the next edition s
nice to make this automated in the futur
 - `rustc` support for `--edition` fla
 - `cargo` support for `edition` setting
 - `rustfmt` support for `--edition` fla
 - `clippy` support for `--edition` fla
 - `rust-analyzer` support for edition
 - `mdbook` support for editions
 2. Teams start their proposals and impleme
2. The Leadership Council sets up a Project Grou
year before the final release).
3. Final deadline phase. This is the period startin
release. This starts the series of final deadlines
Edition. See [Sample timeline](#) below for the set
year.

Feature phases

Each feature goes through a series of phases. This p
process can take a highly variable length of time, so
sometimes taking many years.

1. Individuals and teams propose an Edition char
team, but a common way to start is to post a P
2. An RFC should be posted with the proposed E
3. The team accepts the RFC. This indicates that t
but does not guarantee that it will make it in ti
4. Either the RFC or the team should put togethe
migrations will be handled from the previous e
breakage to require people to make manual e
rare, and ideally it should be noisy (i.e., people

surprising semantics at runtime). **It's up to the implementer to decide what is "rare enough".**

5. Implement the feature and migration support.
6. Informal testing should happen on nightly by the end of the month. Issues should be identified and fixed during the month.
7. The team responsible for the feature should make a final decision for the next edition by the feature cutoff date. This decision should be made with the Edition Project Group.
8. Document the change in places such as the [Edition Project Group](#).

Sample timeline

The following is a sample set of deadlines of the Edition Project Group.

This is shown as milestones relative to the release. The release is stable late in the year (October), but it is highly encouraged to release earlier in the year, such as June.

These dates are not very fixed (for example, Rust release date shuffles around), and the Edition Project Group can adjust as desired.

- 1-3 years before the Edition release date
 - Teams should be planning and implementing the feature.
- T-11 months
 - Leadership Council ensures the Edition Project Group has a plan.
 - Blog post announcing the Edition schedule.
 - Edition Project Group should start coordinating changes, and set up a tracking tool to track progress.
 - Tools should have preliminary support for the feature (to be done soon after the previous Edition).
 - Public blog post calling for the final list of changes. [Example](#)
- T-10 months
 - Last chance for Pre-RFC proposals.
- T-9 months
 - Last chance for RFC approvals.
- T-8 months
 - Final list of Edition changes is complete, and the Edition Project Group is ready to start the next Edition.

- Public blog post informing what is included
- T-7 months
- T-6 months
- T-5 months
 - All features and migrations implemented to be removed.
- T-4 months
 - Crater test all migrations (see [Crater migration](#))
 - Edition Project Group should be tracking progress resolved in time.
- T-3 months
 - Public blog post calling for final testing of
- T-2 months
 - Most issues have been fixed.
 - Documentation finished (the [Edition Guide](#))
 - The Edition Project Group should make a decision on the Edition, or if it has to be delayed to the next
 - Edition is stabilized on nightly for all tools
- T-1 months
 - Edition reaches beta, last chance for any
 - Work with Release team to prepare release
- T-0 months
 - Edition is released on stable.

Crater migration test

Crater does not directly support testing migration like `cargo migrate`. A modified version of `cargo` must be used which will migrate a crate. [#87190](#) contains an example of what it performs the following steps:

1. Crater runs `cargo check` using the previous version
2. Crater runs `cargo check` using the modified version. It will perform the following steps instead of doing a build:
 3. Copies the package to a temp directory (since `cargo` uses `crater`).
 4. Checks if the package's edition is older than the current

- we only want to test migration of the current edition.
5. Runs `cargo fix --edition --allow-no-vcs`.
 6. Modifies `Cargo.toml` to set the new edition.
 7. Runs `cargo check` with a special environment where `cargo check` can run.

The modified cargo also allows setting the new edition to be used.

If the final `cargo fix` or `cargo check` steps fail, and if it's not a previous master build, then that signals a regression.

The Edition Project Group is then responsible for analyzing the failure for any problems, and following up with teams for guidance.

This process may need to be repeated several times until the build can be re-tested.

Beware that the process of running this and analyzing the results, depending on how many changes are in the Edition, can take a month, which involved analyzing hundreds of regressions and re-running crater after fixes had been implemented.

Blog posts and announcements

It is highly encouraged for the Edition Project Group to communicate with everyone early and frequently. Most announcements should be made on the Rust blog. Inside Rust blog posts (usually monthly) with updates about the overall progress and any issues.

Partly in public messaging, but ideally everywhere, **they work (no code breaks! no ecosystem split!)**. If I recall, some reporters reached out to the Rust Foundation. The Edition Project Group should coordinate with the Foundation to ensure they will get questions.

Tracking tools

The Edition Project Group should decide which tools track the progress of the edition. Individual teams will also like to know what is best for them. In the past, we have used a mix of different tools: GitHub issue labels, Google Sheets, HackMd, etc. Use whatever you like, but just keep in mind that it should be publicly accessible.

Examples:

- [2018 Tracking Issue](#)
- [2018 GitHub Project](#)
- [2021 Tracking Spreadsheet](#)
- [2021 GitHub Project](#)
- [2021 HackMd](#)
- [2021 Tracking Issue](#)
- [2024 Tracking Issue](#)
- [2024 HackMd](#)

Examples of individual team tracking:

- [Lang team 2024 roadmap](#)
- [Lang team 2024 planning GitHub Project](#)
- [Lang team issues label](#)
- [Cargo 2024 planning GitHub Project](#)

Implementation notes

Individual teams and projects have their own resources and may make changes. The following are some links for additional information on how Edition changes are implemented.

- [How migrations work](#) if you need a refresher on how to migrate.
- The rustc-dev-guide has an [Editions chapter](#) with information on how to implement edition-specific changes in rustc.
- The Rust Style Guide has information on [Rust edition changes](#) across editions.

Historical context

- [RFC 2052](#) started the Edition system, and kicked off the 2018 Edition.
- [RFC 3085](#) set the plan for the 2021 Edition, as well as the meaning of an Edition.
- [RFC 3501](#) kicked off the 2024 Edition, as well as establishing the process for managing future Editions.

Archive

This section is for content that has become outdated to be read for historical/archival reasons.

Friends of the Tree

The Rust Team likes to occasionally recognize people for their contributions to The Rust Project, its ecosystem, and its community. 'Friends of the Tree', archived here for eternal glory.

2016-02-26 @mitaa

This week we would like to nominate @mitaa as Friend of the Tree. He [sent a wave of fixes to rustdoc](#) (yes those are all separate PRs, but in one way! Rustdoc has historically been a tool in need of love and bugs is especially appreciated. Thanks @mitaa!

2016-02-12 Jeffrey Seyfried (@jsefried)

This week's friend of the tree is Jeffrey Seyfried (@jsefried)

Jeffrey Seyfried (@jsefried) has made some awesome contributions to the compiler. He has fixed a ton of bugs, reported previously unknown, and done some refactorings, all of which have helped improve a corner of the compiler.

2015-12-04 Vadim Petrochenkov

This week we'd like to nominate @petrochenkov for doing some absolutely amazing compiler work recently. He [fixed hygiene bugs](#), [fixing pattern bugs](#), [paving the way](#) for [fixing and closing many privacy holes](#), [refactoring](#) and [old type ascription PR](#). The list of outstanding bugs is growing ever smaller now; thanks @petrochenkov!

2015-11-16 Peter Atashian (WindowsBunny)

In his own words, WindowsBunny is “a hopping encyclopedia of Windows errors and how to solve them.” One of the most helpful people in the Windows community, he actively pushes the frontiers of what Rust can do on Windows, notably the maintainer of the [winapi](#) family of crates that wrap the Windows system APIs. You da bunny, WindowsBunny!

[Source.](#)

2015-10-31 Marcus Klaas

Today @nrc would like to nominate @marcusklaas

Marcus is one of the primary authors of [rustfmt](#). He has been working on rustfmt for many days and is now the [top contributor](#). He has fixed many issues, added many features, reviewed a tonne of PRs, and contributed to the development of rustc. It would not be the software it is today without his hard work on the rustfmt project.

2015-10-16 Ryan Prichard

nmatsakis would also like to declare Ryan Prichard a Rust hero. In the last few months, Ryan has been comparing the Rust compiler to the rust-grammar project, which aims to create a LALR(1) parser for Rust. He has found a number of inconsistencies and bugs between the two, which is useful for two reasons: it finds bugs, obviously, which helps improve the compiler. Second, it helps pave the way for a true Rust reference compiler source itself. So Ryan Prichard, thanks!

2015-10-02 Vikrant Chaudhary

[Vikrant Chaudhary \(nasa42\)](#) is an individual who belongs to the Rust community. In June he has been contributing to [This Week in Rust](#),

and stirring up contributions. He recently rolled out [V](#) brings it more inline with the main website. Today [V](#) weekly newsletter, assisted by [llogiq](#) and other contributors running, Vikrant, you friend of the tree.

[Source.](#)

2015-07-24 Tshepang Lekhonkhobo

[@Gankra](#) has nominated [@tshepang](#) for Friend of the Tree.

Over the last year Tshepang has landed over 100 improvements. Tshepang saw where documentation was not, and saw where it was not.

We should all endeavor to care about docs as much as we care about code.

[Source.](#)

2015-05-19 Chris Morgan

I'd like to nominate Chris Morgan (@chris-morgan) for Friend of the Tree. He recently redesigned the [play.rust-lang.org](#) site for the modern and rustic feel to it. Chris has been contributing to the Rust ecosystem since his first contribution dating back to July 2013 and all the way to the space of HTTP libraries written in Rust. Chris truly is a friend of the tree.

2015-03-24 Andrew Gallant (BurntSushi)

[BurntSushi](#) is an individual who practically needs no introduction. He has created some of the world's most popular crates, including [docopt.rs](#), [byteorder](#). Don't forget his CSV swiss-army-knife, [xs](#). His early work on libraries helped inform the evolution of Rust development, and BurntSushi continues to churn out improvements. He comes from someone who is a skilled friend of the tree.

2015-03-03 Manish Goregaokar (manish)

Manish started working on Servo as part of the GSoC 2014. He implemented XMLHttpRequest. Since then he's been working on Servo while finishing his university studies and organizing Servo events. He took an interest in bors' queue and started making PRs to the integration process. Nursing the PR queue is the kind of thing that creates friends of the tree like Manish, the rollup friend.

2015-02-17 Toby Scrace

Today I would like to nominate Toby Scrace as Friend of the Rust community. The weekend about a login vulnerability on crates.io where the previously logged in user was regardless of whether the login was successful or not. I very much appreciate Toby emailing me about this. I definitely feel that Toby has earned becoming Friend of the Rust community.

2015-02-10 Jonathan Reem (reem)

Jonathan Reem has been making an impact on Rust. His contribution has been as the main author of the project. He has also created several other popular projects including [stainless](#). His practical experience with these projects has helped him upstream rust, most notably his complete rewrite of the project. He does everything he can to advance the Rust cause.

2015-01-20 Barosl Lee (barosl)

Today I would like to nominate Barosl Lee (@barosl) as Friend of the Rust community. He recently rewritten our bors cron job in a new project. The new project has several benefits including:

- Zero "down time" between testing different PRs
- A new rollup button to create separate rollup PRs

- Multiple repositories are supported (Cargo and

Homu was recently deployed for rust-lang/rust than out by Barosl, and it's been working fantastically so responsive to any new issues cropping up. Barosl tr

2015-01-13 Kang Seonghoon (liftl

Seonghoon has been an active member of the Rust although he has made a number of valuable contrib has been in developing key libraries out of tree. [rust](#) crates in Cargo, performs character encoding, and [r](#) of which fill critical holes in the functionality of the s prototype of efficient numerical string conversions t in the standard library. He maintains a [blog](#) where h

2015-01-06 Jorge Aparicio (japari

I nominate Jorge Aparicio (japari) for Friend of the T japari has done tremendous work porting the code features that are now available. First, he converted advantage of DST after it landed. Next, he converted he converted a large portion of the libraries to use a boxed closures from the compiler entirely. He has a the overloaded operators and comparison traits, inc impact on the standard library. And this list exclude deprecating older syntax. The alpha release would r is simply one of the best friends the tree has ever h

2014-12-30 Kevin Ballard (kballa

This is a belated recognition of Kevin Ballard (aka @ tree. Kevin put a lot of work into Unicode issues in R specific constraints. He wrote the current path mod

constraints, and participated in the recent redesign as a dedicated and watchful reviewer. Thanks, Kevin, for

2014-12-16 Gábor Lehel (glaebho)

Gabor's major contributions to Rust have been in the last year he has produced a number of very high quality PRs, not yet been accepted, his ideas are often thought-provoking and influence on the direction of the language. His [trait](#) particularly innovative, as well that [for future-proofing](#) exceedingly clever Friend of the Tree.

2014-11-11 Brian Koropoff (unwound)

In the last few weeks, he has fixed many, many tricky bugs, particularly in the area of unboxed closures and the `Fn` trait, completely rewritten how unboxed closures interact with `move`, huge impact on making them usable. Brian Koropoff

2014-10-07 Alexis Beingessner (Gankra)

Alexis Beingessner (aka @Gankra) began contributing to Rust, having a major impact on several library-related areas. Her PRs completely rewrote `BTree`, providing a vastly more efficient implementation. She proposed and implemented the `collections` crate, extensive new documentation for the `collections` crate reform.

And she added collapse-all to rustdoc!

Alexis is, without a doubt, a FOTT.

2014-09-02 Jorge Aparicio (japari)

Jorge has made several high-impact contributions to Rust, including being the primary author of `rustbyexample.com`, and last week's `rustlings` comparison of language performance on project Euler. Rust is performing quite well. As part of his benchmarking work, he is developing a benchmarking framework to Rust.

2014-07-29 Björn Steinbrink (dot)

Contributing since April 2013. Björn has done many optimizations, including removing allocation bloat in iterators, `fmt`, and many others; adding strategic inlining in the libraries; speeding up `Vec`; and eliminating quadratic blowup in translation, and other things.

He's really done an amazing number of optimizations.

Most recently he earned huge kudos by teaching LLVM to use `asm!()`, allowing Rust to make much more efficient use of the CPU.

Björn is a total FOTT.

2014-07-22 Jonas Hietala (treeman)

Jonas Hietala, aka @treeman, has been contributing to Rust examples recently for modules such as `hashmap`, `tree`, `bigint`, and `vec`. He has also additionally been fixing bugs related to `format!`.

Jonas continues to add new examples/documentation to make Rust more approachable and understandable for all new users.

2014-07-08 Sven Nilson (bvssvni,

Sven Nilson has done a great deal of work to build up with the well-regarded rust-empty project that provides and - crucially - integrates well with other tools like C

His Piston project is one of the most promising Rust number of crates, stressing Rust's tooling at just the learning how to support large-scale external project

Sven is a friend of the tree.

2014-06-24 Jakub Wieczorek (jakub-)

jakub-, otherwise known as Jakub Wieczorek, has recently improve and fix lots of match-related functionality, a venture! Most of this code appears to be untouched receiving some well-deserved love now.

Jakub has fixed 10 bugs this month alone, many of v problems in the compiler. He has also been very res triaging issues that come up from fun match asserti

Jakub truly is a friend of the tree!

2014-04-22 klutzy

klutzy has been doing an amazing amount of Windo issues that affect our quality on Windows and picks doesn't get a ton of thanks, but is hugely appreciate community, he has also done a lot of work for the lo the tree. Thank you!

- Rust on Windows crusader
- Fixed issues with x86 C ABI struct arguments
- Fixed multiple issues with non-US locales

2014-03-18 Clark Gaebel (cgaebe

This week's friend of the tree is Clark Gaebel. He just joined Rust. He dove in and made our hashmaps significantly faster. Hood hashing. He is an excellent friend of the tree.

2014-02-25 Erick Tryzelaar (erick

- Contributing since May 2011
- Wrote the serialization crate
- Organizes the bay area Rust meetups
- Just rewrote the Hash trait

2014-02-11 Flavio Percoco (FlaPe

- Contributing since September
- Does issue triage
- Organizing community events in Italy
- Optimized the 'pow' function
- Recently been fixing lots of small but important bugs

2014-01-27 - Jeff Olson (olsonjeff

- Contributing since February 2012
- Did the original libuv integration
- Implemented our second attempt at I/O, first using libuv
- Ported parts of the C++ runtime to Rust
- Implemented file I/O for the newest runtime
- Last week published an article about file I/O on the Rust blog

2014-01-21 - Steven Fackler (sfackl

- Contributing since last May
- CMU grad
- Lots of library improvements, Base64, Bitv, I/O
- Rustdoc improvements
- Mut/RefCell
- `std::io::util`
- external module loading

2014-01-14 - Eduard Burtescu (ed)

- Contributing since October
- Working on the compiler, including trans
- Reduced rustc memory usage
- Optimized vector operations
- Helping refactor the compiler to eliminate use
- Cleaned up ancient code in the compiler
- Removed our long-standing incorrect use of the `self` param

2014-01-07 - Vadim Chugunov (va)

- Contributing since June
- Fixed numerous bugs on Windows
- Fixing broken tests
- Improved compatibility with newer mingw versions
- Eliminated our runtime C++ dependency by improving `libunwind`

Rust Release history

This is an archive of Rust release artifacts from 0.1–
Rust [GPG signing key](#) (older key, even older key).

1.7.0

- [Announcement](#)
- [Release notes](#)
- [Source code](#) ([signature](#))
- [Windows x86_64 .exe gnu installer](#) ([signature](#))
- [Windows x86_64 .msi gnu installer](#) ([signature](#))
- [Windows x86_64 .exe MSVC installer](#) ([signature](#))
- [Windows x86_64 .msi MSVC installer](#) ([signature](#))
- [Windows i686 .exe gnu installer](#) ([signature](#))
- [Windows i686 .msi gnu installer](#) ([signature](#))
- [Windows i686 .exe MSVC installer](#) ([signature](#))
- [Windows i686 .msi MSVC installer](#) ([signature](#))
- [Linux x86_64 tarball](#) ([signature](#))
- [Linux i686 tarball](#) ([signature](#))
- [Mac OS X i686 pkg](#) ([signature](#))
- [Mac OS X i686 tarball](#) ([signature](#))
- [Mac OS X x86_64 pkg](#) ([signature](#))

1.6.0

- [Announcement](#)
- [Release notes](#)

1.5.0

- [Announcement](#)
- [Release notes](#)

1.4.0

- [Announcement](#)
- [Release notes](#)

1.3.0

- [Announcement](#)
- [Release notes](#)

1.2.0

- [Announcement](#)
- [Release notes](#)

1.1.0

- [Announcement](#)
- [Release notes](#)

1.0.0

- [Announcement](#)
- [Release notes](#)

1.0.0-beta

- [Announcement](#)

1.0.0-alpha.2

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows x86_64 .exe installer \(signature\)](#)
- [Windows i686 .exe installer \(signature\)](#)
- [Windows x86_64 .msi installer \(signature\)](#)
- [Windows i686 .msi installer \(signature\)](#)
- [Linux x86_64 tarball \(signature\)](#)
- [Linux i686 tarball \(signature\)](#)
- [Mac OS X x86_64 pkg \(signature\)](#)
- [Mac OS X i686 pkg \(signature\)](#)
- [Mac OS X x86_64 tarball \(signature\)](#)
- [Mac OS X i686 tarball \(signature\)](#)
- [Documentation](#)

1.0.0-alpha

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows x86_64 installer \(signature\)](#)
- [Windows i686 installer \(signature\)](#)
- [Linux x86_64 tarball \(signature\)](#)
- [Linux i686 tarball \(signature\)](#)
- [Mac OS X x86_64 pkg \(signature\)](#)
- [Mac OS X i686 pkg \(signature\)](#)
- [Mac OS X x86_64 tarball \(signature\)](#)
- [Mac OS X i686 tarball \(signature\)](#)
- [Documentation](#)

Rust 0.x

In addition to the included short-form release in the [release notes](#), there is a longer explanation in the release notes.

0.12.0

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows x86_64 installer \(signature\)](#)
- [Windows i686 installer \(signature\)](#)
- [Linux x86_64 tarball \(signature\)](#)
- [Linux i686 tarball \(signature\)](#)
- [Mac OS X x86_64 pkg \(signature\)](#)
- [Mac OS X i686 pkg \(signature\)](#)
- [Mac OS X x86_64 tarball \(signature\)](#)
- [Mac OS X i686 tarball \(signature\)](#)
- [Documentation](#)

0.11.0

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Linux x86_64 tarball \(signature\)](#)
- [Linux i686 tarball \(signature\)](#)
- [Mac OS X x86_64 pkg \(signature\)](#)
- [Mac OS X i686 pkg \(signature\)](#)
- [Mac OS X x86_64 tarball \(signature\)](#)
- [Mac OS X i686 tarball \(signature\)](#)
- [Documentation](#)

0.10

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)

- [Linux x86_64 tarball \(signature\)](#)
- [Linux i686 tarball \(signature\)](#)
- [Mac OS X x86_64 pkg \(signature\)](#)
- [Mac OS X i686 pkg \(signature\)](#)
- [Mac OS X x86_64 tarball \(signature\)](#)
- [Mac OS X i686 tarball \(signature\)](#)
- [Documentation](#)

0.9

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Documentation](#)

0.8

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
 - [borrowed pointers](#) | [conditions](#) | [containers](#)
- [Manual \(PDF\)](#)
- [Rustpkg manual](#)
- [Standard library docs](#)
- [Extra library docs](#)

0.7

- [Announcement](#)
- [Release notes](#)

- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
- [Manual \(PDF\)](#)
- [Standard library docs](#)
- [Extra library docs](#)

0.6

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
- [Manual \(PDF\)](#)
- [Core library docs](#)
- [Standard library docs](#)

0.5

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
- [Manual \(PDF\)](#)
- [Core library docs](#)
- [Standard library docs](#)

0.4

- [Announcement](#)
- [Release notes](#)

- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
- [Manual \(PDF\)](#)
- [Core library docs](#)
- [Standard library docs](#)

0.3.1

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)

This was an OS X bugfix release.

0.3

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)
- [Tutorial](#)
- [Manual \(PDF\)](#)
- [Core library docs](#)
- [Standard library docs](#)

0.2

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)
- [Windows installer \(signature\)](#)

0.1

- [Announcement](#)
- [Release notes](#)
- [Source code \(signature\)](#)